

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

NGUYỄN THANH PHONG - TRẦN THANH HÒA
VÕ TẤN PHÁT - LÊ ANH VINH

AN TOÀN VÀ PHỤC HỒI DỮ LIỆU ẢNH
TRÊN ĐIỆN THOẠI ANDROID

THỰC TẬP DỰ ÁN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHẤT LƯỢNG CAO

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 08/2024

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

NGUYỄN THANH PHONG	20127274
TRẦN THANH HÒA	20127163
VÕ TẤN PHÁT	20127589
LÊ ANH VINH	20127390

AN TOÀN VÀ PHỤC HỒI DỮ LIỆU ẢNH
TRÊN ĐIỆN THOẠI ANDROID

THỰC TẬP DỰ ÁN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHẤT LƯỢNG CAO

GIÁO VIÊN HƯỚNG DẪN
Th.S THÁI HÙNG VĂN

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 08/2024

LỜI CẢM ƠN

Sau hơn sáu tháng tìm hiểu và thực hiện đề tài “An toàn và phục hồi dữ liệu hình ảnh trên điện thoại Android”, đề tài của nhóm em đã cơ bản hoàn thành. Để đạt được kết quả này, ngoài sự nỗ lực của bản thân cả nhóm, nhóm còn nhận được rất nhiều sự giúp đỡ và hỗ trợ nhiệt tình từ nhiều cá nhân và tổ chức.

Trước hết, nhóm em xin gửi lời cảm ơn chân thành đến bộ môn Công nghệ tri thức, bộ môn Mạng máy tính, khoa Công nghệ thông tin, trường Đại học Khoa học Tự nhiên, đã tạo mọi điều kiện thuận lợi cho nhóm em trong suốt quá trình thực hiện đề tài này. Đặc biệt, nhóm xin bày tỏ lòng biết ơn sâu sắc đến thầy Thái Hùng Văn, người đã tận tình hướng dẫn, hỗ trợ nhóm trong suốt quá trình hoàn thành khóa luận.

Nhóm em xin chân thành cảm ơn các thầy cô trong khoa, những người đã không quản ngại khó khăn, tận tình giảng dạy và trang bị cho em những kiến thức cần thiết để có thể thực hiện thành công đề tài này.

Cuối cùng, xin gửi lời cảm ơn sâu sắc đến gia đình, những người đã luôn luôn động viên, hỗ trợ và tạo điều kiện tốt nhất cho em học tập và hoàn thành khóa luận tốt nghiệp.

Mặc dù khóa luận đã cơ bản hoàn tất, nhưng chắc chắn sẽ không thể tránh khỏi những thiếu sót ngoài ý muốn. Nhóm em rất mong nhận được sự thông cảm và những góp ý quý báu từ mọi người để có thể hoàn thiện hơn.

Tháng 7.2024

Nguyễn Thanh Phong

Trần Thanh Hòa

Lê Anh Vinh

Võ Tấn Phát

MỤC LỤC

LỜI CẢM ƠN	i
MỤC LỤC	ii
DANH SÁCH HÌNH.....	viii
DANH SÁCH CÁC BẢNG	xi
DANH SÁCH CÁC TỪ VIẾT TẮT VÀ THUẬT NGỮ.....	xiii
TÓM TẮT ĐỀ TÀI	xv
Chương 1. MỞ ĐẦU.....	1
1.1 Tính cấp thiết của đề tài.....	1
1.2 Mục tiêu nghiên cứu của đề tài.....	1
1.3 Đối tượng và phạm vi nghiên cứu	2
1.4 Mô hình đề xuất	2
1.5 Đóng góp của đề tài	3
Chương 2. TỔNG QUAN VỀ DỮ LIỆU HÌNH ẢNH TRÊN ĐIỆN THOẠI ANDROID	5
2.1 Dữ liệu hình ảnh và tầm quan trọng	5
2.1.1 Khái niệm về dữ liệu hình ảnh	5
2.1.2 Vai trò của dữ liệu hình ảnh	5
2.2 Tổng quan về hệ điều hành Android	6
2.2.1 Khái niệm hệ điều hành Android	6
2.2.2 Lịch sử và sự phát triển của hệ điều hành Android.....	7
2.2.3 Tầm quan trọng và phổ biến.....	8

2.3	Dữ liệu hình ảnh trên điện thoại Android.....	8
2.3.1	Mức độ phổ dụng và tầm quan trọng của dữ liệu hình ảnh trên điện thoại Android.....	8
2.3.2	Các định dạng hình ảnh phổ biến nhất trên điện thoại Android.....	9
2.3.3	Các mối đe dọa và rủi ro đối với dữ liệu hình ảnh trên điện thoại Android.	9
2.3.4	Các nguyên tắc cơ bản của an toàn dữ liệu.	10
Chương 3.	KIẾN TRÚC TỔ CHỨC BÊN TRONG DỮ LIỆU HÌNH ẢNH	12
3.1	Tổng quan	12
3.1.1	Định nghĩa dữ liệu hình ảnh.....	12
3.1.2	Vai trò của dữ liệu hình ảnh	12
3.1.3	Các loại dữ liệu hình ảnh thường gặp	13
3.1.4	Các định dạng dữ liệu hình ảnh phổ biến.....	14
3.1.5	Các thành phần chính trong tập tin hình ảnh.....	15
3.1.6	Cách thức dữ liệu hình ảnh được lưu trữ.....	16
3.2	EXIF Tổ chức siêu dữ liệu trong tập tin hình ảnh	17
3.2.1	Giới thiệu.....	17
3.2.2	Vai trò.....	17
3.2.3	Cấu trúc	19
3.3	Định dạng tập tin hình ảnh JPEG	21
3.3.1	Giới thiệu.....	21
3.3.2	Cấu trúc file	22
3.4	Định dạng tập tin hình ảnh PNG.....	23

3.4.1	Giới thiệu.....	23
3.4.2	Cấu trúc file	24
Chương 4. TỔNG QUAN HỆ THỐNG QUẢN LÝ TẬP TIN TRÊN HỆ		
ĐIỀU HÀNH ANDROID.....		27
4.1	Cách tổ chức lưu trữ tập tin trên Android.....	27
4.2	Các định dạng hệ thống tập tin trên Android.....	27
4.2.1	Giới thiệu chung	27
4.2.2	Các định dạng hệ thống tập tin trên Android	27
4.2.3	So sánh các hệ thống tập tin trên Android	28
4.3	Giới thiệu định dạng EXT4	28
4.3.1	Giới thiệu chung	28
4.3.2	Lịch sử phát triển.....	28
4.3.3	Đặc điểm kỹ thuật.....	29
4.3.4	Lợi ích của EXT4	30
4.4	Kiến trúc định dạng EXT4.....	30
4.4.1	Superblock.....	30
4.4.2	Block Group Descriptor	34
4.4.3	Block Bitmap.....	39
4.4.4	Inode Bitmap	41
4.4.5	Inode Table.....	41
4.4.6	Inode	41
4.4.7	Directory & Directory entries	52
4.4.8	Journal(jbd2)	55

Chương 5. CÁC PHƯƠNG PHÁP TỔNG QUÁT CHO AN TOÀN VÀ PHỤC HỒI DỮ LIỆU HÌNH ẢNH TRÊN ĐIỆN THOẠI ANDROID.....60

5.1	Tổng quan về an toàn và phục hồi dữ liệu.....	60
5.2	Các phương pháp tổng quát cho an toàn dữ liệu hình ảnh	60
5.2.1	Sao lưu tập tin hình ảnh.....	61
5.2.2	Bảo vệ bằng mật khẩu và sinh trắc học.....	67
5.2.3	Loại bỏ các thông tin nhạy cảm hoặc bất thường trong tập tin hình ảnh	68
5.2.4	Mã hóa tập tin hình ảnh.....	73
5.2.5	Ẩn giấu tập tin hình ảnh	74
5.3	Phục hồi dữ liệu hình ảnh trên Android.....	76
5.3.1	Phục hồi từ sao lưu	76
5.3.2	Sử dụng phần mềm phục hồi dữ liệu có sẵn	76

Chương 6. XÂY DỰNG HỆ THỐNG QUẢN LÝ TẬP TIN CHUYÊN BIỆT PHỤC VỤ CHO AN TOÀN VÀ PHỤC HỒI DỮ LIỆU HÌNH ẢNH78

6.1	Giới thiệu	78
6.2	Phân tích yêu cầu	78
6.2.1	Yêu cầu kỹ thuật.....	78
6.2.2	Yêu cầu phi kỹ thuật.....	79
6.3	Thiết kế hệ thống tập tin chuyên biệt	79
6.3.1	Mô hình quản lý tập tin	79
6.3.2	Sơ đồ hoạt động.....	83
6.3.3	Mô hình chức năng chính.....	84
6.3.4	Mô hình chức năng phụ.....	88

Chương 7.	CÁC GIẢI PHÁP MỞ RỘNG.....	90
7.1	Thiết kế các phương pháp an toàn dữ liệu hình ảnh	90
7.1.1	Sao lưu, mã hóa và ẩn dữ liệu dưới hình thức đặc biệt	90
7.1.2	Xóa thông tin siêu dữ liệu trong tập tin hình ảnh.....	90
7.1.3	Nhúng và ẩn thông tin vào dữ liệu hình ảnh	90
7.1.4	Làm “sạch” ảnh bằng kỹ thuật CDR được thiết kế riêng.....	90
7.2	Thiết kế các phương pháp phục hồi dữ liệu hình ảnh.....	91
7.2.1	Phục hồi trên hệ thống tập tin chuyên biệt	91
7.2.2	Phục hồi ảnh trên điện thoại Android thông qua thư mục Emulated	91
7.2.3	Truy xuất và phục hồi dữ liệu trực tiếp trên hệ thống tập tin.....	92
Chương 8.	XÂY DỰNG ỨNG DỤNG	102
8.1	Ứng dụng chạy trên Windows	102
8.1.1	Sơ đồ hoạt động.....	102
8.1.2	Các tính năng của ứng dụng	102
8.1.3	Các kết quả đạt được	108
8.1.4	Giao diện ứng dụng	109
8.1.5	Các khó khăn hiện hữu.....	111
8.1.6	Hướng phát triển.....	111
8.2	Ứng dụng chạy trên Android	112
8.2.1	Thiết kế giao diện	113
8.2.2	Sơ đồ hoạt động.....	114
8.2.3	Các tính năng của ứng dụng	115
8.2.4	Các kết quả đạt được	119

8.2.5	Giao diện ứng dụng	120
8.2.6	Các khó khăn hiện hữu	122
8.2.7	Hướng phát triển.....	122
Chương 9.	TỔNG KẾT	124
9.1	Thực nghiệm an toàn dữ liệu	124
9.1.1	Thiết lập các biện pháp bảo vệ trên Android	124
9.1.2	Đánh giá hiệu quả của các biện pháp bảo vệ.....	125
9.2	Thực nghiệm phục hồi dữ liệu.....	125
9.3	Thử nghiệm các tính năng	126
9.3.1	Ứng dụng Android.....	126
9.4	Tổng kết các kết quả nghiên cứu	128
9.5	Đánh giá và so sánh	129
PHỤ LỤC		131
TÀI LIỆU THAM KHẢO		134

DANH SÁCH HÌNH

Hình 3.1	Cấu trúc của APP1[3]	19
Hình 3.2	Cấu trúc IFD [3]	20
Hình 3.3	Bảng chú thích kiểu dữ liệu [3]	20
Hình 3.4	Cấu trúc cơ bản của tập tin nén JPEG [5].....	22
Hình 4.1	Mô hình cấu trúc EXT4 [12]	30
Hình 4.2	Bản mẫu Superblock của EXT4	33
Hình 4.3	Bản mẫu Block Group Descriptor 64-bytes của EXT4	36
Hình 4.4	Bản mẫu Block Group Descriptor 64-bytes	36
Hình 4.5	Bản mẫu Block Group Descriptor 32-bytes của EXT4	38
Hình 4.6	Bản mẫu Block Group Descriptor 32-bytes của Block Group đầu tiên	38
Hình 4.7	Bản mẫu Block Bitmap của EXT4	40
Hình 4.8	Bản mẫu I_block trong Inode	50
Hình 4.9	Bản mẫu Directory của EXT4	54
Hình 4.10	Mô hình cấu trúc journal của EXT4	56
Hình 4.11	Mô hình cấu trúc của một journal descriptor Block	57
Hình 5.1	Mô hình áp dụng hard link trong việc phục hồi dữ liệu	66
Hình 6.1	Mô hình cấu trúc chung của hệ thống quản lý tập tin	79
Hình 6.2	Sơ đồ hoạt động của hệ thống quản lý tập tin PPHV	83
Hình 7.1	Bản mẫu Superblock của EXT4(2).....	93
Hình 7.2	Bản mẫu Block Group Descriptor của EXT4 (2)	94
Hình 7.3	Bản mẫu Inode Table của EXT4	95

Hình 7.4	Bản mẫu root Directory của EXT4.....	96
Hình 7.5	Bản mẫu Inode do file “yuta.jpeg” chỉ tới.....	97
Hình 7.6	Dữ liệu do Inode của file “yuta.jpeg” chỉ tới.....	97
Hình 7.7	Dữ liệu directory sau khi xoá file “yuta.jpeg” và “gojo1-2.jpeg” bị xoá	98
Hình 7.8	Dữ liệu inode của file “yuta.jpeg” sau khi file yuta.jpeg đã bị xoá.	98
Hình 7.9	Dữ liệu Journal Superblock của EXT4.....	99
Hình 7.10	Dữ liệu Journal Descriptor Block của Transaction đầu tiên.....	99
Hình 7.11	Dữ liệu Journal Descriptor Block cần tìm	100
Hình 7.12	Dữ liệu Inode cần phục hồi tìm được trong Journal	100
Hình 7.13	Dữ liệu của file cần phục hồi.....	101
Hình 7.14	Chữ ký bắt đầu và kết thúc của JPEG	101
Hình 7.15	Chữ ký bắt đầu và kết thúc của PNG.....	101
Hình 8.1	Sơ đồ hoạt động của ứng dụng chạy trên Windows	102
Hình 8.2	Menu chính cho ứng dụng windows.....	109
Hình 8.3	Chức năng xuất cây thư mục	109
Hình 8.4	Chức năng truy xuất file theo tên	109
Hình 8.5	Chức năng phục hồi ảnh PNG, JPEG trên các loại hệ thống	110
Hình 8.6	Chức năng trích xuất vị trí cụ thể của ảnh(nếu có).....	110
Hình 8.7	Chức năng xoá thông tin nhạy cảm trong dữ liệu hình ảnh.....	110
Hình 8.8	Chức năng tái cấu trúc ảnh	110
Hình 8.9	Chức năng ẩn thông điệp trong ảnh(chỉ PNG)	110
Hình 8.10	Chức năng đọc thông điệp ẩn trong ảnh(chỉ PNG)	111

Hình 8.11	Phục hồi dữ liệu file từ file imgae của EXT4.....	111
Hình 8.12	Giao diện dự kiến của ứng dụng.....	113
Hình 8.13	Flowchart cho ứng dụng Android.....	114
Hình 8.14	Màn hình ngoài của ứng dụng	120
Hình 8.15	Màn hình chính của “Tập tin của bạn”	120
Hình 8.16	Thư viện hiển thị ảnh.....	121
Hình 8.17	Tính năng mã hóa/giải mã	121
Hình 8.18	Tính năng thêm ảnh	121
Hình 8.19	Tính năng xóa ảnh	121
Hình 8.20	Phục hồi ảnh trên điện thoại android.....	122
Hình 8.21	Phục hồi ảnh trên hệ thống PPHV	122

DANH SÁCH CÁC BẢNG

Bảng 4.1	Bảng thông tin dữ liệu Superblock [13]	31
Bảng 4.2	Bảng thông tin dữ liệu thu thập từ bản mẫu Superblock của EXT4	33
Bảng 4.3 [13]	Bảng thông tin dữ liệu Block Group Descriptor 64 bytes của EXT4	35
Bảng 4.4	Bảng thông tin dữ liệu thu thập từ bản mẫu của Block Group Descriptor 64 bytes của EXT4	36
Bảng 4.5	Bảng thông tin dữ liệu Block Group Descriptor 32 bytes của EXT4	38
Bảng 4.6	Bảng thông tin dữ liệu Inode của EXT4 [13]	41
Bảng 4.7	Bảng phân tích giá trị I_mode [13].....	43
Bảng 4.8	Bảng thông tin dữ liệu Extent tree Header [13]	48
Bảng 4.9	Bảng thông tin dữ liệu internal node thuộc Extent tree [13]	49
Bảng 4.10	Bảng thông tin dữ liệu leaf node thuộc Extent tree [13]	50
Bảng 4.11	Bảng thông tin dữ liệu thu thập được từ I_block mẫu.....	50
Bảng 4.12	Bảng thông tin dữ liệu thu thập từ I_block entry mẫu	51
Bảng 4.13	Bảng thông tin dữ liệu Directory entry cơ bản [13]	53
Bảng 4.14 [13]	Bảng thông tin dữ liệu Directory entry nếu tính năng filetype tắt	53
Bảng 4.15	Bảng thông tin dữ liệu thu thập từ Directory entry mẫu	55
Bảng 4.16	Bảng thông tin dữ liệu journal Block Header [13]	56
Bảng 4.17	Bảng thông tin dữ liệu journal descriptor Block [13].....	57
Bảng 4.18	Bảng thông tin dữ liệu journal descriptor Block v3	58

Bảng 6.1	Bảng thông tin dữ liệu hệ thống quản lý tập tin	80
Bảng 6.2	Bảng thông tin dữ liệu phần tử vùng trống.....	81
Bảng 6.3	Bảng thông tin dữ liệu vùng entry table của hệ thống tập tin.....	82
Bảng 7.1	Bảng thông tin dữ liệu thu thập từ Superblock mẫu.....	93
Bảng 7.2	Bảng thông tin dữ liệu thu thập từ Block Group Descriptor mẫu.....	94
Bảng 9.1	Bảng thử nghiệm mã hoá ảnh	124
Bảng 9.2	Bảng thử nghiệm lưu trữ ảnh.....	124
Bảng 9.3	Bảng thử nghiệm phục hồi ảnh.....	125
Bảng 9.4	Bảng kiểm thử tính năng	126
Bảng 9.5	Bảng so sánh với các phần mềm khác	129

DANH SÁCH CÁC TỪ VIẾT TẮT VÀ THUẬT NGỮ

DLHA	Dữ liệu hình ảnh		
LSB	Least Significant Bit		Bit ít quan trọng nhất
NFC	Near	Field	Giao tiếp trường gần
	Communication		
RGB	Red, Green, Blue		Đỏ, Xanh lá, Xanh dương
IFD	Image File Directory		Thư mục tệp hình ảnh
2FA	Two-Factor		Xác thực hai yếu tố
	Authentication		
ECB	Electronic Codebook		Chế độ vận hành của thuật toán mã hóa khối
AES	Advanced	Encryption	Một tiêu chuẩn mã hóa
	Standard		
EXIF	Exchangeable	Image File	Định dạng tập tin hình ảnh
	Format		có thể trao đổi
PNG	Portable	Network	Định dạng tập tin hình ảnh
	Graphics		
CDR	Content	Disarm	Kỹ thuật được sử dụng để bảo vệ chống lại các mối đe dọa từ các tập tin có
	Reconstruction		
		and	

		chứa mã đọc
JPEG	Joint Photographic Experts Group	Định dạng tập tin hình ảnh
exFAT	Extended File Allocation Table	Hệ thống tập tin mở rộng
FAT32	File Allocation Table 32 bits	Hệ thống tập tin với bảng phân bổ dạng 32-bits
OFFSET	OFFSET	Vị trí (tính bằng byte) từ đầu tập tin nơi hoạt động đọc hoặc ghi
Metadata		Các thông tin phụ thêm vào dữ liệu chính của một file ảnh
Pixel	Picture element	Đơn vị cơ bản nhỏ nhất của một hình ảnh kỹ thuật số
Allocation unit		Một đơn vị cơ bản mà hệ thống tập tin sử dụng để quản lý không gian lưu trữ trên các thiết bị lưu trữ

TÓM TẮT ĐỀ TÀI

Đề tài sẽ tập trung vào các mục tiêu chính sau đây: Nghiên cứu và khảo sát để nắm được cách thức lưu trữ dữ liệu trên hệ điều hành Android, chủ yếu là kiến trúc tổ chức hệ thống tập tin Ext4 đang dùng phổ biến nhiều năm qua nhưng lại có khá ít tài liệu mô tả đầy đủ rõ ràng. Ngoài ra, nghiên cứu cấu trúc của các định dạng DLHA thông dụng như .JPG và .PNG, từ đó đưa ra một chương trình ứng dụng có thể lọc bỏ các mã độc hoặc nội dung bất thường trong DLHA. Đề tài cũng sẽ xây dựng hệ thống mã hóa DLHA hiệu quả cao, tạo ra một hệ thống tập tin có độ bảo mật cao tích hợp vào hệ thống tập tin chuyên biệt để chứa các DLHA quý hoặc nhạy cảm cần ẩn giấu. Đồng thời, hệ thống sẽ phục hồi được các DLHA đã bị xóa mất, đảm bảo người dùng có thể khôi phục lại các hình ảnh quan trọng một cách hiệu quả.

Chương 1. MỞ ĐẦU

1.1 Tính cấp thiết của đề tài

Trong cuộc sống hiện đại, dữ liệu hình ảnh ngày càng trở nên quan trọng và thiết yếu. Hình ảnh không chỉ đơn thuần là phương tiện ghi lại khoảnh khắc mà còn là một phần quan trọng của giao tiếp, làm việc và lưu giữ kỷ niệm cá nhân. Sự phát triển mạnh mẽ của công nghệ di động, đặc biệt là điện thoại Android, đã làm cho việc chụp, lưu trữ và chia sẻ hình ảnh trở nên dễ dàng và phổ biến hơn bao giờ hết. Tuy nhiên, cùng với sự tiện lợi này là những mối đe dọa nghiêm trọng đối với bảo mật và an toàn của dữ liệu hình ảnh. Điện thoại Android, dù có nhiều tính năng bảo mật, vẫn không tránh khỏi nguy cơ bị tấn công, mất mát hoặc bị lộ dữ liệu cá nhân. Chính vì thế, việc nghiên cứu và phát triển một hệ thống tập tin chuyên biệt, tích hợp mã hóa để bảo vệ an toàn dữ liệu hình ảnh trên điện thoại Android là hết sức cần thiết.

Đề tài đặt mục đích thực hiện cả hai nhánh xử lý lớn nêu trên trên thiết bị được sử dụng rộng rãi hàng đầu thế giới: các điện thoại di động chạy hệ điều hành Android. Vì lý do trước mắt là dữ liệu hình ảnh (DLHA) thường được lưu trữ rất nhiều trên điện thoại, trong đó có thể có khá nhiều DLHA quý hoặc nhạy cảm nhưng chưa có nhiều công cụ hỗ trợ bảo vệ đầy đủ, hiệu quả hoặc đáng tin cậy. Đề tài sẽ xây dựng một bộ công cụ với nhiều chức năng cho phép người dùng thực hiện được nhiều hình thức bảo vệ, bảo mật hoặc phục hồi DLHA trên nhiều giải pháp để có thể đáp ứng được nhiều nhu cầu khác nhau trên thực tế.

1.2 Mục tiêu nghiên cứu của đề tài

Các mục tiêu cụ thể của đề tài bao gồm: Nghiên cứu và khảo sát để nắm được cách thức lưu trữ dữ liệu trên hệ điều hành Android (chủ yếu là kiến trúc tổ chức hệ thống tập tin EXT4 đang dùng phổ biến nhiều năm qua nhưng lại có khá ít tài liệu mô tả đầy đủ rõ ràng) cũng như cấu trúc của các định dạng DLHA thông dụng (.JPG và .PNG); từ đó đưa ra một chương trình ứng dụng có thể lọc bỏ các mã độc hoặc nội dung bất thường trong DLHA, xây dựng hệ thống mã hóa DLHA hiệu quả cao, tạo được một hệ thống tập tin có độ bảo mật cao (tích hợp vào hệ thống tập tin hiện hữu)

chứa các DLHA quý hoặc nhạy cảm cần ẩn giấu, đồng thời phục hồi được các DLHA đã bị xóa mất.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là những tập tin hình ảnh và các định dạng phổ hình ảnh biến nhất, cấu trúc lưu trữ tập tin của hệ điều hành Android và các truy xuất trực tiếp nội dung, các phương pháp an toàn và phục hồi dữ liệu hình ảnh

Phạm vi nghiên cứu của đề tài tập trung:

Về cấu trúc lưu trữ của hệ điều hành Android, đề tài chỉ tập trung vào cách thức dữ liệu được quản lý trong hệ thống quản lý tập tin EXT4 từ đó đưa ra cách truy xuất dữ liệu trực tiếp không thông qua hệ điều hành bằng cách đọc dữ liệu sector.

Về cấu trúc lưu trữ dữ liệu bên trong của định dạng tập tin JPEG, PNG, đề tài chỉ nêu ra cấu trúc tổng thể để có thể trích xuất thông tin ảnh mà không thông qua hệ điều hành ví dụ như là thông tin GPS của một tấm ảnh.

Về cách triển khai một công cụ để quản lý và bảo mật DLHA, đề tài thiết kế và đưa ra một hệ thống quản lý tập tin chuyên biệt tích hợp mã hóa AES để có thể giúp người dùng an tâm khi lưu trữ những bức ảnh quý.

1.4 Mô hình đề xuất

Đề tài nghiên cứu này tập trung vào việc thiết kế một hệ thống tập tin chuyên biệt, nhằm tích hợp các tính năng an toàn và phục hồi dữ liệu hình ảnh (DLHA) một cách hiệu quả. Các tính năng này bao gồm sao lưu, ẩn giấu, mã hóa, tái tạo và loại bỏ dữ liệu, đảm bảo việc lưu trữ và truy xuất được thực hiện một cách an toàn và hiệu quả. Hệ thống tập tin được xây dựng dựa trên việc nghiên cứu các yêu cầu đặc thù trong quản lý DLHA, từ đó đề xuất một cấu trúc tối ưu. Đồng thời, đề tài cũng tích hợp các phương pháp mã hóa tiên tiến để bảo vệ dữ liệu, tăng cường tính bảo mật cho người dùng mà không ảnh hưởng đến hiệu suất truy xuất dữ liệu.

Một phần quan trọng của đề tài là phân tích chi tiết cách thức lưu trữ và quản lý dữ liệu trên hệ thống tập tin EXT4. Từ những phân tích này, đề tài đề xuất phương pháp truy xuất dữ liệu trực tiếp mà không thông qua của hệ điều hành. Điều này sẽ giúp ích cho việc phục hồi dữ liệu khi một hệ thống quản lý tập tin bị hư hỏng.

1.5 Đóng góp của đề tài

Đề tài sẽ đưa ra một giải pháp rất hiệu quả cho việc an toàn và phục hồi dữ liệu ảnh trên điện thoại Android bao gồm các tính năng như: Ẩn giấu, bảo mật, sao lưu, tái tạo, loại bỏ,... cho dữ liệu hình ảnh trên điện thoại Android. Từ đó giúp bảo vệ quyền riêng tư cho người dùng cá nhân, bảo vệ hình ảnh và thông tin nhạy cảm, đảm bảo tính toàn vẹn và bảo mật của DLHA, tăng cường uy tín cá nhân và tổ chức, ngăn chặn được các hoạt động tội phạm, hỗ trợ phát triển dịch vụ và cải thiện trải nghiệm người dùng. Các sản phẩm cụ thể của đề tài bao gồm:

Một ứng dụng di động chạy trên hệ điều hành Android được phát triển để bảo vệ và bảo mật dữ liệu hình ảnh. Ứng dụng này sẽ cung cấp các tính năng như mã hóa dữ liệu hình ảnh, tạo ra một hệ thống tập tin với độ bảo mật cao để bảo vệ các DLHA quan trọng, nhạy cảm cho phép người dùng sử dụng được trực tiếp bên trong. Ứng dụng còn cung cấp tính năng Phục hồi lại các DLHA đã bị hư hỏng mất mát vì rất nhiều lý do (mã độc, sự cố trên hệ thống phần cứng, lỗi phần mềm, sơ suất của người dùng,...).

Một ứng dụng phục hồi dữ liệu hình ảnh (DLHA) dành cho hệ điều hành Windows đã được phát triển, nhằm đến việc khôi phục dữ liệu từ các thiết bị lưu trữ như thẻ nhớ ngoài của điện thoại hoặc các thiết bị lưu trữ cũ khác. Ứng dụng này không chỉ hỗ trợ hệ thống quản lý tập tin EXT4, mà còn tương thích với các hệ thống tập tin khác như FAT32, exFAT, và NTFS. Đặc biệt, ứng dụng còn có khả năng xử lý các thiết bị bị hư hỏng logic nghiêm trọng, nơi mà cấu trúc hệ thống tập tin không còn khả dụng để khai thác. Giải pháp phục hồi này cung cấp các công cụ chuyên sâu để phân tích và tái tạo dữ liệu bị mất, đảm bảo khả năng phục hồi tối đa cho người dùng dù trong các tình huống phức tạp nhất.

Các tài liệu liên quan đến việc quản lý và truy xuất dữ liệu trên hệ thống quản lý tập tin EXT4, đặc biệt là đối với dữ liệu hình ảnh, cung cấp các kiến thức và phương pháp cần thiết để hiểu và áp dụng hiệu quả. Ngoài ra, các tài liệu còn tập trung vào việc truy xuất và xử lý metadata của hai loại định dạng hình ảnh phổ biến nhất hiện nay là JPEG và PNG. Những kiến thức này không chỉ bao gồm cách thức lưu trữ và

quản lý dữ liệu mà còn nhấn mạnh vào các kỹ thuật bảo mật và tối ưu hóa, giúp bảo vệ và cải thiện hiệu suất truy xuất dữ liệu hình ảnh

Chương 2. TỔNG QUAN VỀ DỮ LIỆU HÌNH ẢNH TRÊN ĐIỆN THOẠI ANDROID

2.1 Dữ liệu hình ảnh và tầm quan trọng

2.1.1 Khái niệm về dữ liệu hình ảnh

Dữ liệu hình ảnh là dạng dữ liệu số chứa thông tin về hình ảnh được biểu diễn dưới dạng số, có thể thu thập từ nhiều nguồn như điện thoại di động, máy ảnh kỹ thuật số, máy quét hoặc các thiết bị thu nhận hình ảnh khác. Dữ liệu này có thể được lưu trữ, xử lý và hiển thị dưới nhiều dạng khác nhau, nhưng chủ yếu bao gồm các thông tin về màu sắc và độ sáng của từng điểm ảnh (pixel) trong hình ảnh. Trên máy tính mỗi dữ liệu hình ảnh thường được lưu trữ dưới dạng một tập tin có phần tên mở rộng là JPG /JPEG, PNG, BMP...

2.1.2 Vai trò của dữ liệu hình ảnh

Dữ liệu hình ảnh có vai trò vô cùng quan trọng trong cuộc sống hiện đại vì nhiều lý do:

- **Ghi lại khoảnh khắc:** Ảnh chụp giúp ghi lại các khoảnh khắc đáng nhớ trong cuộc sống, từ những sự kiện quan trọng như sinh nhật, lễ cưới, đến các khoảnh khắc đời thường.
- **Truyền tải thông tin:** Hình ảnh là một phương tiện mạnh mẽ để truyền tải thông tin và cảm xúc. Một bức ảnh có thể truyền tải thông điệp mà đôi khi lời nói không thể diễn tả hết.
- **Tương tác xã hội:** Trong thời đại mạng xã hội, hình ảnh đóng vai trò quan trọng trong việc chia sẻ thông tin và tương tác với bạn bè, gia đình qua các nền tảng xã hội như Facebook, Instagram,...

- **Công cụ công việc:** Đối với nhiều ngành nghề, đặc biệt là thiết kế, quảng cáo, và truyền thông, dữ liệu hình ảnh là một trong những công cụ quan trọng nhất trong quá trình làm việc.
- **Bảo mật và nhận dạng:** Công nghệ nhận dạng hình ảnh và nhận diện khuôn mặt sử dụng dữ liệu hình ảnh để tăng cường bảo mật và cá nhân hóa trải nghiệm người dùng.
- **Giáo dục và nghiên cứu:** Hình ảnh đóng vai trò quan trọng trong giáo dục và nghiên cứu khoa học. Chúng giúp minh họa và làm rõ các khái niệm phức tạp, từ hình ảnh giải phẫu trong y học đến hình ảnh vệ tinh trong khoa học môi trường. Việc sử dụng hình ảnh trong giảng dạy và nghiên cứu giúp cải thiện sự hiểu biết và ghi nhớ của học sinh và sinh viên.
- **Nghệ thuật và sáng tạo:** Hình ảnh là một phương tiện nghệ thuật và sáng tạo quan trọng. Nhiếp ảnh, hội họa kỹ thuật số, và thiết kế đồ họa đều dựa vào dữ liệu hình ảnh để tạo ra các tác phẩm nghệ thuật độc đáo và truyền cảm hứng. Sự phổ biến của điện thoại thông minh với camera chất lượng cao đã mở ra cơ hội cho mọi người thể hiện sự sáng tạo của mình thông qua hình ảnh.

Như vậy, dữ liệu hình ảnh không chỉ là những mảnh ký ức kỹ thuật số mà còn là công cụ đa năng trong nhiều khía cạnh của cuộc sống và công việc hàng ngày. Việc quản lý và sử dụng hiệu quả dữ liệu hình ảnh trên điện thoại Android đóng vai trò quan trọng trong việc tận dụng tối đa các lợi ích mà chúng mang lại.

2.2 Tổng quan về hệ điều hành Android

2.2.1 Khái niệm hệ điều hành Android

Android là một hệ điều hành dành cho thiết bị di động được phát triển bởi Google và dựa trên nhân Linux. Ban đầu được thiết kế chủ yếu cho điện thoại thông minh và máy tính bảng, Android đã mở rộng ra các thiết bị khác như đồng hồ thông minh (Wear OS), thiết bị đeo công nghệ cao, thiết bị giải trí trong xe hơi (Android Auto), và hệ thống giải trí tại gia (Android TV). Đây là một nền tảng mã nguồn mở, cho phép các nhà phát triển và nhà sản xuất thiết bị tùy chỉnh và mở rộng theo nhu cầu của họ.

2.2.2 Lịch sử và sự phát triển của hệ điều hành Android.

Hệ điều hành Android có một lịch sử phát triển thú vị và đã trở thành một trong những nền tảng di động hàng đầu thế giới. Dưới đây là các giai đoạn chính trong lịch sử và sự phát triển của Android:

- 2003: Android Inc. được thành lập bởi Andy Rubin, Rich Miner, Nick Sears, và Chris White tại Palo Alto, California. Ban đầu, công ty tập trung vào việc phát triển một hệ điều hành thông minh cho máy ảnh kỹ thuật số.
- 2005: Google mua lại Android Inc. Mục tiêu chuyển sang phát triển một hệ điều hành dành cho điện thoại di động, nhằm cạnh tranh với Symbian và Windows Mobile, và sau này là iOS của Apple.
- 2007: Google công bố nền tảng Android và thành lập Open Handset Alliance (OHA) – một liên minh của các nhà sản xuất phần cứng, phần mềm và viễn thông nhằm tạo ra một chuẩn mở cho thiết bị di động.
- 2008: HTC Dream (còn gọi là T-Mobile G1) ra mắt – điện thoại thông minh đầu tiên chạy Android. Phiên bản Android đầu tiên, Android 1.0, được trang bị trình duyệt web và hỗ trợ Google Maps, Gmail, YouTube và lịch.
- 2009 - 2011: Google giới thiệu các bản cập nhật lớn như Eclair (Android 2.0), Froyo (Android 2.2) và Gingerbread (Android 2.3), đưa vào nhiều cải tiến về giao diện người dùng, hiệu suất và hỗ trợ công nghệ mới như NFC.
- 2011: Android 3.0 (Honeycomb) được ra mắt, đánh dấu phiên bản đầu tiên của Android được thiết kế riêng cho máy tính bảng.
- 2011: Android 4.0 (Ice Cream Sandwich) được giới thiệu, nhằm hợp nhất giao diện cho điện thoại và máy tính bảng và bắt đầu sự thống nhất nền tảng.
- 2014-2015: Android 5.0 (Lollipop) và Android 6.0 (Marshmallow) mang đến thiết kế Material Design, cải tiến quản lý pin và cấp quyền ứng dụng.
- 2016-2018: Google phát hành Android Nougat và Android Oreo, tiếp tục tối ưu hóa hệ điều hành và tăng cường bảo mật.
- 2018 - Nay: Android 9 Pie, Android 10, và các phiên bản sau đó như Android 11 và Android 12, đều tập trung vào việc cải thiện quyền riêng tư, bảo mật và trải

nghiệm người dùng với các tính năng như chế độ tối, quản lý thông báo mới và điều khiển cử chỉ.

2.2.3 Tầm quan trọng và phổ biến

Năm 2022, theo Statcounter, Android đang chiếm 71,3% thị phần toàn cầu. Trên thế giới, hầu hết mọi người đều sở hữu điện thoại Android. Thống kê cho thấy, năm ngoái đã có hơn 1 tỷ điện thoại Android được xuất xưởng, nâng tổng số người dùng Android lên mức 2,8 tỷ người.[1]

2.3 Dữ liệu hình ảnh trên điện thoại Android

2.3.1 Mức độ phổ dụng và tầm quan trọng của dữ liệu hình ảnh trên điện thoại Android

Theo thống kê trong mục 2.2.3, điện thoại Android hiện đang chiếm lĩnh thị trường toàn cầu với mức độ phổ biến rất cao. Sự phổ biến rộng rãi của điện thoại Android đã kéo theo việc dữ liệu hình ảnh trên các thiết bị này cũng trở nên phổ dụng hơn. Dữ liệu hình ảnh trên Android được sử dụng rộng rãi và ảnh hưởng lớn đến nhiều khía cạnh khác nhau của cuộc sống hiện đại. Cụ thể, sự phổ dụng của dữ liệu hình ảnh trên Android thể hiện rõ qua các khía cạnh sau:

- **Khả năng tiếp cận và sử dụng dễ dàng:** Điện thoại Android được trang bị các ứng dụng chụp và chỉnh sửa ảnh tích hợp sẵn hoặc có thể dễ dàng tải về từ Google Play Store. Điều này giúp người dùng mọi lứa tuổi và trình độ công nghệ có thể tạo, chỉnh sửa và lưu trữ hình ảnh một cách thuận tiện.
- **Dung lượng lưu trữ lớn:** Các thiết bị Android hiện đại thường có dung lượng lưu trữ lớn, từ 64GB đến 1TB, giúp người dùng có thể lưu trữ một lượng lớn hình ảnh mà không gặp phải vấn đề về không gian lưu trữ.
- **Tích hợp dịch vụ đám mây:** Nhiều thiết bị Android tích hợp sẵn dịch vụ lưu trữ đám mây như Google Photos, giúp người dùng có thể sao lưu và đồng bộ hóa hình ảnh một cách tự động, bảo đảm an toàn dữ liệu ngay cả khi mất thiết bị.
- **Tầm quan trọng trong đời sống hàng ngày:** Hình ảnh trên điện thoại Android không chỉ đơn thuần là những bức ảnh chụp kỷ niệm mà còn bao gồm các tài liệu quan trọng, biên lai, ghi chú và nhiều loại thông tin khác. Việc dễ dàng truy

cập và chia sẻ hình ảnh giúp tối ưu hóa nhiều hoạt động trong đời sống và công việc của người dùng.

2.3.2 Các định dạng hình ảnh phổ biến nhất trên điện thoại Android.

Dữ liệu hình ảnh trên điện thoại Android rất đa dạng và phong phú, đáp ứng nhu cầu của người dùng trong nhiều lĩnh vực khác nhau. Và chúng thường lưu trữ dưới dạng các tập tin có phần mở rộng là JPEG/JPG và PNG.

Những loại dữ liệu hình ảnh này không chỉ phong phú về mặt nội dung mà còn đa dạng về nguồn gốc, cho thấy sự phong phú trong cách mà người dùng Android khai thác và sử dụng điện thoại của mình để xử lý và lưu trữ hình ảnh. Sự đa dạng này cũng phản ánh nhu cầu ngày càng cao của người dùng trong việc sử dụng hình ảnh để ghi lại và chia sẻ thông tin, cũng như trong việc sáng tạo và tiêu dùng nội dung số.

2.3.3 Các mối đe dọa và rủi ro đối với dữ liệu hình ảnh trên điện thoại Android.

Dữ liệu nói chung và dữ liệu hình ảnh trên điện thoại Android nói riêng có thể đối mặt với nhiều mối đe dọa và rủi ro tiềm ẩn. Việc nhận biết và phòng tránh các rủi ro này là vô cùng quan trọng để bảo vệ thông tin cá nhân và đảm bảo an toàn cho dữ liệu. Sự mất mát hoặc xâm phạm dữ liệu không chỉ gây ra thiệt hại về vật chất mà còn có thể ảnh hưởng nghiêm trọng đến quyền riêng tư của người dùng. Dưới đây là một số mối đe dọa và rủi ro phổ biến nhất mà dữ liệu hình ảnh trên điện thoại Android thường phải đối mặt:

- Các phần mềm độc hại (malware) có thể xâm nhập vào điện thoại thông qua các ứng dụng không an toàn hoặc các liên kết độc hại. Những phần mềm này có thể đánh cắp, xóa, hoặc làm hỏng dữ liệu hình ảnh của người dùng.
- Các mối đe dọa và rủi ro liên quan đến DLHA không chỉ dừng lại ở việc bị mất dữ liệu hay truy cập trái phép, mà còn bao gồm các mối đe dọa từ mã độc ẩn hoặc nhúng bên trong dữ liệu hình ảnh.
- Ransomware là loại phần mềm độc hại mã hóa dữ liệu của người dùng và yêu cầu một khoản tiền chuộc để khôi phục dữ liệu. Nếu điện thoại bị tấn công bởi ransomware, dữ liệu hình ảnh có thể bị mất hoặc không thể truy cập được nếu không trả tiền chuộc.

- Khi điện thoại bị mất hoặc bị đánh cắp, dữ liệu hình ảnh trên thiết bị cũng có nguy cơ bị mất hoặc bị truy cập bởi những người không có thẩm quyền. Điều này có thể dẫn đến việc lộ thông tin cá nhân hoặc các hình ảnh nhạy cảm.
- Một số ứng dụng có thể thu thập và sử dụng dữ liệu hình ảnh của người dùng mà không có sự đồng ý rõ ràng. Những ứng dụng này có thể chia sẻ dữ liệu với bên thứ ba hoặc sử dụng cho các mục đích quảng cáo.
- Các hacker có thể tấn công vào điện thoại Android thông qua mạng Wi-Fi công cộng hoặc các điểm truy cập không an toàn khác. Khi bị tấn công, dữ liệu hình ảnh có thể bị đánh cắp hoặc bị thay đổi mà người dùng không hề hay biết.
- Việc sao lưu dữ liệu hình ảnh trên các dịch vụ đám mây không an toàn hoặc không có biện pháp bảo mật phù hợp có thể dẫn đến việc dữ liệu bị lộ hoặc bị truy cập trái phép. Điều này đặc biệt nguy hiểm nếu các dịch vụ sao lưu bị tấn công hoặc có lỗ hổng bảo mật.
- Các lỗi phần cứng hoặc phần mềm có thể gây ra mất dữ liệu hình ảnh. Ví dụ, lỗi hệ điều hành có thể làm hỏng các tập hình ảnh hoặc làm cho chúng không thể truy cập được.
- Việc sử dụng mật khẩu yếu hoặc không sử dụng các biện pháp bảo vệ như xác thực hai yếu tố có thể làm tăng nguy cơ dữ liệu bị truy cập trái phép. Điều này đặc biệt nguy hiểm khi có nhiều hình ảnh nhạy cảm lưu trữ trên thiết bị.

2.3.4 Các nguyên tắc cơ bản của an toàn dữ liệu.

Việc lưu trữ và chia sẻ dữ liệu hình ảnh trên điện thoại Android cũng đặt ra nhiều thách thức về an toàn và bảo mật. Dữ liệu hình ảnh chứa đựng nhiều thông tin nhạy cảm và riêng tư, do đó việc bảo vệ chúng khỏi các mối đe dọa tiềm ẩn là vô cùng quan trọng. Các nguyên tắc cơ bản dưới đây sẽ giúp đảm bảo an toàn dữ liệu hình ảnh trên điện thoại Android:

Mã hóa dữ liệu: Mã hóa là biện pháp quan trọng nhất để bảo vệ dữ liệu hình ảnh lưu trữ trên thiết bị. Khi dữ liệu được mã hóa, chỉ có những ai có khóa giải mã mới có thể truy cập vào nội dung của nó. Android cung cấp các cơ chế mã hóa mạnh mẽ

như AES (Advanced Encryption Standard), giúp bảo vệ dữ liệu ngay cả khi thiết bị bị mất hoặc bị đánh cắp.

Quyền truy cập ứng dụng: Hạn chế quyền truy cập vào hình ảnh và các dữ liệu nhạy cảm khác cho các ứng dụng là một nguyên tắc quan trọng. Người dùng nên cẩn trọng khi cấp quyền truy cập cho các ứng dụng, chỉ nên cấp quyền cho các ứng dụng tin cậy và thật sự cần thiết để giảm thiểu nguy cơ lộ lọt dữ liệu.

Xác thực mạnh mẽ: Sử dụng các phương pháp xác thực mạnh mẽ như mật khẩu, mã PIN, hoặc sinh trắc học (vân tay, khuôn mặt) để bảo vệ truy cập vào thiết bị và dữ liệu hình ảnh. Xác thực mạnh mẽ đảm bảo rằng chỉ có người dùng được ủy quyền mới có thể truy cập vào thiết bị và dữ liệu của họ.

Cập nhật phần mềm thường xuyên: Đảm bảo rằng hệ điều hành Android và các ứng dụng liên quan luôn được cập nhật phiên bản mới nhất. Việc cập nhật phần mềm thường xuyên giúp vá các lỗ hổng bảo mật và cung cấp các tính năng bảo mật mới nhất để bảo vệ dữ liệu người dùng.

Sao lưu dữ liệu định kỳ: Thực hiện sao lưu dữ liệu hình ảnh định kỳ là một biện pháp quan trọng để bảo vệ dữ liệu. Sao lưu dữ liệu đảm bảo rằng người dùng có thể khôi phục dữ liệu trong trường hợp thiết bị bị hỏng hóc hoặc dữ liệu bị mất.

Sử dụng ứng dụng bảo mật: Cài đặt và sử dụng các ứng dụng bảo mật uy tín để quét và phát hiện các mối đe dọa bảo mật. Các ứng dụng bảo mật giúp bảo vệ dữ liệu khỏi phần mềm độc hại và các cuộc tấn công mạng, đồng thời cung cấp các công cụ quản lý và bảo vệ dữ liệu.

Kiểm soát và quản lý thiết bị từ xa: Sử dụng các dịch vụ quản lý thiết bị từ xa như Google Find My Device để khóa, xóa dữ liệu hoặc định vị thiết bị từ xa trong trường hợp bị mất hoặc bị đánh cắp. Quản lý thiết bị từ xa giúp người dùng bảo vệ dữ liệu của họ ngay cả khi họ không có quyền truy cập trực tiếp vào thiết bị.

Chương 3. KIẾN TRÚC TỔ CHỨC BÊN TRONG DỮ LIỆU HÌNH ẢNH

3.1 Tổng quan

Trong mục này, chúng ta sẽ thảo luận về các khái niệm cơ bản liên quan đến dữ liệu hình ảnh và vai trò quan trọng của chúng trong nền công nghiệp hiện đại. Đồng thời, chúng ta cũng sẽ đi sâu vào phân tích cấu trúc lưu trữ của các định dạng ảnh phổ biến như JPEG và PNG.

3.1.1 Định nghĩa dữ liệu hình ảnh

Dữ liệu hình ảnh là thông tin kỹ thuật số đại diện cho hình ảnh. Dữ liệu này bao gồm các điểm ảnh (pixel), mỗi điểm ảnh đại diện cho một giá trị màu sắc và cường độ sáng, với độ phân giải được xác định bởi số lượng điểm ảnh theo chiều ngang và chiều dọc. Các kênh màu chính thường là đỏ, xanh lục và xanh lam (RGB).

Dữ liệu hình ảnh có 2 dạng lưu trữ là bitmap và vector, trong đó ảnh bitmap bao gồm các điểm ảnh cụ thể còn ảnh vector sử dụng các đường thẳng và đường cong để biểu diễn hình ảnh. Còn phân loại theo màu sắc thì hình ảnh có thể là đơn sắc.[2] (chỉ gồm các mức độ xám - Grayscale) hoặc ảnh màu (gồm nhiều kênh màu).

Mỗi tập tin hình ảnh bao gồm không chỉ bản thân dữ liệu hình ảnh mà còn có các siêu dữ liệu (metadata) đi kèm như thời gian chụp, địa điểm chụp, thông số camera (ISO, khẩu độ, tốc độ màn trập), và thông tin về thiết bị chụp. Siêu dữ liệu này đóng vai trò quan trọng trong việc quản lý, sắp xếp và tìm kiếm hình ảnh trên điện thoại Android.

3.1.2 Vai trò của dữ liệu hình ảnh

Dữ liệu hình ảnh đóng vai trò vô cùng quan trọng trong nhiều khía cạnh của cuộc sống và công nghệ. Hình ảnh giúp con người ghi lại, lưu trữ và chia sẻ thông tin một cách trực quan và sinh động. Chúng ta sử dụng hình ảnh để truyền tải thông điệp, thể hiện ý tưởng và cảm xúc mà lời nói hoặc văn bản khó có thể diễn đạt hết. Một bức

ảnh có thể truyền tải hàng ngàn từ ngữ, tạo ra sự kết nối mạnh mẽ và nhanh chóng giữa người gửi và người nhận thông điệp.

Hình ảnh là phương tiện mạnh mẽ để thu hút sự chú ý, tạo ra ấn tượng và gợi cảm xúc. Chúng ta dễ dàng bị thu hút bởi hình ảnh màu sắc và sinh động hơn là những đoạn văn bản dài và khô khan. Nhờ đó, hình ảnh trở thành công cụ quan trọng trong việc truyền tải thông tin và quảng bá sản phẩm, dịch vụ.

Trong môi trường học tập và nghiên cứu, hình ảnh giúp minh họa và giải thích các khái niệm phức tạp, làm cho thông tin trở nên dễ hiểu và dễ tiếp cận hơn. Một hình ảnh minh họa đúng lúc có thể làm sáng tỏ một ý tưởng, giúp người học nắm bắt và ghi nhớ thông tin tốt hơn. Ví dụ, trong sách giáo khoa, hình ảnh minh họa các quá trình sinh học, hóa học hay vật lý giúp học sinh dễ dàng hình dung và hiểu rõ hơn các khái niệm trừu tượng.

Ngoài ra, dữ liệu hình ảnh còn hỗ trợ việc phân tích và xử lý thông tin. Công nghệ xử lý hình ảnh và nhận dạng mẫu giúp chúng ta phân tích một lượng lớn dữ liệu hình ảnh, phát hiện và nhận diện các đối tượng, xu hướng và mẫu hình trong dữ liệu. Điều này có ứng dụng rộng rãi trong nhiều lĩnh vực, từ y tế, an ninh, đến nghiên cứu khoa học và công nghệ.

Hình ảnh cũng đóng vai trò quan trọng trong việc bảo tồn và chia sẻ di sản văn hóa. Qua hình ảnh, chúng ta có thể lưu giữ và truyền tải những giá trị văn hóa, lịch sử từ thế hệ này sang thế hệ khác. Các bức ảnh, tranh vẽ, và hình ảnh kỹ thuật số giúp ghi lại những khoảnh khắc lịch sử, bảo tồn những tác phẩm nghệ thuật và di sản văn hóa quan trọng.

3.1.3 Các loại dữ liệu hình ảnh thường gặp

Trong cuộc sống hàng ngày, dữ liệu hình ảnh xuất hiện trong nhiều lĩnh vực và tình huống khác nhau.

Nhiếp ảnh kỹ thuật số: Bao gồm chụp ảnh cá nhân (gia đình, bạn bè, sự kiện) và chụp ảnh chuyên nghiệp (thương mại, thời trang, phong cảnh).

Giám sát và An ninh: Sử dụng camera an ninh tại các khu vực công cộng, nhà riêng, cơ quan, và camera giao thông để giám sát và đảm bảo an ninh.

Y tế: Chẩn đoán hình ảnh (X-quang, MRI, CT scan) và siêu âm để theo dõi sức khỏe bệnh nhân và phát hiện các vấn đề y tế.

Vệ tinh và Khí tượng: Thu thập ảnh vệ tinh và sử dụng radar để theo dõi và dự báo thời tiết, khí hậu.

Thương mại điện tử: Chụp ảnh sản phẩm và sử dụng hình ảnh quảng cáo cho các nền tảng thương mại điện tử và quảng cáo trực tuyến.

Giáo dục và Nghiên cứu: Sử dụng hình ảnh trong tài liệu giảng dạy và nghiên cứu khoa học.

Giải trí: Sản xuất phim, chương trình truyền hình, video âm nhạc và trò chơi điện tử.

Mạng xã hội: Chia sẻ ảnh và livestream trên các nền tảng như Facebook, Instagram.

Công nghệ và Khoa học Máy tính: Thị giác máy tính (nhận diện khuôn mặt, vật thể) và thực tế ảo (VR) hoặc thực tế tăng cường (AR).

Nông nghiệp và Địa lý: Giám sát cây trồng, phát hiện sâu bệnh và khảo sát địa hình, lập bản đồ môi trường.

Những dữ liệu hình ảnh này không chỉ đa dạng về nguồn gốc và ứng dụng mà còn đóng vai trò quan trọng trong việc nâng cao chất lượng cuộc sống và hiệu quả công việc trong nhiều lĩnh vực.

3.1.4 Các định dạng dữ liệu hình ảnh phổ biến

Mỗi tập tin hình ảnh được lưu trữ bằng các định dạng khác nhau, mỗi định dạng có đặc điểm và ứng dụng riêng, phù hợp với nhu cầu cụ thể của người dùng. Dưới đây là một số định dạng phổ biến của tập tin lưu trữ hình ảnh:

JPEG (Joint Photographic Experts Group) là định dạng nén mất mát phổ biến nhất, thường được sử dụng cho ảnh kỹ thuật số. Định dạng này giúp cân bằng giữa chất lượng hình ảnh và kích thước tệp, làm cho nó trở thành lựa chọn lý tưởng cho việc lưu trữ và chia sẻ ảnh trên các thiết bị và nền tảng khác nhau.

PNG (Portable Network Graphics) sử dụng kỹ thuật nén không mất mát, đảm bảo chất lượng hình ảnh cao mà không bị mất thông tin. Định dạng này thường được sử dụng

dụng cho đồ họa và hình ảnh web, đặc biệt là những hình ảnh cần hỗ trợ kênh alpha (độ trong suốt).

GIF (Graphics Interchange Format) là định dạng nén không mất mát, chủ yếu được sử dụng cho hình ảnh động. Mặc dù hạn chế về số lượng màu sắc (tối đa 256 màu), GIF vẫn rất phổ biến trong việc tạo các ảnh động ngắn và các biểu tượng cảm xúc trên mạng xã hội.

BMP (Bitmap) là định dạng không nén, lưu trữ dữ liệu hình ảnh thô dưới dạng lưới các điểm ảnh. Do không sử dụng nén, tệp BMP thường có kích thước lớn, điều này làm cho nó ít phổ biến hơn trên web, nhưng vẫn được sử dụng trong các ứng dụng xử lý ảnh nơi mà chất lượng hình ảnh không bị ảnh hưởng bởi nén.

TIFF (Tagged Image File Format) là định dạng hình ảnh linh hoạt, hỗ trợ cả nén mất mát và không mất mát. Định dạng này thường được sử dụng trong các ứng dụng yêu cầu chất lượng hình ảnh cao và lưu trữ dữ liệu hình ảnh chi tiết, chẳng hạn như in ấn và nhiếp ảnh chuyên nghiệp.

Những định dạng này đáp ứng các nhu cầu khác nhau về chất lượng, kích thước tệp và mục đích sử dụng, giúp người dùng lựa chọn giải pháp tối ưu cho việc lưu trữ và xử lý hình ảnh của mình.

3.1.5 Các thành phần chính trong tập tin hình ảnh

Header:

Chứa thông tin về định dạng tệp, kích thước hình ảnh, độ phân giải, số lượng màu sắc và các thông tin cần thiết khác để hiển thị hình ảnh.

Data:

Phần chứa dữ liệu hình ảnh thực tế, được mã hóa dưới dạng pixel.

Cách mã hóa và lưu trữ pixel phụ thuộc vào định dạng tệp. Các định dạng tệp khác nhau (như JPEG, PNG, GIF,...) có phương pháp nén và lưu trữ dữ liệu khác nhau.

Metadata:

Siêu dữ liệu chứa thông tin bổ sung về hình ảnh như tên tệp, ngày chụp, thông số máy ảnh (dữ liệu EXIF), vị trí GPS, và các thẻ mô tả khác.

Siêu dữ liệu có thể được nhúng trong tệp hình ảnh (như trong định dạng JPEG) hoặc lưu trữ riêng biệt (như trong một số hệ thống quản lý hình ảnh chuyên nghiệp).

3.1.6 Cách thức dữ liệu hình ảnh được lưu trữ

3.1.6.1 Nén không mất mát (Lossless Compression)

Nén không mất mát là phương pháp lưu trữ dữ liệu mà không làm mất bất kỳ thông tin nào của hình ảnh gốc. Các thuật toán nén không mất mát thường được sử dụng để lưu trữ các hình ảnh có chi tiết cao hoặc trong các trường hợp yêu cầu bảo toàn chất lượng tuyệt đối của hình ảnh.

Các định dạng phổ biến:

PNG (Portable Network Graphics): Hỗ trợ nén không mất mát, thường dùng cho đồ họa và hình ảnh web. PNG cũng hỗ trợ kênh alpha để thể hiện độ trong suốt.

GIF (Graphics Interchange Format): Cũng là định dạng nén không mất mát nhưng chỉ hỗ trợ tối đa 256 màu, thích hợp cho hình ảnh động hoặc đồ họa đơn giản.

TIFF (Tagged Image File Format): Hỗ trợ cả nén không mất mát và nén mất mát, thường được sử dụng trong in ấn và lưu trữ hình ảnh chất lượng cao.

Ưu điểm:

Không mất mát dữ liệu, đảm bảo chất lượng hình ảnh gốc.

Thích hợp cho lưu trữ hình ảnh y tế, kỹ thuật, và các hình ảnh cần bảo toàn chất lượng.

Nhược điểm:

Kích thước tệp lớn hơn so với các phương pháp nén mất mát.

3.1.6.2 Nén mất mát (Lossy Compression)

Nén mất mát là phương pháp lưu trữ dữ liệu mà một phần thông tin của hình ảnh gốc bị loại bỏ nhằm giảm kích thước tệp. Mức độ nén có thể điều chỉnh để cân bằng giữa chất lượng hình ảnh và kích thước tệp.

Các định dạng phổ biến:

JPEG (Joint Photographic Experts Group): Là định dạng nén mất mát phổ biến nhất cho ảnh kỹ thuật số. JPEG cho phép điều chỉnh mức độ nén, cân bằng giữa chất lượng hình ảnh và kích thước tệp.

WebP: Một định dạng nén mất mát và không mất mát được phát triển bởi Google, cung cấp kích thước tệp nhỏ hơn so với JPEG và PNG với chất lượng tương đương.

Ưu điểm:

Giảm đáng kể kích thước tệp, tiết kiệm không gian lưu trữ.

Thích hợp cho lưu trữ và chia sẻ ảnh trên web và các nền tảng truyền thông xã hội.

Nhược điểm:

Chất lượng hình ảnh giảm khi mức độ nén tăng, mất mát dữ liệu không thể khôi phục.

Không thích hợp cho các hình ảnh cần độ chính xác cao như hình ảnh y tế hoặc kỹ thuật.

3.2 EXIF Tổ chức siêu dữ liệu trong tập tin hình ảnh

3.2.1 Giới thiệu

EXIF (Exchangeable Image File Format) là một tiêu chuẩn được thiết lập bởi Hiệp hội Công nghiệp Điện tử và Công nghệ Thông tin Nhật Bản (JEIDA) để xác định cấu trúc lưu trữ thông tin và siêu dữ liệu trong các tập tin hình ảnh. EXIF đặc biệt hữu ích cho các máy ảnh kỹ thuật số vì nó cho phép lưu trữ không chỉ hình ảnh mà còn các thông tin như ngày giờ chụp, cài đặt máy ảnh, và vị trí địa lý. Thành phần này chủ yếu được sử dụng trong các tập tin JPEG và các tập tin PNG gần đây; giúp các phần mềm xem ảnh và chỉnh sửa ảnh dễ dàng truy xuất và hiển thị các thông tin mở rộng có liên quan.

3.2.2 Vai trò

Thông tin EXIF (Exchangeable Image File Format) là một dạng siêu dữ liệu được nhúng vào trong các file ảnh kỹ thuật số, cung cấp chi tiết về các thông số kỹ thuật và môi trường khi ảnh được chụp. Vai trò của thông tin EXIF trong ảnh có thể được phân tích qua nhiều khía cạnh khác nhau như sau:

Hỗ trợ quá trình quản lý và tổ chức ảnh

EXIF cung cấp các thông tin chi tiết như ngày giờ chụp, loại máy ảnh, ống kính, và các thiết lập như độ mở ống kính (aperture), tốc độ màn trập (shutter speed), ISO, và tiêu cự. Những thông tin này giúp người dùng dễ dàng quản lý, phân loại, và tìm

kiểm ảnh dựa trên các tiêu chí cụ thể mà không cần phải xem từng ảnh một cách thủ công. Ví dụ, người dùng có thể dễ dàng tìm ra tất cả các ảnh được chụp vào một ngày cụ thể hoặc với một loại máy ảnh cụ thể.

Hỗ trợ quá trình chỉnh sửa và tối ưu hóa ảnh

Đối với các nhiếp ảnh gia và nhà chỉnh sửa ảnh chuyên nghiệp, thông tin EXIF cung cấp cái nhìn sâu sắc về các điều kiện và thiết lập ban đầu khi ảnh được chụp. Điều này giúp họ hiểu rõ hơn về những điều chỉnh cần thiết để cải thiện chất lượng ảnh. Ví dụ, nếu ảnh bị nhiễu (noise) do ISO quá cao, họ có thể điều chỉnh các thông số này trong quá trình hậu kỳ để đạt được kết quả tốt nhất.

Phục vụ cho mục đích học tập và phân tích

Thông tin EXIF cũng rất hữu ích cho các nhiếp ảnh gia học hỏi và cải thiện kỹ năng của mình. Bằng cách xem xét thông số EXIF của những bức ảnh họ yêu thích, họ có thể học hỏi cách mà những nhiếp ảnh gia khác thiết lập máy ảnh của họ trong các điều kiện chụp khác nhau. Điều này đặc biệt hữu ích trong việc hiểu và áp dụng các kỹ thuật nhiếp ảnh một cách hiệu quả.

Tăng cường khả năng bảo mật và xác thực ảnh

Trong một số trường hợp, thông tin EXIF có thể giúp xác thực nguồn gốc của bức ảnh, đặc biệt là trong các tình huống liên quan đến pháp lý hoặc tranh chấp bản quyền. EXIF lưu trữ thông tin về thời gian, địa điểm chụp, và các thiết lập máy ảnh, do đó có thể cung cấp bằng chứng xác thực về tính chân thực của ảnh. Tuy nhiên, cần lưu ý rằng thông tin EXIF cũng có thể bị giả mạo, do đó không thể hoàn toàn phụ thuộc vào nó cho mục đích pháp lý.

Hỗ trợ trong lĩnh vực địa lý và bản đồ học

Một số máy ảnh và điện thoại thông minh hiện nay có khả năng gắn thêm thông tin vị trí địa lý vào EXIF, bao gồm kinh độ, vĩ độ, và độ cao. Điều này rất hữu ích trong các ứng dụng liên quan đến bản đồ và địa lý, giúp người dùng biết chính xác vị trí mà một bức ảnh được chụp. Ví dụ, các nhà khảo cổ học có thể sử dụng thông tin này để theo dõi vị trí của các di vật trong quá trình khai quật.

3.2.3 Cấu trúc

3.2.3.1 Cấu trúc dữ liệu EXIF

Trong định dạng tập tin JPEG, các vùng đánh dấu ứng dụng (Application Marker Segments) được sử dụng để lưu trữ thông tin bổ sung liên quan đến hình ảnh. Các vùng này được đánh dấu bằng các mã APPn, với n là một giá trị từ 0 đến 15. Hai vùng APP phổ biến nhất là APP0 và APP1, nhưng cũng có thể có các vùng APP khác được định nghĩa bởi các nhà sản xuất hoặc các tiêu chuẩn khác.

Dữ liệu EXIF được lưu trữ trong vùng APP1, bắt đầu bằng 2 byte ký tự dưới dạng hex “FF E1”. Tiếp đó tới 2 byte chứa kích thước của vùng APP1 này “SSSS” dưới dạng hex. Kế đó là chuỗi 4 byte ASCII "Exif" và 2 byte 0x00 (tổng cộng 06 byte). Sau đó, dữ liệu EXIF sẽ được tổ chức như định dạng TIFF. Cấu trúc cơ bản của dữ liệu EXIF (APP1) được mô tả như sau:

FFE1	APP1 Marker		
SSSS	APP1 Data	APP1 Data Size	
45786966 0000		Exif Header	
49492A00 08000000		TIFF Header	
XXXX....		IFD0 (main image)	Directory
LLLLLLLL			Link to IFD1
XXXX....		Data area of IFD0	
XXXX....		Exif SubIFD	Directory
00000000			End of Link
XXXX....		Data area of Exif SubIFD	
XXXX....		IFD1(thumbnail image)	Directory
00000000			End of Link
XXXX....		Data area of IFD1	
FFD8XXXX. . . XXXXFFD9		Thumbnail image	

Hình 3.1 Cấu trúc của APP1[3]

3.2.3.2 Cấu trúc TIFF Header (Tagged Image File Format)

8 byte đầu tiên là TIFF Header. 2 byte đầu xác định byte align của dữ liệu TIFF, giá trị 0x4949 cho kiểu đọc Little Endian hoặc 0x4d4d cho kiểu đọc Big Endian. 2 byte tiếp theo luôn là giá trị 0x002A. Cuối cùng là 4 byte cuối của TIFF Header là

một Offset chỉ đến IFD (Image File Directory) đầu tiên, thường bắt đầu ngay sau TIFF Header, với giá trị Offset là 0x00000008.

3.2.3.3 Cấu trúc IFD (Image File Directory)

Ngay sau TIFF Header là IFD đầu tiên, chứa thông tin hình ảnh. Cấu trúc cơ bản của một IFD:

EEEE				No. of directory entry
TTTT	ffff	NNNNNNNN	DDDDDDDD	Entry 0
TTTT	ffff	NNNNNNNN	DDDDDDDD	Entry 1
.....				
TTTT	ffff	NNNNNNNN	DDDDDDDD	Entry EEEE-1
LLLLLLLL				Offset to next IFD

Hình 3.2 Cấu trúc IFD [3]

Trong đó 2 byte đầu tiên sẽ là số lượng Directory entry có trong IFD này, 'TTTT' là số Tag, 'ffff' là định dạng dữ liệu, 'NNNNNNNN' là số lượng thành phần và 'DDDDDDDD' chứa nội dung dữ liệu hoặc Offset đến địa chỉ dữ liệu (nếu dữ liệu nhỏ hơn 4 byte thì nội dung dữ liệu sẽ lưu trong phần này). Bảng dưới đây là chú thích cho định dạng dữ liệu 'ffff':

Value	1	2	3	4	5	6
Format	unsigned byte	ascii strings	unsigned short	unsigned long	unsigned rational	signed byte
Bytes/component	1	1	2	4	8	1

Value	7	8	9	10	11	12
Format	undefined	signed short	signed long	signed rational	single float	double float
Bytes/component	1	2	4	8	4	8

Hình 3.3 Bảng chú thích kiểu dữ liệu [3]

Dưới đây là ví dụ cụ thể về một IFD:

- “88 25 00 04 00 00 00 01 00 00 0A 1C” hex

88 25 là tag GPSInfo

00 04 là định dạng dữ liệu, lúc này là unsigned long

00 00 00 01 là số lượng thành phần, lúc này là 1

-> Từ định dạng dữ liệu và số lượng thành phần, ta có thể tính được độ dài (byte) của phần dữ liệu, độ dài byte = 4 * 1 (4 là số byte/1 thành phần của kiểu unsigned

long) = 4 byte. Do đây là 1 tag đặt biệt lưu nhóm các dữ liệu về GPS nên 4 byte cuối sẽ lưu địa chỉ

00 00 0A 1C là vị trí bắt đầu của dữ liệu (tính từ vị trí đầu TIFF Header)

3.2.3.4 Cấu trúc dữ liệu của IFD

Dữ liệu của IFD cũng sẽ được đọc giống cấu trúc của IFD được giới thiệu ở mục 3.2.2.3. Để thực tế hơn thì phần này sẽ tập trung phân tích dữ liệu GPS của 1 tấm ảnh (Nếu có). Những dữ liệu khác được đọc tương tự. Thông tin của các tag dữ liệu khác được tham khảo ở [4].

Dưới đây là 1 ví dụ về tags để đọc vĩ độ, và cách làm tương tự với kinh độ từ đó ta sẽ có được vị trí dựa theo 2 thông tin này:

- “02 00 05 00 00 00 03 00 00 0A D6 00”

00 02 là tag GPSLatitude

00 05 là định dạng dữ liệu, lúc này là unsigned rational

00 00 00 03 3 thành phần -> $8 * 3 = 24$ byte

00 00 0A D6 là vị trí bắt đầu của dữ liệu (tính từ vị trí đầu TIFF Header)

Sau khi dựa vào vị trí tìm được dữ liệu vĩ độ như sau:

- ”00 00 00 0A 00 00 00 01 00 00 00 30 00 00 00 01 00 00 00 5D 00 00 00 64”

ta sẽ đọc dựa bằng cách:

$00\ 00\ 00\ 0A / 00\ 00\ 00\ 01 = 10$

$00\ 00\ 00\ 30 / 00\ 00\ 00\ 01 = 48$

$00\ 00\ 00\ 5D / 00\ 00\ 00\ 64 = 0.93$

vậy ta sẽ có vĩ độ là $10^{\circ} 48' 0.93''$ N (“N” có thể đọc trong tag GPSLatitudeRef).

3.3 Định dạng tập tin hình ảnh JPEG

3.3.1 Giới thiệu

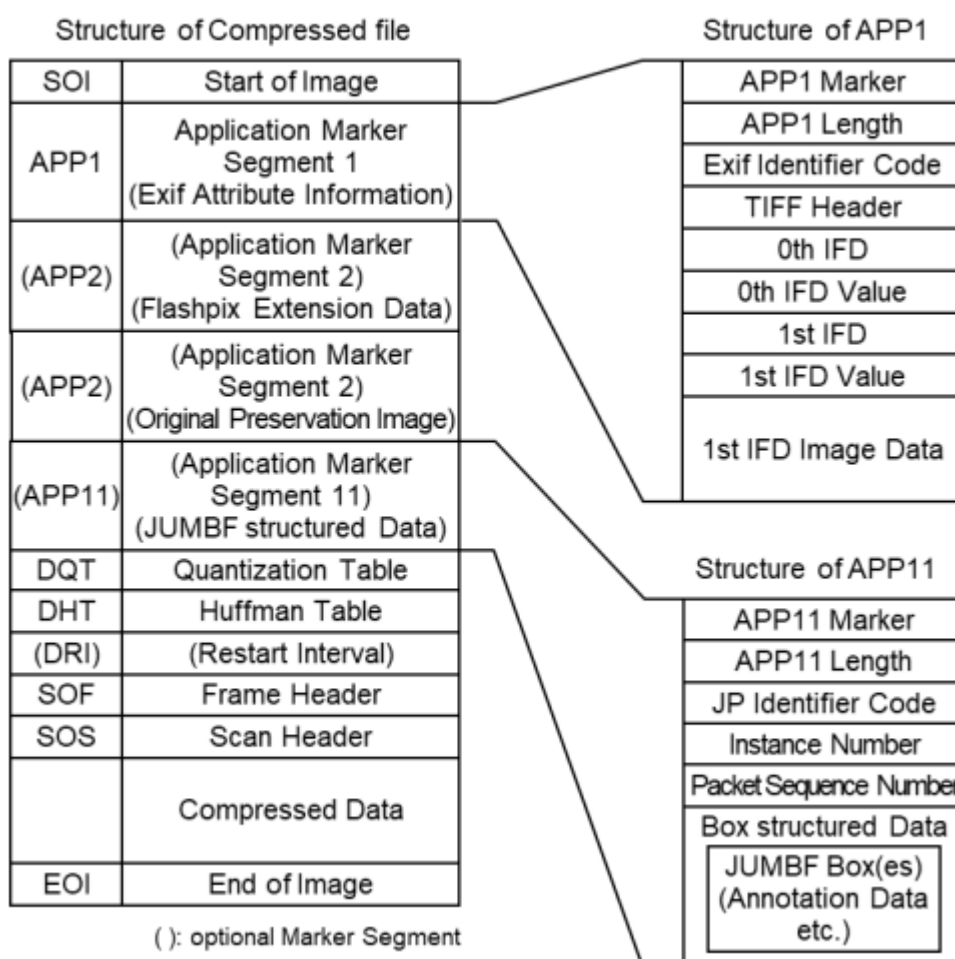
JPEG (Joint Photographic Experts Group) là một phương pháp nén ảnh được phát triển bởi nhóm chuyên gia cùng tên thuộc CCITT và ISO/IEC. Nhóm này được thành lập vào năm 1986 để thiết lập một tiêu chuẩn cho việc mã hóa liên tiếp tiến bộ của ảnh có độ tông màu liên tục, cả ảnh xám và ảnh màu. Mục tiêu của tiêu chuẩn JPEG là cung cấp các quá trình và đại diện mã hóa hình ảnh nén để trao đổi giữa các ứng

dụng, áp dụng cho nhiều ứng dụng như xuất bản trên máy tính để bàn, nghệ thuật đồ họa, hình ảnh y tế và hình ảnh khoa học.

JPEG được thiết kế để nén các ảnh tĩnh liên tục tông màu, không áp dụng cho dữ liệu hình ảnh hai mức. Tiêu chuẩn này bao gồm hai phần: Yêu cầu và hướng dẫn triển khai, và Kiểm tra tuân thủ. Phần thứ nhất đặt ra các yêu cầu và hướng dẫn cho các quá trình mã hóa và giải mã ảnh tĩnh liên tục tông màu, cũng như đại diện mã hóa của dữ liệu ảnh nén để trao đổi giữa các ứng dụng.

3.3.2 Cấu trúc file

Một tệp JPEG thông thường bao gồm nhiều marker và đoạn (segment) để xác định các thuộc tính và dữ liệu hình ảnh. Các đoạn này được xác định theo thứ tự tuần tự, bắt đầu với marker SOI (Start of Image) và kết thúc với marker EOI (End of Image).



Hình 3.4 Cấu trúc cơ bản của tệp tin nén JPEG [5]

3.3.2.1 Chữ ký của tập tin JPEG

Một tập JPEG bắt đầu với marker SOI (Start of Image), được biểu diễn bằng chuỗi byte “FF D8” trong hệ thập lục phân. Marker này ngay lập tức được theo sau bởi marker APP1 nếu có dữ liệu EXIF. Và kết thúc file là chữ ký EOI (End of Image) với chuỗi byte “FF D9” trong hệ thập lục phân.

3.3.2.2 Các thành phần chính trong JPEG [5]

APP1 (Application Segment 1): Chứa metadata EXIF, bao gồm các dữ liệu hình ảnh như cài đặt máy ảnh, thời gian, và bản xem trước thu nhỏ. Đề tài chỉ tập trung vào phần này là chủ yếu và đã được phân tích chi tiết ở mục 3.2.

DQT (Define Quantization Table): Định nghĩa một hoặc nhiều bảng lượng tử hóa, được sử dụng để kiểm soát mức độ nén.

DHT (Define Huffman Table): Định nghĩa các bảng mã hóa Huffman được sử dụng trong quá trình mã hóa entropy của JPEG.

SOF (Start of Frame): Chỉ định các tham số cơ bản của hình ảnh, như chiều cao, chiều rộng, số lượng thành phần, và độ chính xác.

SOS (Start of Scan): Báo hiệu sự bắt đầu của dữ liệu hình ảnh thực tế.

EOI (End of Image): Marker này báo hiệu kết thúc của dữ liệu hình ảnh JPEG

3.4 Định dạng tập tin hình ảnh PNG

3.4.1 Giới thiệu

PNG (Portable Network Graphics) là một định dạng tập tin hình ảnh raster sử dụng nén không mất dữ liệu. Điều này có nghĩa là khi bạn nén và lưu trữ một hình ảnh PNG, chất lượng của hình ảnh không bị giảm đi. PNG được tạo ra để thay thế định dạng GIF và cung cấp nhiều tính năng hơn, như hỗ trợ độ trong suốt và dải màu rộng. PNG thường được sử dụng trong các ứng dụng đồ họa cần hình ảnh chất lượng cao và không bị mất chi tiết khi chỉnh sửa hoặc chia sẻ. Định dạng này cũng rất phổ biến và được hỗ trợ bởi hầu hết các trình duyệt và phần mềm chỉnh sửa hình ảnh. [6]

3.4.2 Cấu trúc file

3.4.2.1 Chữ ký của tập tin PNG

Mỗi tập tin PNG bắt đầu với một chữ ký 8 byte cụ thể để xác định định dạng tệp.

Chữ ký này là (dạng hexadecimal): “89 50 4E 47 0D 0A 1A 0A”

Với 8 byte này xuất hiện ở đầu sẽ xác định là file PNG hợp lệ[7]

3.4.2.2 Các khối (Chunks)

Mỗi khối Chunk chia thành 4 phần: [7]

- Độ dài

Một số nguyên không dấu 4 byte cho biết số byte trong trường dữ liệu của khối. Độ dài chỉ đếm trường dữ liệu, không tính trường độ dài, mã loại khối, hoặc CRC. Giá trị bằng không là một độ dài hợp lệ. Mặc dù các bộ mã hóa và giải mã nên xử lý độ dài như một số không dấu, giá trị của nó không được vượt quá 231 byte.

- Mã loại khối

Mã loại khối dài 4 byte. Để tiện lợi trong mô tả và kiểm tra các tệp PNG, các mã loại này được giới hạn trong các ký tự chữ cái ASCII viết hoa và viết thường (A-Z và a-z, hoặc 65-90 và 97-122 trong hệ thập phân). Tuy nhiên, các bộ mã hóa và giải mã phải xử lý các mã này như các giá trị nhị phân cố định, không phải chuỗi ký tự. Ví dụ, không được biểu diễn mã loại IDAT bằng các ký tự tương đương trong EBCDIC.

- Dữ liệu khối

Các byte dữ liệu ứng với mã loại khối, nếu có. Trường dữ liệu này có thể có độ dài bằng 0.

- CRC

Mã CRC 4 byte, được tính trên các byte trước đó trong khối, bao gồm cả mã loại khối và trường dữ liệu, nhưng không bao gồm trường độ dài. CRC luôn có mặt, ngay cả khi khối không chứa dữ liệu.

Trong 1 file PNG có rất nhiều Chunk khác nhau, một file PNG hợp lệ phải có chứa Chunk IHDR, một hoặc nhiều Chunk IDAT và Chunk IEND đánh dấu kết thúc file PNG có 4 byte dạng hexadecimal như sau: “49 45 4E 44”.

3.4.2.3 Cấu trúc EXIF trong PNG

Trong phần này để ngắn gọn, đề tài chỉ nêu ra các Chunk có khả năng chứa thông tin EXIF để hỗ trợ cho việc phục hồi dữ liệu ảnh PNG.

- EXIF: Exchangeable Image File (EXIF) Profile [8]

Khối EXIF chứa thông tin siêu dữ liệu về hình ảnh, thường được sử dụng trong máy ảnh kỹ thuật số để lưu trữ các thông tin như ngày giờ chụp, cài đặt máy ảnh, thông tin địa lý, v.v.

Mã loại khối sẽ gồm 4 byte hexadecimal sau:

65 58 49 66

Dữ liệu khối EXIF, thường được mã hóa theo chuẩn TIFF (Tagged Image File Format)

- tEXt: Textual data

Khối tEXt chứa dữ liệu văn bản không nén, thường dùng để lưu các thông tin như tiêu đề, tác giả, bản quyền, v.v.

Mã loại khối sẽ gồm 4 byte hexadecimal sau:

74 45 58 74

Dữ liệu khối tEX định dạng “keyword\0text string”:

Keyword (từ khóa): Mô tả ngắn gọn về nội dung của văn bản, kết thúc bằng ký tự null (0x00).

Text string (chuỗi văn bản): Văn bản thực tế.

- zTXt: Compressed textual data

Mã loại khối sẽ gồm 4 byte hexadecimal sau:

7A 54 58 74

Dữ liệu văn bản nén ở định dạng “keyword\0compression method\0compressed text”:

Keyword (từ khóa): Mô tả ngắn gọn về nội dung của văn bản, kết thúc bằng ký tự null (0x00).

Compression method (phương pháp nén): Một byte cho biết phương pháp nén (ví dụ, 0 là zlib).

Compressed text (văn bản nén): Văn bản đã được nén.

- iTXt: International textual data

Mã loại khối sẽ gồm 4 byte hexadecimal sau:

69 54 58 74

Dữ liệu văn bản quốc tế ở định dạng

“keyword\0compression flag\0compression method\0language tag\0translated keyword\0text”

Keyword (từ khóa): Mô tả ngắn gọn về nội dung của văn bản, kết thúc bằng ký tự null (0x00).

Compression flag (cờ nén): Một byte chỉ định xem văn bản có được nén hay không (0 là không nén, 1 là nén).

Compression method (phương pháp nén): Một byte cho biết phương pháp nén nếu compression flag là 1.

Language tag (thẻ ngôn ngữ): Một chuỗi cho biết ngôn ngữ của văn bản, theo chuẩn BCP 47.

Translated keyword (từ khóa đã dịch): Phiên bản dịch của keyword, kết thúc bằng ký tự null (0x00).

Text (văn bản): Văn bản thực tế, có thể được nén hoặc không tùy thuộc vào compression flag.

Chương 4. TỔNG QUAN HỆ THỐNG QUẢN LÝ TẬP TIN TRÊN HỆ ĐIỀU HÀNH ANDROID

4.1 Cách tổ chức lưu trữ tập tin trên Android

Tổ chức dữ liệu trên Android được thiết kế để tối ưu hóa hiệu suất và bảo mật. Mỗi thư mục có một chức năng cụ thể và quyền truy cập rõ ràng, giúp bảo vệ dữ liệu người dùng và duy trì hoạt động ổn định của hệ thống.

Trên Android, hệ thống tệp được tổ chức theo cấu trúc cây thư mục. Các thư mục chính bao gồm:

- Internal Storage: Dùng để lưu trữ các dữ liệu mà chỉ ứng dụng của bạn có thể truy cập.
- External Storage: Bao gồm cả bộ nhớ ngoài và bộ nhớ của thiết bị, cho phép các ứng dụng khác truy cập. Ví dụ: Thẻ SD hoặc bộ nhớ chung của thiết bị.

4.2 Các định dạng hệ thống tập tin trên Android

4.2.1 Giới thiệu chung

Hệ điều hành Android hỗ trợ nhiều định dạng hệ thống tập tin để đảm bảo khả năng tương thích với nhiều loại thiết bị và nhu cầu sử dụng khác nhau. Các định dạng này ảnh hưởng trực tiếp đến hiệu suất, bảo mật và độ bền của thiết bị.

4.2.2 Các định dạng hệ thống tập tin trên Android

Một số định dạng hệ thống tập tin phổ biến trên Android bao gồm:

- EXT4: Đây là định dạng hệ thống tập tin mặc định trên hầu hết các thiết bị Android hiện nay. EXT4 nổi bật với khả năng quản lý tập tin lớn, hiệu suất cao và tính ổn định.
- FAT32: Thường được sử dụng trên các thẻ nhớ và ổ USB do tính tương thích cao với nhiều hệ điều hành khác nhau. Tuy nhiên, FAT32 có giới hạn về kích thước tập tin tối đa là 4GB.

- exFAT: Là phiên bản nâng cấp của FAT32, hỗ trợ các tập tin lớn hơn 4GB và thường được sử dụng trên các thẻ nhớ và ổ cứng ngoài có dung lượng lớn.

4.2.3 So sánh các hệ thống tập tin trên Android

Dưới đây là so sánh các định dạng hệ thống tập tin phổ biến trên Android:

- EXT4: Đây là định dạng được ưa chuộng nhất trên các thiết bị Android do hiệu suất cao, độ ổn định và khả năng quản lý tập tin lớn. EXT4 hỗ trợ tính năng như kiểm tra và sửa lỗi tự động, giúp bảo vệ dữ liệu người dùng một cách hiệu quả.
- FAT32: Dù có tính tương thích cao, FAT32 lại bị giới hạn về kích thước tập tin và không hỗ trợ các tính năng bảo mật hiện đại.
- exFAT: Là lựa chọn tốt cho các thiết bị lưu trữ ngoài có dung lượng lớn, tuy nhiên không có nhiều tính năng bảo mật như EXT4.

4.3 Giới thiệu định dạng EXT4

4.3.1 Giới thiệu chung

EXT4, viết tắt của “Fourth Extended Filesystem”, là hệ thống tập tin được sử dụng rộng rãi trong các hệ điều hành linux. Đây là phiên bản cải tiến từ Ext3, mang lại nhiều cải tiến quan trọng về mặt hiệu suất và tính năng, nhằm đáp ứng nhu cầu ngày càng cao về phương diện lưu trữ và quản lý dữ liệu trong thời kỳ công nghệ hiện đại.[11]

4.3.2 Lịch sử phát triển

EXT4 được phát triển, cải tiến dựa trên hệ thống tập tin Ext3. Mặc dù bản Ext3 tiền nhiệm được phát hành năm 2001 và sớm trở thành chuẩn mực trong các hệ điều hành linux nhờ vào tính ổn định và độ tin cậy cao. Nhưng với sự phát triển nhanh chóng và không ngừng nghỉ của công nghệ lưu trữ, yêu cầu về mặt hiệu suất tăng cao đã làm cho Ext3 lộ rõ những khuyết điểm cũng như các giới hạn. Vì vậy, EXT4 đã được giới thiệu vào năm 2006 nhằm khắc phục những hạn chế này, với phiên bản ổn định đầu tiên được phát hành vào năm 2008.[11]

4.3.3 Đặc điểm kỹ thuật

EXT4 là một hệ thống tập tin phổ biến trên hệ điều hành Linux, được thiết kế để cung cấp hiệu suất cao, độ tin cậy và khả năng mở rộng cho việc lưu trữ và quản lý dữ liệu. Dưới đây là một cái nhìn tổng quan về các đặc điểm kỹ thuật của EXT4:

- **Extent-Based Storage:** EXT4 sử dụng khái niệm "Extent" thay vì các Block đơn lẻ như trước đó. Điều này giúp giảm phân mảnh và tăng hiệu suất truy xuất dữ liệu bằng cách lưu trữ thông tin về các phạm vi dữ liệu liên tục.
- **Journaling:** EXT4 tiếp tục hỗ trợ tính năng journaling, cho phép ghi lại các thay đổi vào một journal trước khi ghi vào ổ đĩa. Điều này giúp tăng cường độ tin cậy và khả năng phục hồi dữ liệu sau sự cố.
- **Delayed Allocation:** Tính năng này cho phép EXT4 trì hoãn việc cấp phát không gian trên đĩa cho đến khi thực sự cần thiết, giúp tối ưu hóa hiệu suất ghi dữ liệu và giảm phân mảnh.
- **MultiBlock Allocation:** Cấp phát nhiều khối dữ liệu một lúc, giúp tăng hiệu suất khi ghi dữ liệu lớn bằng cách giảm số lượng thao tác I/O cần thiết.
- **Large File System and File Size Support:** Hỗ trợ kích thước tập tin lớn lên đến 16 terabytes và kích thước hệ thống tập tin lớn lên đến 1 exabyte, đáp ứng nhu cầu lưu trữ dữ liệu ngày càng lớn.
- **Backward Compatibility:** EXT4 có khả năng tương thích ngược với EXT3, cho phép nâng cấp từ EXT3 lên EXT4 mà không cần định dạng lại hệ thống tập tin.
- **Checksumming:** EXT4 hỗ trợ checksumming trên journal, giúp phát hiện lỗi dữ liệu và đảm bảo tính toàn vẹn của dữ liệu.
- **Flexible Block Group Descriptor:** Cho phép tùy chỉnh kích thước Block Group descriptor, tăng khả năng mở rộng và hiệu suất cho các hệ thống lớn.
- **Directory Indexing:** Hỗ trợ cơ chế index cho các thư mục, giúp tìm kiếm và truy xuất dữ liệu nhanh chóng hơn trong các thư mục lớn.
- **Fast File System Check (fsck):** Quá trình kiểm tra và sửa chữa hệ thống tập tin nhanh chóng hơn so với các phiên bản trước, giúp giảm thời gian downtime của hệ thống.

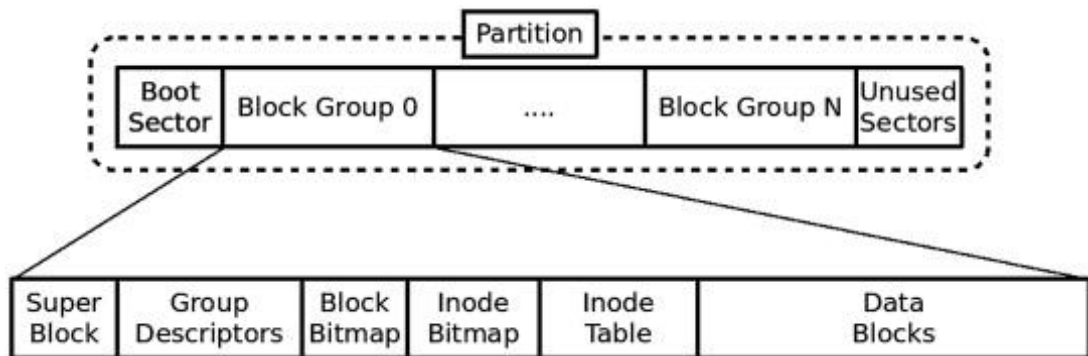
4.3.4 Lợi ích của EXT4

EXT4 mang lại nhiều lợi ích đáng kể, giúp nó trở thành một lựa chọn phổ biến và mạnh mẽ cho các hệ thống lưu trữ dữ liệu trên Linux. Dưới đây là một số lợi ích quan trọng của EXT4:

- Hiệu suất cao: Các tính năng như Extent-based storage, delayed allocation và multiBlock allocation giúp tăng hiệu suất truy xuất và ghi dữ liệu.
- Độ tin cậy cao: Tính năng journaling và checksumming bảo vệ dữ liệu trước các sự cố và đảm bảo tính toàn vẹn của dữ liệu.
- Hỗ trợ tập tin và hệ thống tập tin lớn: Khả năng hỗ trợ tập tin và hệ thống tập tin lớn đáp ứng nhu cầu lưu trữ của các ứng dụng hiện đại.
- Tính linh hoạt và tương thích: Khả năng tương thích ngược với EXT3 và khả năng tùy chỉnh Block Group Descriptor giúp tăng tính linh hoạt và khả năng mở rộng của hệ thống.

4.4 Kiến trúc định dạng EXT4

Cấu trúc hệ thống của EXT4 về cơ bản vẫn được sử dụng cấu trúc của các phiên bản EXT trước đó. Dưới đây là cấu trúc cơ bản của EXT4:



Hình 4.1 Mô hình cấu trúc EXT4 [12]

4.4.1 Superblock

Superblock là một thành phần quan trọng trong hệ thống tập tin EXT4, chứa các thông tin tổng quát về toàn bộ hệ thống tập tin. Nó cung cấp các thông tin cần thiết để quản lý và duy trì hệ thống tập tin, như kích thước, trạng thái, và các tham số cấu hình. Superblock đóng vai trò quan trọng trong việc xác định tính toàn vẹn và cấu trúc của hệ thống tập tin.

Superblock sẽ luôn nằm ở vị trí 1024 bytes do 1024 bytes đầu tiên của hệ thống tập tin EXT4 sẽ luôn được dùng để hỗ trợ boot

Cấu trúc và Offset cụ thể của một số các thành phần chính của Superblock được trình bày như sau:

Bảng 4.1 Bảng thông tin dữ liệu Superblock [13]

Offset (hex)	Kích thước (bytes)	Nội dung
0x0	4	Tổng số Inode có trong hệ thống tập tin
0x4	4	Tổng số Block có trong hệ thống tập tin
0x8	4	Số lượng Block mà chỉ có được phân bổ bởi super-user
0xC	4	Số lượng Block còn trống
0x10	4	Số lượng Inode còn trống
0x14	4	Block dữ liệu đầu tiên (1 đối với hệ thống 1k-Block và 0 đối với các kích thước Block khác)
0x18	4	Kích thước của Block là $[2^{(10 + X)}]$ với X là giá trị của trường này
0x1C	4	Kích thước của cluster là $[2^{(10 + X)}]$ nếu tính năng “bigalloc” được bật và phải bằng kích thước nếu tính năng này tắt
0x20	4	Số lượng Block có trong mỗi group
0x24	4	Số lượng cluster có trong mỗi group nếu “bigalloc” bật và bằng số lượng Block có trong mỗi group nếu tắt

0x28	4	Số lượng Inode có trong mỗi group
0x2C	4	Thời điểm Mount, tính theo thời gian Unix
0x30	4	Thời điểm ghi lần cuối, tính theo thời gian Unix
0x34	2	Số lần mount kể từ lần fsck cuối cùng
0x36	2	Số lần mount vượt quá mà fsck là cần thiết
0x38	2	Magic signature, giá trị đặc biệt để xác định hệ thống tập tin EXT4, giá trị này luôn là 0xEF53
0x5C	4	Cờ xác định những tính năng tương thích, thông qua trường này ta có thể xác định được hệ thống tập tin EXT4 có tính năng “journaling” hay không
0x78	16(char)	Tên volume
0x88	64	Vị trí mount cuối cùng của hệ thống tập tin
0xFE	2	kích thước của group descriptor, tính bằng bytes chỉ được đặt khi tính năng “64bit” tắt
0x120	60	Extent tree chứa vị trí của journal nếu tính năng “journaling” được bật

```

Offset(h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F Decoded text
000003F0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000400 30 DD 01 00 59 73 07 00 5E 5F 00 00 AE 2B 07 00 0Ý..Ys..^_...@+..
00000410 17 DD 01 00 00 00 00 00 02 00 00 00 02 00 00 00 .Ý.....
00000420 00 80 00 00 00 80 00 00 D0 1F 00 00 52 B4 0E 66 .€...€...Đ...R'.f
00000430 52 B4 0E 66 02 00 FF FF 53 EF 01 00 01 00 00 00 R'.f..ÿÿSi.....
00000440 6A AD 07 66 00 00 00 00 00 00 00 00 01 00 00 00 j..f.....
00000450 00 00 00 00 0B 00 00 00 00 01 00 00 3C 00 00 00 .....<...
00000460 C6 02 00 00 6B 04 00 00 0F 0E 56 4E 3E 01 49 30 E...k.....VN>.IO
00000470 80 FA 43 0D 4A 46 75 B9 00 00 00 00 00 00 00 00 €úC.JFu¹.....
00000480 00 00 00 00 00 00 00 00 00 2F 6D 65 64 69 61 2F 76 ...../media/v
00000490 69 6E 68 2F 30 66 30 65 35 36 34 65 2D 33 65 30 inh/0f0e564e-3e0
000004A0 31 2D 34 39 33 30 2D 38 30 66 61 2D 34 33 30 64 l-4930-80fa-430d
000004B0 34 61 34 36 37 35 62 39 00 00 00 00 00 00 00 00 4a4675b9.....
000004C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 EE 00 .....î.
000004D0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000004E0 08 00 00 00 00 00 00 00 00 00 00 00 00 B9 0E D8 29 .....¹.Ø)
000004F0 86 87 45 F6 95 EA 09 B2 A1 30 79 CF 01 01 40 00 †#Eö•ê.²;0yİ..@.
00000500 0C 00 00 00 00 00 00 00 00 6A AD 07 66 0A F3 01 00 .....j..f.ó..
00000510 04 00 00 00 00 00 00 00 00 00 00 00 00 20 00 00 .....
00000520 00 00 03 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000530 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000540 00 00 00 00 00 00 00 00 00 00 00 00 00 00 02 .....
00000550 00 00 00 00 00 00 00 00 00 00 00 00 20 00 20 00 .....
00000560 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000570 00 00 00 00 04 01 00 00 1D 8E 00 00 00 00 00 00 .....Ž.....

```

Hình 4.2 Bản mẫu Superblock của EXT4

Khi tiến hành đọc Superblock của hệ thống tập tin EXT4 điều đầu tiên cần chú trọng là magic signature của EXT4, vì đây là đặc điểm nhận dạng của hệ thống tập tin EXT4, nếu không có tức là hệ thống tập tin này không phải là EXT4 hoặc là hệ thống tập tin đã bị lỗi. Ta có thể tìm magic signature này ở vị trí Offset 0x38 nếu giá trị của nó bằng 0xEF53 ta có thể xác định đây là hệ thống tập tin EXT4[14]. Khi đã xác định được điều này ta có thể xác định các giá trị quan trọng khác như:

Bảng 4.2 Bảng thông tin dữ liệu thu thập từ bản mẫu Superblock của EXT4

Offset	Kích thước	Tên	Hex	Dec
0x400	4	Tổng số Inode	0x0001DD30	122160
0x404	4	Tổng số Block	0x00077359	488281
0x40C	4	Số lượng Block còn trống	0x00072BAE	469934

0x410	4	số lượng Inode còn trống	0x0001DD17	122135
0x414	4	Block dữ liệu đầu tiên	0x00000000	0
0x418	4	Kích thước của Block [$2^{(10+X)}$]	0x00000002	$2^{(10+2)}$ = 4096
0x41C	4	Kích thước của cluster [$2^{(10+X)}$]	0x00000002	$2^{(10+2)}$ = 4096
0x420	4	Số lượng Block có trong mỗi group	0x00000800	2048
0x424	4	Số lượng cluster có trong mỗi group	0x00000800	2048
0x428	4	số lượng Inode có trong mỗi group	0x00001FD0	8144
0x488	64	Vị trí mount	/media/vinh/0f0e564e-3e01-4930-80fa-430d4a4675b9	
0x520	60	Vị trí bắt đầu của journal(nếu journaling được bật)	0x030000	196608

4.4.2 Block Group Descriptor

Block Group Descriptor là một thành phần quan trọng của hệ thống tập tin EXT4, mỗi hệ thống tập tin EXT4 sẽ bao gồm nhiều Block Group Descriptor, cung cấp thông tin về cấu trúc và trạng thái của từng Block Group trong hệ thống tập tin.

Các Block Group Descriptor được lưu trữ trong Block nằm ngay sau Superblock . Mỗi Block Group Descriptor mặc định sẽ có kích thước 32 bytes(nếu tính năng “64bit “ được bật thì kích thước của mỗi Block Group Descriptor sẽ được mở rộng thành 64

bytes) và chứa lần lượt các thông tin như vị trí của Block Bitmap, Inode Bitmap, Inode Table, và số lượng Block và Inode trống trong Block Group đó. dưới đây là bảng Offset cụ thể của các thông tin có được từ mỗi Block Group descriptor.

Bảng 4.3 Bảng thông tin dữ liệu Block Group Descriptor 64 bytes của EXT4 [13]

	Kích thước (bytes)	Nội dung
0x0	4	32-bits thấp vị trí của Block Bitmap
0x4	4	32-bits thấp vị trí của Inode Bitmap
0x8	4	32-bits thấp vị trí của Inode Table
0xC	2	16-bits thấp số lượng Block còn trống
0xE	2	16-bits thấp số lượng Inode còn trống
0x10	2	16-bits thấp số lượng Directory
0x12	2	Cờ dùng để hỗ trợ hệ thống quản lý
0x1C	2	16-bits thấp số lượng Inode chưa sử dụng
Trường này tồn tại nếu tính năng 64 bit được bật và kích thước của group descriptor >32		
0x20	4	32-bits cao vị trí của Block Bitmap
0x24	4	32-bits cao vị trí của Inode Bitmap
0x28	4	32-bits cao vị trí của Inode Table
0x2C	2	16-bits cao số lượng Block còn trống
0x2E	2	16-bits cao số lượng Inode còn trống

0x30	2	16-bits cao số lượng Directory
0x32	2	16-bits cao số lượng Inode chưa sử dụng

Dưới đây là hình ảnh ví dụ về Block Group Descriptor với tính năng “64bit” bật::

```

Offset(h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F Decoded text
00001000 F0 00 00 00 FF 00 00 00 0E 01 00 00 12 61 B6 1F 8...ÿ.....a¶.
00001010 05 00 04 00 00 00 00 00 52 72 48 CC B5 1F AB 70 .....RrHİp.«p
00001020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001030 00 00 00 00 00 00 00 00 D8 0A CA 2F 00 00 00 00 .....Ø.Ê/....
00001040 F1 00 00 00 00 01 00 00 0B 03 00 00 FE 7A CE 1F ñ.....pzİ.
00001050 02 00 04 00 00 00 00 00 04 21 24 F3 CE 1F 02 4C .....!$óİ..L
00001060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001070 00 00 00 00 00 00 00 00 4B 41 80 E1 00 00 00 00 .....KA€á....
00001080 F2 00 00 00 01 01 00 00 08 05 00 00 00 80 CE 1F ò.....€İ.
00001090 02 00 04 00 00 00 00 00 55 2E 24 F3 CE 1F 4A 7A .....U.$óİ.Jz
000010A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000010B0 00 00 00 00 00 00 00 00 F6 61 80 E1 00 00 00 00 .....öa€á....
000010C0 F3 00 00 00 02 01 00 00 05 07 00 00 10 7F D0 1F ó.....Đ.
000010D0 00 00 07 00 00 00 00 00 00 00 00 00 00 1F BB 49 .....Đ.»I
000010E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000010F0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

Hình 4.3 Bản mẫu Block Group Descriptor 64-bytes của EXT4

Như đã biết rằng mỗi Block Group Descriptor có kích thước là 64bytes vậy thì Block Group Descriptor của Block Group đầu tiên sẽ bắt đầu từ Offset(h) 1000 đến 103F:

```

Offset(h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F Decoded text
00001000 F0 00 00 00 FF 00 00 00 0E 01 00 00 12 61 B6 1F 8...ÿ.....a¶.
00001010 05 00 04 00 00 00 00 00 52 72 48 CC B5 1F AB 70 .....RrHİp.«p
00001020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001030 00 00 00 00 00 00 00 00 D8 0A CA 2F 00 00 00 00 .....Ø.Ê/....

```

Hình 4.4 Bản mẫu Block Group Descriptor 64-bytes

Sau khi xác định được Block Group Descriptor thì ta có thể tiến hành đọc các giá trị quan trọng.

Bảng 4.4 Bảng thông tin dữ liệu thu thập từ bản mẫu của Block Group Descriptor 64 bytes của EXT4

Offset	Kích thước (bytes)	Nội dung	Hex
--------	-----------------------	----------	-----

0x1000	4	32-bits thấp vị trí của Block Bitmap	0x000000F0
0x1004	4	32-bits thấp vị trí của Inode Bitmap	0x000000FF
0x1008	4	32-bits thấp vị trí của Inode Table	0x0000010E
0x100C	2	16-bits thấp số lượng Block còn trống	0x6112
0x100E	2	16-bits thấp số lượng Inode còn trống	0x1FB6
0x1010	2	16-bits thấp số lượng Directory	0x0005
0x1012	2	Cờ dùng để hỗ trợ hệ thống quản lý	0x0004
0x101C	2	16-bits thấp số lượng Inode chưa sử dụng	0x1FB5
0x1020	4	32-bits cao vị trí của Block Bitmap	0x00000000
0x1024	4	32-bits cao vị trí của Inode Bitmap	0x00000000
0x1028	4	32-bits cao vị trí của Inode Table	0x00000000
0x102C	2	16-bits cao số lượng Block còn trống	0x0000
0x102E	2	16-bits cao số lượng Inode còn trống	0x0000
0x1030	2	16-bits cao số lượng Directory	0x0000
0x1032	2	16-bits cao số lượng Inode chưa sử dụng	0x0000

Thông qua các giá trị ta có thể xác định vị trí của các thành phần quan trọng của một Block Group như:

- Vị trí của Block Bitmap = 32-bits cao vị trí của Block Bitmap + 32-bits thấp vị trí của Block Bitmap = $0x00000000 + 0x000000F0 = 0x00000000000000F0$

- Vị trí của Inode Bitmap = 32-bits cao vị trí của Inode Bitmap + 32-bits thấp vị trí của Inode Bitmap = $0x00000000 + 0x000000FF = 0x00000000000000FF$

- Vị trí của Inode Table = 32-bits cao vị trí của Inode Table + 32-bits thấp vị trí của Inode Table = $0x00000000 + 0x0000010E = 0x000000000000010E$

Đối với các giá trị vị trí, trong EXT4 được biểu thị bằng Block. Vì vậy giả sử ta có vị trí của Block Bitmap là $0xF0$ thì tức là Block Bitmap của Block Group này nằm ở Block thứ $0xF0$ (hex). Qua đó nếu muốn tìm vị trí Offset bắt đầu này ta chỉ cần nhân nó với kích thước của Block

- Offset bắt đầu của Block Bitmap = $0xF0(\text{hex}) \times 0x1000(\text{hex}) = 0xF0000(\text{hex})$
- Offset bắt đầu của Inode Bitmap = $0xFF(\text{hex}) \times 0x1000(\text{hex}) = 0xFF000(\text{hex})$
- Offset bắt đầu của Inode Table = $0x10E(\text{hex}) \times 0x1000(\text{hex}) = 0x10E000(\text{hex})$

Trong hệ thống tập tin EXT4, nếu tính năng “64bit” không được bật thì mỗi Block Group Descriptor sẽ chỉ có kích thước 32 bytes, dưới đây là 1 ví dụ về nó:

```
Offset(h)  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  Decoded text
000001000 03 00 00 00 13 00 00 00 23 00 00 00 D8 6F EF 0F .....#...0i.
000001010 02 00 04 00 00 00 00 00 00 00 00 00 EB 0F 49 6D .....ë.Im
000001020 04 00 00 00 14 00 00 00 23 01 00 00 38 5F 00 10 .....#...8_..
000001030 00 00 05 00 00 00 00 00 00 00 00 00 00 10 33 6A .....3j
000001040 05 00 00 00 15 00 00 00 23 02 00 00 00 00 00 10 .....#.....
```

Hình 4.5 Bản mẫu Block Group Descriptor 32-bytes của EXT4

Với kích thước 32 bytes vậy thì Block Group Descriptor của Block Group đầu tiên là:

```
Offset(h)  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  Decoded text
000001000 03 00 00 00 13 00 00 00 23 00 00 00 D8 6F EF 0F .....#...0i.
000001010 02 00 04 00 00 00 00 00 00 00 00 00 EB 0F 49 6D .....ë.Im
```

Hình 4.6 Bản mẫu Block Group Descriptor 32-bytes của Block Group đầu tiên

Do chỉ có kích thước 32 bytes nên khi xác định các giá trị ta sẽ không xác định các giá trị bits cao:

Bảng 4.5 Bảng thông tin dữ liệu Block Group Descriptor 32 bytes của EXT4

Offset	Kích thước (bytes)	Nội dung	Hex

0x1000	4	32-bits thấp vị trí của Block Bitmap	0x00000003
0x1004	4	32-bits thấp vị trí của Inode Bitmap	0x00000013
0x1008	4	32-bits thấp vị trí của Inode Table	0x00000023
0x100C	2	16-bits thấp số lượng Block còn trống	0x6FD8
0x100E	2	16-bits thấp số lượng Inode còn trống	0x0FEF
0x1010	2	16-bits thấp số lượng Directory	0x0002
0x1012	2	Cờ dùng để hỗ trợ hệ thống quản lý	0x0004
0x101C	2	16-bits thấp số lượng Inode chưa sử dụng	0x0FEB

Vì không có các bits cao nên vị trí của các thành phần sẽ dùng luôn giá trị của 32-bits thấp:

- Vị trí của Block Bitmap = 0x00000003
- vị trí của Inode Bitmap = 0x00000013
- vị trí của Inode Table = 0x00000023

Giả sử ta có kích thước Block của hệ thống tập tin này là 4096(dec) = 0x1000(hex) thì ta có được vị trí Offset bắt đầu của các thành phần là:

- Offset bắt đầu của Block Bitmap = 0x3000
- Offset bắt đầu của Inode Bitmap = 0x13000
- Offset bắt đầu của Inode Table = 0x23000

4.4.3 Block Bitmap

Block Bitmap là một cấu trúc dữ liệu quan trọng trong hệ thống tệp EXT4, đóng vai trò quan trọng trong việc quản lý việc phân bổ và giải phóng các khối dữ liệu. Chức năng chủ yếu của Block Bitmap là quản lý và phân bổ các Block. Block Bitmap theo dõi trạng thái của các Block trong Block Group. Mỗi bit trong Block Bitmap đại diện cho một khối trong nhóm, cho biết khối đó đang được sử dụng (1) hay trống (0).

Bằng cách theo dõi các khối đã sử dụng và còn trống, hệ thống tệp có thể phân bổ không gian lưu trữ một cách hiệu quả, giảm thiểu lãng phí và tối ưu hóa hiệu suất. không chỉ vậy, Quản lý việc phân bổ và giải phóng khối một cách thông minh giúp giảm thiểu sự phân mảnh, đảm bảo dữ liệu được lưu trữ liên tục và dễ truy cập.

Mỗi Block Group trong hệ thống tệp EXT4 có một Block Bitmap riêng. Block Bitmap sẽ được lưu trữ ở Block Group Descriptor của Block Group, giúp hệ thống tệp dễ dàng truy cập và cập nhật. Khi một tệp mới được tạo hoặc mở rộng, hệ thống tệp sẽ cập nhật Block Bitmap để phản ánh các khối mới được sử dụng. Tương tự, khi một tệp bị xóa hoặc giảm kích thước, các khối được giải phóng và Block Bitmap được cập nhật để đánh dấu các khối đó là trống.

Về mặt cấu trúc, Block Bitmap là một mảng các bit, với mỗi bit tương ứng với một Block trong Block Group. Ví dụ, nếu bit thứ n có giá trị 1, điều đó có nghĩa là Block thứ n đang được sử dụng. Ngược lại, nếu bit có giá trị 0, Block đó đang trống và có thể được phân bổ. [14]

Giả sử ta có 1 phần của bảng Block Bitmap như sau

```
000F03C0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF  YYYYYYYYYYYYYYYY
000F03D0  FF FF FF FF FF FF FF FF FF FF FF FF FF 3F 00 00  YYYYYYYYYYYY?..
```

Hình 4.7 Bản mẫu Block Bitmap của EXT4

Trong bảng Block Bitmap, mỗi 1 bytes sẽ biểu diễn trạng thái của 8 Block, qua ảnh trên ta có thể thấy 2 ví dụ như FF và 3F

3F => 00111111

Khối 1: trống

Khối 2: trống

Khối 3: Đã được sử dụng

Khối 4: Đã được sử dụng

Khối 5: Đã được sử dụng

Khối 6: Đã được sử dụng

Khối 7: Đã được sử dụng

Khối 8: Đã được sử dụng

FF => 11111111

Khối 1: Đã được sử dụng

Khối 2: Đã được sử dụng

Khối 3: Đã được sử dụng

Khối 4: Đã được sử dụng

Khối 5: Đã được sử dụng

Khối 6: Đã được sử dụng

Khối 7: Đã được sử dụng

Khối 8: Đã được sử dụng

4.4.4 Inode Bitmap

Inode Bitmap trong hệ thống tệp EXT4 là một cấu trúc dữ liệu tương tự như Block Bitmap, nhưng thay vì quản lý các khối dữ liệu, chức năng của nó là quản lý các Inode. Tương tự như Block Bitmap, mỗi bit trong Inode Bitmap đại diện cho trạng thái của một Inode trong Block Group.

4.4.5 Inode Table

Inode Table là một thành phần quan trọng trong hệ thống tệp, đặc biệt là trong hệ thống tệp EXT4. Inode Table lưu trữ các Inode, mỗi Inode chứa thông tin về một tệp hoặc thư mục trong hệ thống tệp. Mỗi Block Group trong hệ thống tệp có một bảng Inode riêng.

Inode Table giúp hệ thống tệp quản lý không gian đĩa bằng cách giữ thông tin về các khối dữ liệu được phân bổ cho từng tệp và thư mục. Nó cũng được dùng kết hợp với Inode Bitmap để theo dõi trạng thái của các Inode (sử dụng hoặc trống) để hệ thống tệp có thể phân bổ lại các Inode trống khi cần thiết.

4.4.6 Inode

Trong hệ thống tệp EXT4, một Inode (index node) là một cấu trúc dữ liệu quan trọng được sử dụng để lưu trữ thông tin về một tệp hoặc thư mục. Mỗi tệp hoặc thư mục trên hệ thống tệp EXT4 đều có ít nhất một Inode tương ứng.

Inode không lưu trữ tên tệp. Tên tệp được lưu trữ trong các Directory entry, nơi chúng liên kết với các Inode tương ứng.

Hệ thống tệp EXT4 sử dụng Inode để quản lý tệp và thư mục hiệu quả, cho phép truy xuất nhanh chóng và dễ dàng các thuộc tính của tệp mà không cần phải đọc toàn bộ nội dung của tệp.

Thông tin các dữ liệu quan trọng mà Inode lưu trữ nằm trong bảng sau:

Bảng 4.6 Bảng thông tin dữ liệu Inode của EXT4 [13]

Offset	Kích thước (bytes)	Nội dung
--------	-----------------------	----------

0x0	2	(i_mode) Phần này giúp xác định loại của Inode(tập tin, thư mục,...) và các quyền truy cập
0x2	2	16-bits thấp uid của người dùng chủ Inode (owner UID)
0x4	4	32-bits thấp giá trị kích thước (tính theo bytes)
0x8	4	Thời gian truy cập lần cuối(tính theo thời gian UNIX)
0xC	4	Thời gian thay đổi Inode lần cuối (tính theo thời gian UNIX)
0x10	4	Thời gian sửa đổi dữ liệu lần cuối (tính theo thời gian UNIX)
0x14	4	Thời gian xoá Inode (tính theo thời gian UNIX, chỉ bắt đầu tính thời gian khi số lượng hard link trở về 0)
0x18	2	16-bits thấp id của nhóm người dùng (GID)
0x1A	2	Số lượng hard link của Inode
0x1C	4	32-bits thấp số lượng Block
0x20	4	Cờ xác định một số các tính chất của Inode (i_flag)
0x28	60	Trường xác định vị trí dữ liệu của Inode (I_Block)
0x64	4	Phiên bản của file (dùng để hỗ trợ NFS)
0x84	4	Phần mở rộng của trường thời gian thay đổi Inode lần cuối (cung cấp thời gian chính xác dưới giây)

0x88	4	Phần mở rộng của trường thời gian sửa đổi dữ liệu lần cuối (cung cấp thời gian chính xác dưới giây)
0x8C	4	Phần mở rộng của trường thời gian truy cập lần cuối (cung cấp thời gian chính xác dưới giây)
0x90	4	Thời gian file được tạo (tính theo thời gian UNIX)
0x94	4	Phần mở rộng của trường thời gian file được tạo (cung cấp thời gian chính xác dưới giây)

4.4.6.1 Nội dung I_mode

Trong hệ thống tệp EXT4, I_mode là một trường trong cấu trúc Inode được sử dụng để lưu trữ các thông tin quan trọng về quyền truy cập và kiểu của tệp hoặc thư mục. Trường này chứa các bit chỉ định quyền truy cập (read, write, execute) cho chủ sở hữu, nhóm và những người khác, cũng như chỉ định kiểu của tệp (tệp thông thường, thư mục, liên kết, v.v.).

Để có thể xác định được kiểu tệp và quyền truy cập người ta dựa vào bảng sau mà phân tích:

Bảng 4.7 Bảng phân tích giá trị I_mode [13]

Giá trị	Nội dung
0x1	Người dùng khác cũng có thể thực thi
0x2	Người dùng khác cũng có thể ghi
0x4	Người dùng khác cũng có thể đọc
0x8	người dùng trong nhóm có thể thực thi
0x10	người dùng trong nhóm có thể ghi
0x20	người dùng trong nhóm có thể đọc

0x40	Người dùng chủ sở hữu có thể thực thi
0x80	Người dùng chủ sở hữu có thể ghi
0x100	Người dùng chủ sở hữu có thể đọc
0x200	Sticky bit (được thiết lập trên một thư mục ngăn không cho người dùng xóa hoặc di chuyển các tệp mà họ không sở hữu, ngay cả khi họ có quyền ghi trên thư mục đó)
0x400	Set GID
0x800	Set UID
	Đây là các loại tệp loại trừ lẫn nhau:
0x1000	FIFO
0x2000	Character device
0x4000	Directory
0x6000	Block device
0x8000	Regular file
0xA000	Symbolic link
0xC000	Socket

Thông thường người ta sẽ biểu diễn các giá trị này thành các bit trong dạng nhị phân cụ thể để dễ dàng phân tích như sau:

Kiểu tệp (File Type):

- 0x1000 = 0001 0000 0000 0000: FIFO (named pipe)
- 0x2000 = 0010 0000 0000 0000: Tệp đặc biệt (Character device)
- 0x4000 = 0100 0000 0000 0000: Thư mục (Directory)
- 0x6000 = 0110 0000 0000 0000: Tệp đặc biệt khối (Block device)

- 0x8000 = 1000 0000 0000 0000: Tập thông thường (Regular file)
- 0xA000 = 1010 0000 0000 0000: Liên kết mềm (Symbolic link)
- 0xC000 = 1100 0000 0000 0000: Socket

Quyền truy cập:

Các bit thấp hơn của I_mode được sử dụng để thiết lập quyền truy cập cho chủ sở hữu (owner), nhóm (group), và những người khác (others). Các quyền này lần lượt bao gồm:

- r: Read (đọc)
- w: Write (ghi)
- x: Execute (thực thi)

Từ đó mỗi loại người dùng là chủ sở hữu (owner), nhóm (group), và những người khác (others) sẽ có 3 bit để biểu diễn cho phần quyền truy cập mà họ có

Ví dụ: ta có một I_mode có giá trị là “0x8180”

- 0x8180 = 1000 0001 1000 0000
- Kiểu tệp: “1000” (0000 0000 0000) = 0x8000 => đây là tệp thông thường(regular file)
- Quyền truy cập: 000(1 10)(00 0)(000)
- Quyền của chủ sở hữu(110)=>(rw-): Chủ sở hữu có thể đọc, ghi, không thể thực thi
- Quyền của nhóm(000)=>(---): không thể đọc, ghi hay thực thi
- Quyền của những người khác(000)=>(---): không thể đọc, ghi hay thực thi

4.4.6.2 Nội dung của I_Block

Trong hệ thống tệp EXT4, trường I_block trong Inode được sử dụng để lưu trữ thông tin về vị trí của các khối dữ liệu chứa nội dung của tệp. Tùy thuộc vào kích thước và loại tệp, nội dung của I_block có thể thay đổi, nhưng nói chung, nó được sử dụng để quản lý và truy cập dữ liệu tệp một cách hiệu quả.

Trường I_block có độ dài 60 byte và có thể chứa các thông tin khác nhau tùy theo loại tệp và cách thức lưu trữ dữ liệu. Có tổng cộng 4 loại thông tin có thể xuất hiện

trong 60 byte này, có thể xác định được loại thông tin nào sẽ xuất hiện bằng cách phân tích cờ (*i_flag*) cùng với xác định kiểu tệp trong (*i_mode*). [15]

Khi xác định kiểu tệp, nếu xác định được nếu kiểu tệp là symbolic links thì nội dung của *I_block* trong Inode này sẽ là symbolic links.

Khi phân tích cờ này giống với (*i_mode*) ta có thể xác định được 1 trong 2 tính chất là:

- Inode sử dụng inline data (*EXT4_INLINE_DATA_FL*), nếu có cờ này thì nội dung của *I_block* sẽ là inline data
- Inode sử dụng Extents (*EXT4_EXTENTS_FL*), nếu có cờ này thì *I_block* của Inode này sẽ chứa Extent tree.

Nếu kiểu tệp không phải symbolic links, lại đồng thời không xuất hiện 2 cờ trên thì tức là *I_block* này sẽ chứa Direct/Indirect Block Addressing(Block mapping).

Symbolic links

Symbolic Links, còn được gọi là liên kết mềm, là một loại tệp đặc biệt trong hệ thống tệp Unix/Linux. Liên kết mềm chứa đường dẫn tới một tệp hoặc thư mục khác, cho phép bạn tạo các liên kết tới các tệp hoặc thư mục mà không cần sao chép chúng. Đây là một công cụ hữu ích để quản lý tệp và thư mục một cách linh hoạt và tiện lợi.

Tác dụng chính của symbolic links này là chứa đường dẫn tới tệp hoặc thư mục đích, không phải nội dung thực tế của tệp hay thư mục đó.

Inline data

Inline Data là một tính năng trong hệ thống tệp EXT4 cho phép lưu trữ dữ liệu của tệp trực tiếp trong Inode. Điều này giúp tăng hiệu suất truy cập tệp và tiết kiệm không gian đĩa đối với các tệp nhỏ.

Direct/Indirect Block Addressing (Block mapping)

Trong hệ thống tệp EXT4, dữ liệu của tệp được lưu trữ trong các khối dữ liệu. Để quản lý và truy xuất các khối dữ liệu này, EXT4 sử dụng một cấu trúc gọi là Direct và Indirect Block Addressing (hay còn gọi là Block Mapping). Đây là một phương pháp truyền thống để theo dõi vị trí của các khối dữ liệu trong hệ thống tệp.

Cấu trúc cụ thể của Block mapping trong I_block chứa 15 con trỏ để quản lý khối dữ liệu:

- Pointer[0] đến Pointer[11]: Các con trỏ trực tiếp (Direct Pointers)
- Pointer_1[12]: Con trỏ gián tiếp cấp một (Single Indirect Pointer)
- Pointer_2[13]: Con trỏ gián tiếp cấp hai (Double Indirect Pointer)
- Pointer_3[14]: Con trỏ gián tiếp cấp ba (Triple Indirect Pointer)

Đối với Block mapping, hệ thống đơn giản là sử dụng 12 con trỏ trực tiếp đầu dẫn đến data trước tiên, nếu không đủ nó sẽ triển khai con trỏ gián tiếp cấp 1, con trỏ này sẽ chứa vị trí đến Block chứa hàng loạt các con trỏ trực tiếp. Tiếp tục vẫn không đủ thì nó sẽ dùng tới con trỏ gián tiếp cấp 2, con trỏ này lại chứa vị trí của 1 Block chứa hàng các con trỏ gián tiếp cấp 1. Tương tự vậy đối với con trỏ gián tiếp cấp 3.

Extent tree & Extent

Trong hệ thống file EXT4, Extent tree là một cấu trúc dữ liệu “chính” được sử dụng để quản lý cách các tệp được lưu trữ trên đĩa cứng. Mục đích chính của Extent tree là cung cấp một phương pháp hiệu quả và tối ưu hóa không gian lưu trữ, đồng thời giảm thiểu độ phân mảnh dữ liệu.

Trong EXT4, một Extent là một khối dữ liệu liên tục trên đĩa cứng. Mỗi Extent được định nghĩa bởi ba yếu tố chính: địa chỉ khối đầu tiên trên đĩa, số lượng khối liên tục, và số khối logic bắt đầu. Điều này giúp cải thiện hiệu suất bởi vì hệ điều hành có thể đọc hoặc ghi một lượng lớn dữ liệu mà không cần phải chuyển đầu đọc ghi nhiều lần trên bề mặt đĩa.

Cấu trúc của Extent tree có thể thay đổi để tương thích với các trường hợp kích thước của dữ liệu cần lưu trữ:

Extent Header: Đây là phần đầu tiên của mỗi node trong Extent tree, là phần chắc chắn xuất hiện nếu Inode dùng Extent tree, chứa thông tin về magic number của Extent tree (0xF30A), trường này giúp xác định chính xác trường I_block có chứa Extent tree. Bên cạnh đó, Extent Header này cũng cung cấp cho ta về số lượng Extents hiện có trong node, số lượng tối đa có thể chứa, độ sâu của cây, và thông tin thế hệ.

Cấu trúc của một Extent tree Header sẽ chiếm 12 bytes đầu tiên của I_block với thông tin cụ thể như sau:

Bảng 4.8 Bảng thông tin dữ liệu Extent tree Header [13]

Offset	Kích thước (bytes)	Nội dung
0x0	2	Magic number, 0xF30A, trường này giúp hệ thống xác định được đây đúng là 1 Extent tree
0x2	2	Số lượng Extents hoặc Extent index entries hiện tại trong node. Đối với leaf nodes, đây là số lượng Extents thực tế lưu trong node đó; đối với Internal Nodes, nó đại diện cho số lượng con trỏ đến các nodes khác.
0x4	2	Số lượng tối đa các entries mà node có thể chứa. Giá trị này phụ thuộc vào kích thước Block của hệ thống file và kích thước của mỗi entry.
0x6	2	Độ sâu của node trong Extent tree. Một giá trị bằng 0 biểu thị rằng node là một leaf node, chứa các Extent entries trực tiếp mô tả dữ liệu. Một giá trị lớn hơn 0 biểu thị rằng node là một internal node, và các entries trong đó là con trỏ đến các nodes khác.
0x8	4	Một giá trị thế hệ, thường được sử dụng trong các quá trình như snapshotting để quản lý các phiên bản của dữ liệu và cấu trúc. Giá trị này giúp theo dõi các thay đổi trong Extent tree qua thời gian.

Thông qua Extent Header, ta xác định được rằng node này là internal node hay là leaf node

- **Internal Nodes:** Nếu tệp lớn đòi hỏi nhiều Extents hơn số có thể chứa trong một leaf node, Extent tree sẽ bao gồm nhiều cấp độ của Internal Nodes, mỗi node chứa các con trỏ đến các node khác. Mỗi entry sẽ có kích thước 12 bytes, trừ đi 12 bytes ban đầu dành cho Header, tức sẽ chỉ có thể có tối đa 4 entry tồn tại trong I_Block, mỗi entry của internal node sẽ có cấu trúc như sau:

Bảng 4.9 Bảng thông tin dữ liệu internal node thuộc Extent tree [13]

Offset	Kích thước (bytes)	Nội dung
0x0	4	Số khối logic tối thiểu mà các entries trong node con tham chiếu đến. Điều này cho phép Extent tree định vị nhanh chóng các phần dữ liệu cụ thể dựa trên chỉ số khối
0x4	4	32-bits thấp của địa chỉ Block trên đĩa của node con. Đây là một phần của địa chỉ vật lý mà ở đó node con được lưu trữ.
0x8	2	16-bits cao của địa chỉ Block trên đĩa của node con, cho phép địa chỉ vượt quá giới hạn 32-bit, hỗ trợ không gian địa chỉ lớn hơn.
0x10	2	Các byte không sử dụng, dành riêng cho việc mở rộng trong tương lai hoặc để duy trì căn chỉnh.

- **Leaf Nodes:** Chứa các Extent entries thực tế, mỗi entry mô tả một Extent cụ thể trên đĩa, cung cấp thông tin vị trí về dữ liệu mà Inode này đang chỉ tới. Tương tự

như internal node, node này cũng chứa Extent Header và tối đa 4 entry, mỗi entry của leaf node sẽ có cấu trúc như sau:

Bảng 4.10 Bảng thông tin dữ liệu leaf node thuộc Extent tree [13]

Offset	Kích thước (bytes)	Nội dung
0x0	4	Số khối logic bắt đầu của Extent trong tệp. Đây là vị trí bắt đầu của Extent trong không gian logic của tệp
0x4	2	Số lượng khối trong Extent. Giá trị này cũng có thể chứa các cờ (flags) chỉ ra các thuộc tính nhất định của Extent, như có bị mã hóa, nén, hoặc có bị lỗi
0x6	2	16-bits cao của địa chỉ khối đầu tiên trên đĩa, cho phép địa chỉ có thể mở rộng tới không gian lớn hơn
0x8	4	32-bits thấp của địa chỉ khối đầu tiên trên đĩa, cho phép địa chỉ có thể mở rộng tới không gian lớn hơn

Giả sử ta có 1 I_block chứa Extent tree như sau:

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0010E120	00	00	08	00	0C	00	00	00	0A	F3	01	00	04	00	00	00	ố.....
0010E130	00	00	00	00	00	00	00	00	01	00	00	00	E1	1E	00	00	á.....
0010E140	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010E150	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010E160	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Hình 4.8 Bản mẫu I_block trong Inode

Trước hết ta phải phân tích Extent Header, 12 bytes đầu tiên của I_Block:

“0A F3 01 00 04 00 00 00 00 00 00 00”

Bảng 4.11 Bảng thông tin dữ liệu thu thập được từ I_block mẫu

Offset	Kích thước	Nội dung	Giá trị
--------	------------	----------	---------

	(bytes)		
0x0	2	Magic number	0xF30A
0x2	2	Số lượng Extents hoặc Extent index entries hiện tại trong node.	0x0001
0x4	2	Số lượng tối đa các entries mà node có thể chứa.	0x0004
0x6	2	Độ sâu của node trong Extent tree.	0x0000
0x8	4	Giá trị thể hệ	0x000000 00

Đầu tiên ta có thể thấy được rằng magic number của Header này là 0xF30A, tức là đây đúng là 1 Extent tree, sau đó giá trị cần được quan tâm tiếp theo là độ sâu của cây, thông qua giá trị này ta có thể phân biệt được rằng node này là internal node hay là leaf node từ đó có thể tiến hành đọc các entry còn lại trong I_Block. Ở đây, độ sâu này bằng “0” nên ta có thể chắc chắn rằng đây là 1 leaf node. Còn 2 giá trị về số lượng Extent hiện tại và số lượng entries tối đa mà node có thể chứa được dùng để ta có thể dễ dàng quản lý các thông tin liên quan.

Như đã phân tích trong bảng trên, trong I_block này chỉ có 1 entry duy nhất, mỗi entry sẽ chiếm 12 bytes. Vậy entry này sẽ có giá trị là:”00 00 00 00 01 00 00 00 E1 1E 00 00”.

Thông qua đó ta sẽ có bảng giá trị như sau:

Bảng 4.12 Bảng thông tin dữ liệu thu thập từ I_block entry mẫu

Offset	Kích thước (bytes)	Nội dung	Giá trị
0x0	4	Số khối logic bắt đầu của Extent trong	0x0000

		tệp.	
0x4	2	Số lượng khối trong Extent.	0x0001
0x6	2	16-bits cao của địa chỉ khối đầu tiên trên đĩa	0x0000
0x8	4	32-bits thấp của địa chỉ khối đầu tiên trên đĩa	0x00001EE1

Từ các giá trị thu được ta có thể biết được rằng có 1 khối trong Extent này cũng như vị trí của khối này là Block thứ 0x1EE1 (12-bits cao + 32-bits thấp = 0x0000 + 0x00001EE1). Nếu ta có kích thước Block là 0x1000 = 4096 bytes thì vị trí Offset cụ thể của Block này sẽ là 0x1EE1000.

4.4.7 Directory & Directory entries

Trong hệ thống file EXT4, một Directory là một loại đặc biệt của tệp chứa các Directory entries. Các Directory không chỉ đơn giản là những thư mục lưu trữ tệp, mà còn là cấu trúc dữ liệu cho phép tổ chức và quản lý dữ liệu trong hệ thống file. Mỗi Directory entry trong thư mục tham chiếu đến một tệp hoặc thư mục con khác và bao gồm thông tin về tên tệp và số Inode liên quan.

Cách Directory quản lý trong EXT4:

Tạo và Xóa: Khi một Directory mới được tạo, hệ thống file phân bổ một Inode cho nó và thiết lập một bản ghi trong bảng Inode. Khi Directory bị xóa, các entries trong đó cũng cần được xóa, và các Inode liên quan phải được giải phóng hoặc giảm số lượng liên kết.

Duyệt và Tìm kiếm: Khi truy cập vào một tệp từ một đường dẫn, hệ điều hành sẽ duyệt qua các Directory để tìm các entries phù hợp với mỗi phần của đường dẫn.

Độ Phân mảnh: Trong EXT4, directories có thể trở nên phân mảnh nếu có nhiều thay đổi (ví dụ, xóa nhiều tệp nhỏ). Hệ thống có thể cần tổ chức lại các entries để tối ưu hóa hiệu suất truy cập.

Mỗi Directory entry trong Directory là một cấu trúc chứa tên của một tệp hoặc thư mục và số Inode của tệp hoặc thư mục đó. Số Inode đó chỉ đến một bản ghi trong bảng Inode, nơi lưu trữ các metadata của tệp như quyền truy cập, chủ sở hữu, nhóm, kích thước tệp, và các thông tin quan trọng khác.

Một Directory entry cơ bản sẽ có cấu trúc như nhau:

Bảng 4.13 Bảng thông tin dữ liệu Directory entry cơ bản [13]

Offset	Kích thước (bytes)	Nội dung
0x0	4	Thứ tự Inode mà Directory này chỉ tới
0x4	2	Độ dài của Directory entry
0x6	1	Độ dài tên file
0x7	1	Kiểu tệp
0x8	Độ dài tên file	Tên file

Bên cạnh đó, còn có 1 cấu trúc khác của Directory entry nếu tính năng “filetype” không được bật:

Bảng 4.14 Bảng thông tin dữ liệu Directory entry nếu tính năng filetype tắt [13]

Offset	Kích thước (bytes)	Nội dung
0x0	4	Thứ tự Inode mà Directory này chỉ tới
0x4	2	Độ dài của Directory entry
0x6	2	Độ dài tên file

0x8	Độ dài tên file (char)	Tên file
-----	---------------------------	----------

Giả sử ta có 1 Directory cơ bản như sau:

```

Offset(h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F Decoded text
01EE1000 02 00 00 00 0C 00 01 02 2E 00 00 00 02 00 00 00 .....
01EE1010 0C 00 02 02 2E 2E 00 00 0B 00 00 00 14 00 0A 02 .....
01EE1020 6C 6F 73 74 2B 66 6F 75 6E 64 00 00 0C 00 00 00 lost+found.....
01EE1030 14 00 0B 01 6B 61 6E 65 6B 69 2E 6A 70 65 67 00 ....kaneki.jpeg.

```

Hình 4.9 Bản mẫu Directory của EXT4

Trong hệ thống file EXT4 và các hệ thống file Unix-like khác, các entries đặc biệt như ., .., và thư mục lost+found đóng vai trò quan trọng trong cấu trúc và bảo mật của hệ thống file, là các entry cơ bản sẽ có trong mỗi Directory.

- . (Dot): Entry này đại diện cho chính Directory hiện tại. Nó là một liên kết đến chính Directory đó, giúp các hoạt động và các lệnh có thể tham chiếu đến "thư mục hiện tại". Số Inode của entry . sẽ trùng với số Inode của Directory đó.
- .. (Dot Dot): Entry này đại diện cho thư mục cha của Directory hiện tại. Điều này cho phép người dùng và các ứng dụng điều hướng lên thư mục một cấp trên. Trong trường hợp của thư mục gốc (root), .. cũng trỏ đến chính nó, giống như ..
- lost+found trong các hệ thống file như EXT4 là một cơ chế phục hồi dữ liệu, sử dụng để lưu trữ các file hoặc phần của file không thể gắn với một thư mục cụ thể do lỗi hệ thống. Khi công cụ kiểm tra và sửa chữa hệ thống file (fsck) phát hiện dữ liệu lạc không có liên kết rõ ràng đến bất kỳ thư mục nào, nó sẽ đặt các dữ liệu này vào lost+found. Điều này cho phép người dùng có cơ hội khôi phục các file quan trọng mà có thể đã bị mất trong quá trình sự cố hệ thống.

Ngoài các entry cơ bản ở trên, ta có thể thấy còn 1 entry khác đó là "0C 00 00 00 14 00 0B 01 6B 61 6E 65 6B 69 2E 6A 70 65 67 00", thông qua phân tích ta có bảng giá trị sau:

Bảng 4.15 Bảng thông tin dữ liệu thu thập từ Directory entry mẫu

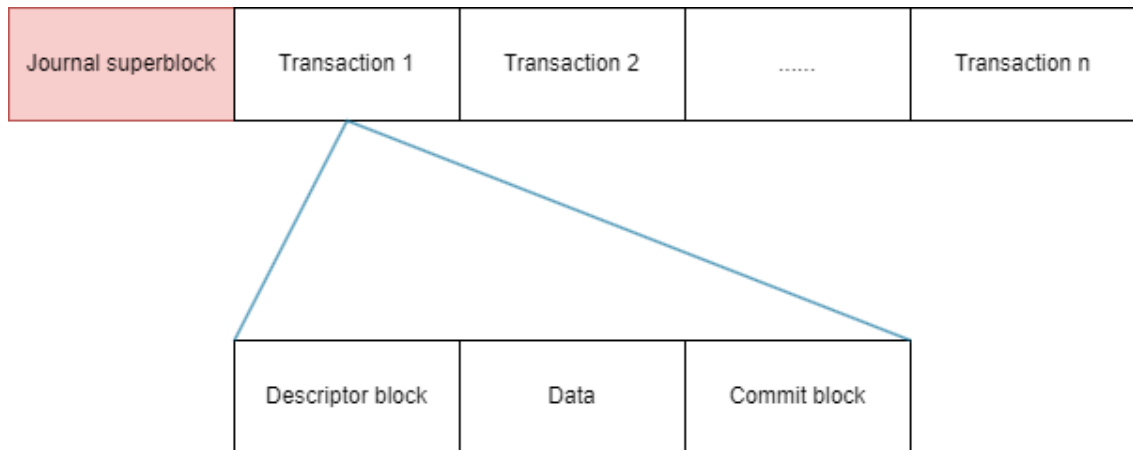
Offset	Kích thước (bytes)	Nội dung	Giá trị
0x0	4	Thứ tự Inode mà Directory này chỉ tới	0x0000000C
0x4	2	Độ dài của Directory entry	0x0014
0x6	1	Độ dài tên file	0x0B
0x7	1	Kiểu tệp	0x01
0x8	Độ dài tên file(char)	Tên file	0x0B 01 6B 61 6E 65 6B 69 2E 6A 70 65 67 00 (kaneki.jpeg)

4.4.8 Journal(jbd2)

Trong hệ thống file EXT4, journaling là một tính năng thiết kế nhằm đảm bảo tính toàn vẹn của dữ liệu và nâng cao khả năng phục hồi của hệ thống sau các sự cố như mất điện đột ngột hoặc lỗi hệ thống. Journaling giúp cải thiện độ tin cậy của hệ thống file bằng cách giảm thiểu rủi ro mất mát dữ liệu và hư hỏng file system.

Journaling trong EXT4 có mục đích chính là ghi lại các thay đổi sắp xảy ra trên hệ thống file trước khi chúng thực sự được áp dụng lên đĩa. Quá trình này được gọi là Write-Ahead Logging. Bằng cách ghi nhật ký trước (journaling) các thay đổi, hệ thống có thể sử dụng journal để phục hồi lại trạng thái trước khi sự cố xảy ra, đảm bảo dữ liệu không bị hỏng và hệ thống file vẫn nhất quán.

Về cấu trúc, một journal sẽ có một journal Superblock và rất nhiều transaction, mỗi transaction biểu thị cho mỗi lần hệ thống thực hiện ghi lại, cấu trúc của một transaction lại được chia thành 3 phần là descriptor Block, data Block, commit Block. Cấu trúc cụ thể có thể tham khảo hình dưới:



Hình 4.10 Mô hình cấu trúc journal của EXT4

Journal Block Header

Phần Header này chiếm 12 bytes đầu của mỗi Block thuộc journal, giúp xác định nó là loại Block nào trong journal. Mỗi Header có cấu trúc như sau:

Bảng 4.16 Bảng thông tin dữ liệu journal Block Header [13]

Offset	Kích thước (bytes)	Nội dung
0x0	4	Magic number của journal, 0xC03B3998
0x4	4	Loại Block, là 1 trong 5 loại sau: • 1 Descriptor • 2 Block commit record • 3 journal Superblock, v1 • 4 journal Superblock, v2 • 5 Block revocation records
0x8	4	ID của transaction

Journal super Block

Giống với Superblock, journal Superblock này giúp hệ thống quản lý các giao dịch trong journal và là phần đầu tiên được đọc khi hệ thống khởi động lại sau sự cố, đảm bảo rằng hệ thống file có thể phục hồi một cách chính xác.

Trong journal super Block, có 2 giá trị cần đặc biệt quan tâm đó là kích thước Block của journal và cờ tính năng không tương thích nếu journal Superblock này là v2, cờ này sẽ quyết định xem cấu trúc của descriptor Block

Journal descriptor Block

Trong cơ chế journaling của hệ thống file EXT4, Journal Descriptor Block đóng một vai trò trọng tâm bằng cách mô tả và liên kết các thay đổi dữ liệu với các Block dữ liệu cụ thể trong hệ thống file. Các Journal Descriptor Blocks là thành phần không thể thiếu trong quá trình ghi nhật ký các giao dịch, giúp hệ thống phục hồi dữ liệu một cách chính xác và hiệu quả sau sự cố.

Journal Descriptor Block chứa thông tin về các Block dữ liệu trong hệ thống file mà sẽ được thay đổi trong giao dịch đang được nhật ký. Mỗi journal descriptor Block gồm các trường sau:

Journal header	Tags 1	Tags 2		Tags n (end tag)
----------------	--------	--------	-------	------------------

Hình 4.11 Mô hình cấu trúc của một journal descriptor Block

Cấu trúc của nó bao gồm một Header cùng một hoặc một dãy các tags dùng để lưu trữ thông tin về những Block được ghi lại.

Một tags có 2 loại cấu trúc như sau tùy vào giá trị của cờ tính năng không tương thích:

- Nếu cờ JBD2_FEATURE_INCOMPAT_CSUM_V3 được đặt

Bảng 4.17 Bảng thông tin dữ liệu journal descriptor Block [13]

Offset	Kích thước (bytes)	Nội dung
0x0	4	32-bits thấp vị trí nơi Block sẽ được ghi lại trong journal data
0x4	4	Cờ đi kèm, cờ này chủ yếu để xác định rằng đầu là end tag

0x8	4	32-bits cao vị trí nơi Block sẽ được ghi lại trong journal data
0xC	4	Checksum

- Nếu cờ JBD2_FEATURE_INCOMPAT_CSUM_V3 không được đặt:

Bảng 4.18 Bảng thông tin dữ liệu journal descriptor Block v3

Offset	Kích thước (bytes)	Nội dung
0x0	4	32-bits thấp vị trí nơi Block sẽ được ghi lại trong journal data
0x4	2	Checksum
0x8	2	32-bits cao vị trí nơi Block sẽ được ghi lại trong journal data
0xC	4	checksum

Commit Block

Journal Commit Block là một thành phần quan trọng trong quá trình journaling, đánh dấu sự hoàn thành của một giao dịch. Commit Block cho phép hệ thống xác định rằng tất cả dữ liệu liên quan đến giao dịch đã được ghi một cách an toàn vào journal và có thể được áp dụng đến hệ thống file chính nếu cần.

Commit Block xác nhận rằng tất cả dữ liệu liên quan đến giao dịch đã được ghi đầy đủ và chính xác vào journal. Điều này là cần thiết để hệ thống có thể tiến hành áp dụng các thay đổi vào hệ thống file chính mà không gặp rủi ro mất mát hoặc hư hỏng dữ liệu.

Trong trường hợp sự cố như mất điện hoặc sập hệ thống, Commit Block cho phép hệ thống xác định chính xác những giao dịch nào đã hoàn tất và có thể được phục hồi

an toàn. Nếu một giao dịch không có Commit Block, hệ thống sẽ không cố gắng phục hồi dữ liệu từ giao dịch đó, do có thể dữ liệu chưa được ghi hoàn chỉnh.

Chương 5. CÁC PHƯƠNG PHÁP TỔNG QUÁT CHO AN TOÀN VÀ PHỤC HỒI DỮ LIỆU HÌNH ẢNH TRÊN ĐIỆN THOẠI ANDROID

5.1 Tổng quan về an toàn và phục hồi dữ liệu

An toàn dữ liệu là việc bảo vệ dữ liệu khỏi các mối đe dọa, nhằm đảm bảo tính bảo mật, toàn vẹn và sẵn sàng của dữ liệu. Trên điện thoại Android, dữ liệu lưu trữ có thể bao gồm hình ảnh, tài liệu, tin nhắn, và thông tin cá nhân. An toàn dữ liệu hình ảnh (DLHA) đòi hỏi các biện pháp đặc biệt để bảo vệ hình ảnh khỏi bị truy cập trái phép, mất mát hoặc hỏng hóc.

5.2 Các phương pháp tổng quát cho an toàn dữ liệu hình ảnh

Sao lưu (Backup):

Sao lưu dữ liệu là việc tạo ra các bản sao của dữ liệu quan trọng để có thể khôi phục lại trong trường hợp mất mát dữ liệu. Trên Android, sao lưu có thể được thực hiện thông qua các dịch vụ đám mây như Google Drive hoặc các ứng dụng sao lưu bên thứ ba.

Mã hóa (Encryption):

Mã hóa là quá trình biến đổi dữ liệu thành dạng mà chỉ có thể đọc được bằng cách giải mã, thường thông qua một khóa bảo mật. Trên Android, người dùng có thể mã hóa toàn bộ thiết bị hoặc chỉ mã hóa các tập tin cụ thể như hình ảnh để bảo vệ thông tin cá nhân.

Ẩn dữ liệu (Data Hiding):

Ẩn dữ liệu bao gồm việc giấu dữ liệu trong các định dạng không dễ nhận biết, như giấu ảnh trong các tập tin âm thanh hoặc video. Trên Android, có nhiều ứng dụng hỗ

trợ người dùng ẩn hình ảnh và video trong các thư mục bảo mật hoặc dưới các định dạng khác.

Sử dụng mật khẩu/sinh trắc học (Password/Biometric Authentication):

Sử dụng mật khẩu mạnh hoặc các phương pháp xác thực sinh trắc học (như vân tay, nhận diện khuôn mặt) để bảo vệ truy cập vào thiết bị và các ứng dụng chứa dữ liệu hình ảnh. Trên Android, các phương pháp này được tích hợp sâu vào hệ điều hành, cung cấp một lớp bảo mật mạnh mẽ cho người dùng.

Xóa vĩnh viễn không thể phục hồi

Khi cần loại bỏ dữ liệu một cách an toàn, đặc biệt là các hình ảnh nhạy cảm hoặc thông tin cá nhân, việc xóa dữ liệu thông thường không đảm bảo rằng dữ liệu đó không thể phục hồi. Để xóa vĩnh viễn dữ liệu, bạn có thể sử dụng các phương pháp như ghi đè dữ liệu, hủy vật lý hoặc phần mềm xóa an toàn. Ghi đè dữ liệu sử dụng phần mềm để viết lại các dữ liệu ngẫu nhiên hoặc mẫu lên các tập tin cần xóa, làm cho dữ liệu gốc không thể khôi phục được. Hủy vật lý bao gồm việc đập vỡ, đốt cháy hoặc nghiền nát các thiết bị lưu trữ như ổ cứng hoặc SSD để đảm bảo dữ liệu không thể phục hồi.

5.2.1 Sao lưu tập tin hình ảnh

5.2.1.1 Khái niệm, lợi ích và hạn chế của các phương pháp sao lưu

Khái niệm:

Sao lưu dữ liệu (Backup) là quá trình tạo bản sao dự phòng của dữ liệu hiện có nhằm bảo vệ dữ liệu trước các rủi ro như mất mát, hỏng hóc hoặc tấn công mạng. Các bản sao lưu có thể được lưu trữ tại cùng một vị trí hoặc tại các địa điểm khác nhau nhằm đảm bảo an toàn và khả năng phục hồi nhanh chóng khi cần thiết.

Lợi ích của sao lưu dữ liệu:

Bảo vệ dữ liệu: Sao lưu giúp bảo vệ dữ liệu quan trọng khỏi các rủi ro như hỏng hóc phần cứng, lỗi phần mềm, tấn công mạng và sai sót của con người.

Phục hồi nhanh chóng: Khi xảy ra sự cố, các bản sao lưu cho phép phục hồi dữ liệu nhanh chóng, giảm thiểu thời gian gián đoạn hoạt động.

Đảm bảo tính toàn vẹn của dữ liệu: Dữ liệu sao lưu được lưu trữ ở trạng thái nguyên vẹn, không bị thay đổi hoặc hỏng hóc theo thời gian.

Duy trì hoạt động kinh doanh: Sao lưu dữ liệu giúp duy trì hoạt động kinh doanh liên tục ngay cả khi gặp sự cố, tránh thiệt hại lớn về tài chính và uy tín.

Hạn chế của sao lưu dữ liệu:

Chi phí: Việc thiết lập và duy trì hệ thống sao lưu dữ liệu có thể tốn kém, đặc biệt đối với các doanh nghiệp lớn với lượng dữ liệu khổng lồ.

Thời gian: Quá trình sao lưu dữ liệu có thể mất thời gian, đặc biệt khi sao lưu dữ liệu lớn hoặc toàn bộ hệ thống.

Không đảm bảo hoàn toàn: Dù sao lưu có thể giảm thiểu rủi ro mất mát dữ liệu, nhưng không phải là giải pháp tuyệt đối. Một số tình huống như lỗi cấu hình sao lưu hoặc sự cố cùng lúc tại nhiều vị trí lưu trữ có thể gây mất mát dữ liệu.

Quản lý phức tạp: Việc quản lý các bản sao lưu dữ liệu đòi hỏi kiến thức chuyên môn và sự chú ý để đảm bảo dữ liệu luôn được cập nhật và sẵn sàng phục hồi.

Các phương pháp sao lưu phổ biến nhất hiện nay:

- **Sao lưu bằng ổ cứng gắn ngoài**

Lợi ích:

Chi phí thấp: Đầu tư ban đầu không quá cao và không cần trả phí duy trì hàng tháng.

Dễ sử dụng: Cắm và sao chép dữ liệu dễ dàng, không cần kiến thức kỹ thuật cao.

Hạn chế:

Dễ hỏng: Ổ cứng gắn ngoài có thể bị hỏng hóc, mất hoặc bị đánh cắp.

Không tự động: Phải sao lưu thủ công, dễ bị quên hoặc không đồng bộ hóa kịp thời.

- **Sao lưu internet**

Lợi ích:

Tự động: Sao lưu dữ liệu tự động qua internet, không cần can thiệp thủ công.

An toàn: Dữ liệu được mã hóa và bảo vệ bằng mật khẩu, giảm nguy cơ mất mát.

Hạn chế:

Phụ thuộc vào kết nối internet: Nếu kết nối internet không ổn định, quá trình sao lưu có thể bị gián đoạn.

Chi phí: Thường yêu cầu trả phí hàng tháng cho dịch vụ sao lưu.

- **Lưu trữ đám mây**

Lợi ích:

Truy cập mọi nơi: Dữ liệu có thể truy cập từ bất kỳ đâu có kết nối internet.

An toàn: Dữ liệu được lưu trữ trên các máy chủ an toàn, giảm nguy cơ mất mát do sự cố phần cứng.

Hạn chế:

Chi phí: Dịch vụ lưu trữ đám mây thường đi kèm với chi phí hàng tháng hoặc hàng năm.

Bảo mật: Dữ liệu có thể gặp nguy cơ bị truy cập trái phép nếu bảo mật không tốt.

5.2.1.2 Sao lưu tập tin hình ảnh bằng các công cụ có sẵn

- **Google Photos**

Lợi ích:

Tự động sao lưu: Google Photos cho phép tự động sao lưu hình ảnh từ điện thoại hoặc máy tính bảng, đảm bảo mọi hình ảnh mới đều được lưu trữ ngay lập tức.

Truy cập dễ dàng: Người dùng có thể truy cập hình ảnh từ bất kỳ thiết bị nào kết nối internet.

Tính năng tìm kiếm mạnh mẽ: Google Photos sử dụng trí tuệ nhân tạo để giúp tìm kiếm hình ảnh dễ dàng bằng cách sử dụng từ khóa, địa điểm, và khuôn mặt.

Hạn chế:

Dung lượng giới hạn: Phiên bản miễn phí của Google Photos có giới hạn dung lượng lưu trữ. Người dùng cần phải trả phí nếu muốn lưu trữ nhiều hơn.

Quyền riêng tư: Do dữ liệu được lưu trữ trên máy chủ của Google, có nguy cơ về quyền riêng tư và bảo mật thông tin cá nhân. Ví dụ như việc Google đã ‘lỡ tay’ xóa hoàn toàn tài khoản và dữ liệu sao lưu của UniSuper, một quỹ hưu trí lớn của Úc, do một lỗi cấu hình trong quá trình triển khai dịch vụ. Sự cố này đã khiến UniSuper bị

tê liệt hoạt động trong suốt 2 tuần, gây thiệt hại nghiêm trọng cho hơn 600.000 người dùng.[9]

- **Sao lưu đám mây**

Các dịch vụ sao lưu đám mây khác ngoài Google Photos bao gồm Dropbox, iCloud, OneDrive, và nhiều dịch vụ khác.

Lợi ích:

Đa dạng lựa chọn: Người dùng có nhiều lựa chọn phù hợp với nhu cầu và ngân sách cá nhân.

Truy cập từ xa: Hình ảnh có thể truy cập từ bất kỳ đâu với kết nối internet.

An toàn và bảo mật: Hầu hết các dịch vụ đều cung cấp mã hóa dữ liệu để đảm bảo an toàn.

Hạn chế:

Chi phí: Hầu hết các dịch vụ lưu trữ đám mây tính phí hàng tháng hoặc hàng năm, đặc biệt khi cần dung lượng lớn.

Phụ thuộc vào internet: Việc sao lưu và truy cập dữ liệu phụ thuộc vào tốc độ và độ ổn định của kết nối internet.

- **Sao lưu ngoại tuyến**

Lợi ích:

Chi phí thấp: Đầu tư ban đầu không cao, không có phí duy trì hàng tháng.

Kiểm soát hoàn toàn: Người dùng có toàn quyền kiểm soát và sở hữu thiết bị lưu trữ.

Hạn chế:

Dễ hỏng hóc: Thiết bị lưu trữ có thể bị hỏng hoặc mất.

Không tự động: Phải sao lưu thủ công, dễ bị quên hoặc không đồng bộ hóa kịp thời.

Khả năng di động hạn chế: Người dùng phải mang theo thiết bị nếu muốn truy cập hình ảnh khi di chuyển.

5.2.1.3 Sao lưu tập tin hình ảnh bằng công cụ tự xây dựng.

- **Hard link: Cách sao lưu trong hệ thống tập tin EXT4**

Khái niệm về hard link

Hard link là một khái niệm quan trọng trong hệ thống tập tin EXT4. Khi tạo một hard link đến một tệp, bạn thực sự tạo ra một liên kết trực tiếp giữa một tên tệp và một Inode, cấu trúc dữ liệu chứa thông tin về tệp như kích thước, quyền truy cập và vị trí dữ liệu trên đĩa. Hard link cho phép nhiều tên tệp trỏ đến cùng một Inode, điều này có nghĩa là chúng sẽ chia sẻ cùng một không gian lưu trữ trên đĩa và bất kỳ thay đổi nào trên tệp thông qua một hard link cũng sẽ ảnh hưởng đến tất cả các hard link khác.

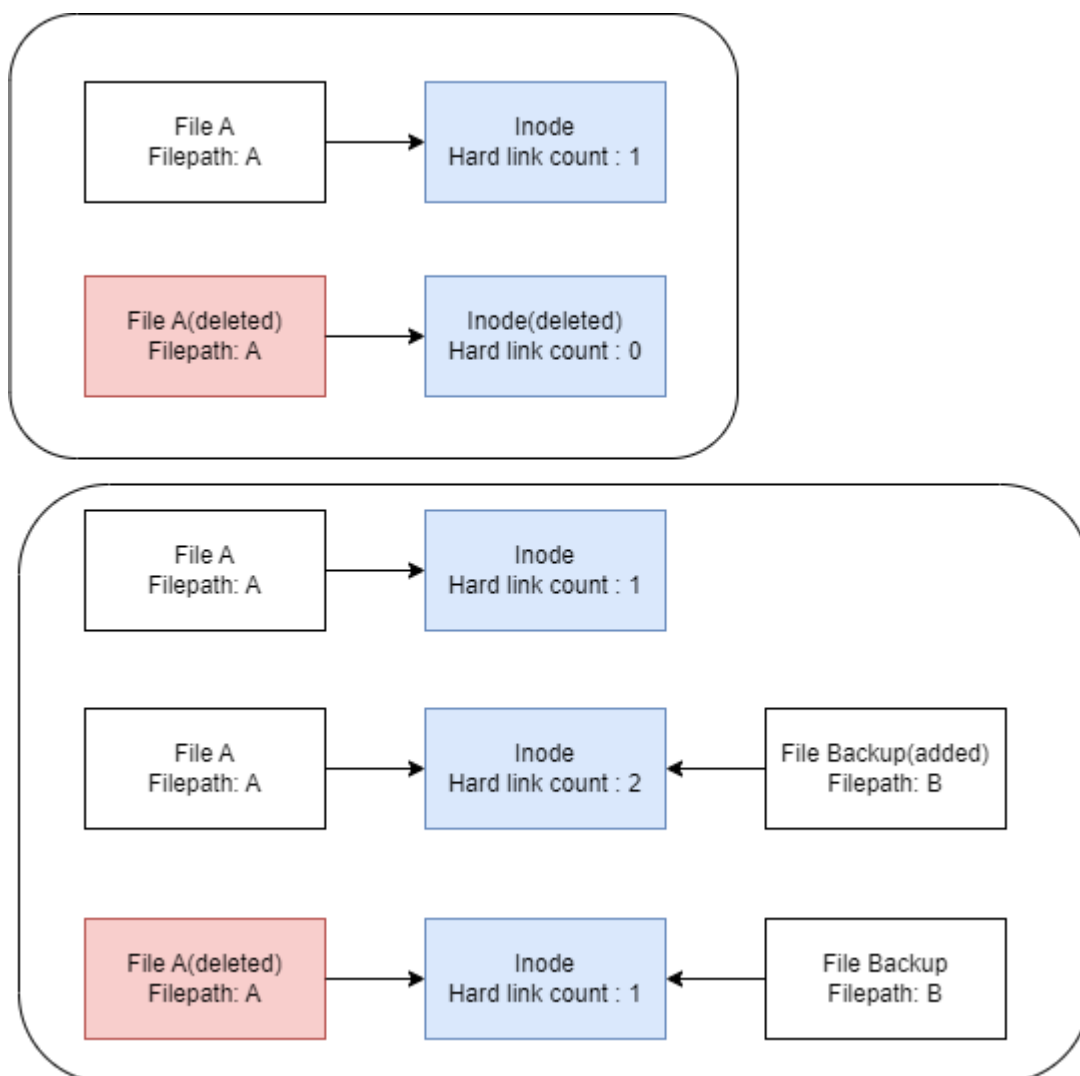
Ưu điểm của việc sử dụng hard link trong sao lưu hình ảnh

Hard link có nhiều điểm mạnh trong việc sao lưu dữ liệu nói chung cũng như dữ liệu ảnh nói riêng. Cụ thể có các ưu điểm sau đây:

- **Tiết kiệm không gian lưu trữ:** Mỗi hard link chỉ tạo ra một liên kết tới cùng một dữ liệu vật lý trên đĩa. Do đó, không có dữ liệu nào mới được tạo ra khi tạo hard link, giúp tiết kiệm không gian lưu trữ. Điều này rất quan trọng đối với các tập tin hình ảnh lớn có dung lượng cao.
- **Bảo toàn tính toàn vẹn dữ liệu:** Dữ liệu hình ảnh gốc và bản sao sao lưu đều liên kết trực tiếp đến cùng một dữ liệu vật lý. Bất kỳ thay đổi nào được thực hiện trên một bản sao sao lưu cũng sẽ được áp dụng trên tất cả các hard link khác. Điều này đảm bảo tính toàn vẹn của dữ liệu trong quá trình sao lưu và phục hồi.
- **Dễ dàng quản lý:** Vì tất cả các hard link trỏ đến cùng một tệp dữ liệu gốc, việc quản lý các bản sao sao lưu trở nên đơn giản. Bạn có thể truy cập vào bất kỳ bản sao nào từ bất kỳ vị trí nào trên hệ thống mà không cần sao chép hoặc di chuyển dữ liệu.
- **Hiệu suất cao:** Do không có dữ liệu mới được tạo ra khi tạo hard link, quá trình sao lưu và phục hồi trở nên nhanh chóng và hiệu quả. Thời gian và công sức cần thiết để tạo và duy trì các bản sao sao lưu cũng được giảm thiểu.
- **Tính hiệu quả:** Sử dụng hard link giúp tối ưu hóa việc sử dụng không gian lưu trữ và giảm thiểu sự lãng phí. Điều này rất quan trọng đối với việc quản lý hình ảnh, đặc biệt là khi có một lượng lớn các tập tin hình ảnh cần sao lưu.

Cách thức hoạt động của hard link trong việc khôi phục dữ liệu

Như đã biết về cấu trúc Inode trong phần giới thiệu về cấu trúc EXT4, ta biết được rằng trong Inode có 1 trường mang tên “hard link count” dùng để chứa số lượng hard link đang được gắn trên Inode này. Biết rằng nếu “hard link count” trở về 0 thì một Inode sẽ được cho là đã bị xoá. Ý tưởng ở đây là dùng cơ chế “hard link count” này để tạo ra một bản sao lưu. Theo cách này, chúng ta vẫn sẽ có thêm 1 bản sao lưu an toàn mà còn có thể tiết kiệm dung lượng.



Hình 5.1 Mô hình áp dụng hard link trong việc phục hồi dữ liệu

- **Sao lưu bằng hệ thống tự thiết kế**

Đề tài đưa ra thiết kế về một hệ thống quản lý tập tin để người dùng có thể sao lưu ảnh của cá nhân vào hệ thống. Điều này cho phép bức ảnh quý giá có một bản sao

của nó trong hệ thống này. Những tấm ảnh bị xóa trong thư viện sẽ không ảnh hưởng gì đến những tấm ảnh đã được lưu trong hệ thống.

5.2.2 Bảo vệ bằng mật khẩu và sinh trắc học

5.2.2.1 Sử dụng mật khẩu và các phương pháp sinh trắc học

Mật khẩu:

Mật khẩu là phương pháp bảo vệ thông tin cá nhân và dữ liệu bằng cách sử dụng một chuỗi ký tự mà chỉ người dùng mới biết. Đây là một trong những phương pháp bảo mật phổ biến và lâu đời nhất.

Phương pháp sinh trắc học:

Các phương pháp sinh trắc học bao gồm việc sử dụng các đặc điểm vật lý của người dùng như vân tay, khuôn mặt, giọng nói để xác thực và bảo vệ dữ liệu.

Vân tay:

Việc sử dụng vân tay để xác thực mang lại lợi ích nhanh chóng và tiện lợi, không cần phải nhớ mật khẩu. Tuy nhiên, phương pháp này có thể bị giả mạo bằng cách sao chép dấu vân tay.

Nhận diện khuôn mặt:

Nhận diện khuôn mặt là một phương pháp tiện lợi và nhanh chóng, giúp cải thiện trải nghiệm người dùng. Mặc dù vậy, đôi khi nhận diện khuôn mặt có thể không chính xác trong điều kiện ánh sáng yếu, dẫn đến nhận diện sai.

Nhận diện giọng nói:

Nhận diện giọng nói là phương pháp thân thiện và dễ sử dụng, đặc biệt trong các thiết bị không có màn hình. Tuy nhiên, phương pháp này có thể bị ảnh hưởng bởi tiếng ồn xung quanh hoặc sự thay đổi giọng nói của người dùng.

5.2.2.2 Cách thiết lập và quản lý bảo mật mật khẩu và sinh trắc học.

Việc thiết lập và quản lý mật khẩu cùng các phương pháp sinh trắc học đóng vai trò quan trọng trong việc bảo vệ thông tin cá nhân và dữ liệu quan trọng. Bằng cách áp dụng các biện pháp bảo mật phù hợp, người dùng có thể giảm thiểu rủi ro bị tấn công mạng và đảm bảo an toàn cho dữ liệu của mình. Dưới đây là hướng dẫn chi tiết về cách thiết lập và quản lý mật khẩu cũng như các phương pháp sinh trắc học:

- **Tạo mật khẩu mạnh:**

Độ dài tối thiểu: Ít nhất 12 ký tự.

Sử dụng ký tự đặc biệt: Bao gồm chữ hoa, chữ thường, số và ký tự đặc biệt.

Tránh các từ dễ đoán: Không sử dụng tên, ngày sinh hoặc từ phổ biến.

- **Sử dụng phần mềm quản lý mật khẩu:**

Lợi ích: Giúp tạo và lưu trữ mật khẩu mạnh, dễ dàng quản lý nhiều mật khẩu.

Đề xuất: Sử dụng các công cụ như LastPass, 1Password, hoặc Bitwarden.

- **Cập nhật mật khẩu định kỳ:**

Thay đổi mật khẩu thường xuyên để tăng cường bảo mật.

- **Kích hoạt xác thực hai yếu tố (2FA):**

Sử dụng mã xác thực hoặc ứng dụng xác thực để tăng cường bảo mật khi đăng nhập.

- **Thiết lập bảo mật sinh trắc học:**

Thiết lập vân tay:

Hướng dẫn chi tiết: Theo các bước thiết lập của thiết bị hoặc ứng dụng.

Đăng ký nhiều ngón tay: Để đảm bảo truy cập dễ dàng khi một ngón tay bị tổn thương.

Thiết lập nhận diện khuôn mặt:

Hướng dẫn chi tiết: Theo các bước thiết lập của thiết bị hoặc ứng dụng.

Thiết lập nhiều góc độ khuôn mặt: Để tăng độ chính xác.

Thiết lập nhận diện giọng nói:

Hướng dẫn chi tiết: Theo các bước thiết lập của thiết bị hoặc ứng dụng.

Đảm bảo môi trường yên tĩnh: Khi thiết lập để tăng độ chính xác.

5.2.3 Loại bỏ các thông tin nhạy cảm hoặc bất thường trong tập tin hình ảnh

5.2.3.1 Giới thiệu chung

Khi chia sẻ hoặc lưu trữ hình ảnh, việc bảo vệ thông tin cá nhân và loại bỏ những dữ liệu không cần thiết là rất quan trọng. Các thông tin nhạy cảm có thể bao gồm vị trí địa lý, thời gian chụp, hoặc các thông tin cá nhân khác mà người dùng không muốn công khai. Bên cạnh đó, các thông tin bất thường hoặc không liên quan cũng cần được loại bỏ để tính an toàn của người dùng. Phần này sẽ hướng dẫn cách loại bỏ các thông

tin nhạy cảm (thông tin EXIF) và thông tin bất thường (văn bản ẩn hoặc mã độc) trong dữ liệu hình ảnh.

5.2.3.2 Các thông tin EXIF

Khái niệm:

Như đã được giới thiệu ở mục 3.2, các bức ảnh được chụp bằng điện thoại sẽ chứa các thông tin cá nhân của người dùng, những thông tin này bao gồm vị trí địa lý, thời gian chụp, hoặc các thông tin cá nhân khác mà người dùng không muốn công khai. Vì vậy mà để đảm bảo tính riêng tư, nên loại bỏ thông tin EXIF này ra khỏi ảnh trước khi gửi đi.

Lợi ích:

Cung cấp thông tin hữu ích: Giúp hiểu rõ hơn về cách và thời điểm bức ảnh được chụp.

Hỗ trợ tổ chức và quản lý ảnh: Dễ dàng sắp xếp và tìm kiếm hình ảnh dựa trên các thông tin EXIF.

Hạn chế:

Rủi ro về quyền riêng tư: Thông tin EXIF có thể tiết lộ vị trí địa lý, thời gian và các chi tiết khác mà người chụp không muốn chia sẻ.

Cách loại bỏ thông tin EXIF:

Sử dụng phần mềm chỉnh sửa ảnh: Các phần mềm như Adobe Photoshop, GIMP, hoặc các công cụ trực tuyến cho phép xóa thông tin EXIF trước khi chia sẻ hình ảnh.

Công cụ EXIF Remover: Sử dụng các công cụ chuyên dụng để loại bỏ EXIF như EXIFTool, JPEGsnoop, hoặc các ứng dụng di động.

Xây dựng công cụ: Sử dụng các hàm hoặc thư viện để nhận dạng thông tin EXIF trong ảnh từ đó xóa hoặc đổi giá trị về mặc định để đảm bảo quyền riêng tư.

5.2.3.3 Các thông tin bất thường

Khái niệm:

Trong lĩnh vực bảo mật thông tin, các kỹ thuật ẩn thông điệp trong hình ảnh (steganography) là một phương pháp phổ biến để giấu các thông điệp trong dữ liệu số. Một trong những phương pháp thông dụng nhất là phương pháp LSB (Least

Significant Bit - bit ít quan trọng nhất). Mục tiêu của phần này là giới thiệu cách loại bỏ các thông tin bất thường, đặc biệt là thông điệp ẩn hoặc mã độc được chèn vào trong hình ảnh bằng phương pháp LSB.

Cơ chế hoạt động:

Mỗi giá trị màu của một pixel trong hình ảnh kỹ thuật số thường được biểu diễn bằng 8 bit. Ví dụ, giá trị màu đỏ (R), xanh lá (G), và xanh dương (B) của một pixel có thể được biểu diễn như sau: (01111010,00101101,01000011) -> Bit ít quan trọng nhất (LSB) là bit cuối cùng bên phải trong chuỗi này. Trong ví dụ, LSB của các giá trị màu lần lượt là 0, 1, và 1.

Thông điệp: "A"

Chuyển đổi thành nhị phân: 01'000'001

Nhúng vào các pixel:

Pixel đầu tiên: RGB ban đầu là (123, 45, 67)

R: 123 -> 0111101**1** -> 0111101**0**

G: 45 -> 0010110**1** -> 0010110**0**

B: 67 -> 0100001**1** -> 0100001**1**

Pixel thứ hai: Giả sử pixel thứ hai có giá trị RGB ban đầu là (200, 150, 100)

R: 200-> 1100100**0**-> 1100100**0**

G: 150-> 1001011**0**-> 1001011**0**

B: 100-> 0110010**0**-> 0110010**0**

Pixel thứ ba: Giả sử pixel thứ ba có giá trị RGB ban đầu là (50, 25, 75)

R: 50-> 0011001**0**-> 0011001**0**

G: 25-> 0001100**1**-> 0001100**1**

B: 75-> 0100101**1**-> 0100101**1**

Giá trị mới của G vẫn là 25.

Kênh B của pixel thứ ba không cần thay đổi vì chúng ta chỉ có 8 bit trong thông điệp.

Lợi ích:

Bảo mật thông tin: Thông điệp ẩn giúp bảo mật thông tin bằng cách giấu nó trong các tệp hình ảnh. Điều này làm cho việc phát hiện và đánh cắp thông tin trở nên khó khăn hơn.

Truyền tải thông tin một cách bí mật: Phương pháp này cho phép truyền tải thông tin một cách bí mật qua các kênh không an toàn. Người nhận có thể sử dụng kỹ thuật đặc biệt để trích xuất thông tin mà không cần phải lo ngại về việc bị phát hiện.

Bảo vệ quyền sở hữu trí tuệ: Các tác giả có thể sử dụng kỹ thuật này để nhúng thông tin về quyền sở hữu trí tuệ hoặc chữ ký số vào trong hình ảnh, giúp chứng minh nguồn gốc và quyền sở hữu của nội dung số.

Hạn chế:

Tăng kích thước tệp tin: Việc nhúng thông tin vào hình ảnh có thể làm tăng kích thước tệp tin, đặc biệt nếu lượng thông tin ẩn lớn. Điều này có thể gây khó khăn trong việc lưu trữ và truyền tải tệp tin.

Nguy cơ:

Chứa mã độc nguy hiểm: Thông tin ẩn trong hình ảnh có thể chứa mã độc hoặc các lệnh nguy hiểm. Nếu thông tin ẩn này được trích xuất và thực thi, nó có thể gây hại cho hệ thống hoặc đánh cắp thông tin nhạy cảm [16]

5.2.3.4 Cách loại bỏ thông tin bất thường:

Dùng phần mềm có sẵn

Có nhiều phần mềm chuyên dụng có thể phát hiện và loại bỏ thông tin ẩn trong hình ảnh. Các phần mềm này thường sử dụng các kỹ thuật phân tích hình ảnh để tìm ra những bất thường trong các bit ít quan trọng nhất (LSB) và loại bỏ chúng. Dưới đây là một số phần mềm phổ biến:

- **StegDetect** là một công cụ mã nguồn mở được sử dụng để phát hiện các tệp hình ảnh có chứa thông tin ẩn bằng kỹ thuật steganography. Nó có thể phát hiện nhiều định dạng steganography khác nhau.
- **Xiao Steganography** là một công cụ cho phép người dùng giấu và phát hiện thông tin ẩn trong các tệp hình ảnh và cả những tệp tin âm thanh

Dùng phương pháp CDR

Phương pháp CDR (Content Disarm and Reconstruction) bao gồm việc phân tích và tái tạo lại nội dung của các tập tin để loại bỏ bất kỳ thông tin ẩn hoặc mã độc nào mà không làm ảnh hưởng đến chất lượng và nội dung chính của tệp. CDR thường được sử dụng để xử lý các tệp đính kèm trong email hoặc các tệp được tải xuống từ Internet nhằm đảm bảo chúng không chứa mã độc hoặc các thông tin ẩn nguy hiểm.

- Nguyên lý hoạt động của CDR

Phương pháp Content Disarm and Reconstruction (CDR) hoạt động bằng cách phân tích và tái tạo lại nội dung của các tập tin để loại bỏ bất kỳ thông tin ẩn hoặc mã độc nào mà không làm ảnh hưởng đến chất lượng và nội dung chính của tệp. Quá trình này bắt đầu bằng việc quét và phân tích từng phần của tập tin để tìm ra các đoạn mã hoặc dữ liệu không hợp lệ hoặc nghi ngờ. Sau khi phân tích, CDR tái tạo lại tập tin với nội dung hợp lệ và bỏ qua hoặc loại bỏ các phần dữ liệu không hợp lệ hoặc có khả năng chứa mã độc. Bằng cách này, CDR đảm bảo rằng các tập tin được xử lý sẽ không chứa bất kỳ thành phần nguy hiểm nào có thể gây hại cho hệ thống.

- Lợi ích của CDR

Content Disarm and Reconstruction (CDR) mang lại nhiều lợi ích quan trọng cho bảo mật thông tin. Trước hết, CDR giúp bảo vệ chống lại mã độc bằng cách loại bỏ mọi mã độc hoặc các lệnh nguy hiểm có thể ẩn trong các tập tin. Điều này đặc biệt hữu ích trong việc ngăn chặn các cuộc tấn công bằng email hoặc các tệp đính kèm từ các nguồn không đáng tin cậy. Thứ hai, CDR đảm bảo tính toàn vẹn của dữ liệu bằng cách tái tạo lại tập tin với chất lượng và nội dung gốc, nhưng không chứa các thông tin nguy hiểm. Cuối cùng, CDR cải thiện an ninh mạng tổng thể bằng cách giảm thiểu nguy cơ nhiễm mã độc và bảo vệ hệ thống khỏi các mối đe dọa tiềm ẩn.

- Ứng dụng của CDR

Content Disarm and Reconstruction (CDR) có nhiều ứng dụng thực tiễn trong việc bảo vệ an ninh thông tin. Trong lĩnh vực bảo mật email, CDR được sử dụng để xử lý các tệp đính kèm trong email, loại bỏ mọi mã độc hoặc thông tin ẩn trước khi gửi đến người nhận, đảm bảo rằng email không chứa các nguy cơ tiềm ẩn. Ngoài ra, CDR còn được áp dụng trong bảo mật tập tin tải xuống từ Internet. Các tệp này được quét và

tái tạo lại để đảm bảo chúng không chứa thông tin ẩn hoặc mã độc, bảo vệ người dùng khỏi các mối đe dọa trực tuyến. CDR cũng được sử dụng trong nhiều ứng dụng khác như bảo mật chia sẻ tệp qua các kênh không an toàn và bảo vệ hệ thống mạng khỏi các tập tin nguy hiểm.

5.2.4 Mã hóa tập tin hình ảnh

5.2.4.1 Các phương pháp mã hóa dữ liệu trên điện thoại Android

Full Disk Encryption (FDE)

Mã hóa toàn bộ ổ đĩa (Full Disk Encryption - FDE) là mã hóa toàn bộ dữ liệu trên ổ đĩa, bao gồm cả chương trình mã hóa các phân vùng hệ điều hành khởi động. Nó được thực hiện bởi phần mềm mã hóa ổ đĩa hoặc phần cứng được cài đặt trên đĩa trong quá trình sản xuất hoặc thông qua một trình điều khiển phần mềm bổ sung. FDE chuyển đổi tất cả các dữ liệu thiết bị thành một dạng có thể được chỉ hiểu được một trong những người có chìa khóa để giải mã dữ liệu được mã hóa. Một chìa khóa xác thực được sử dụng để đảo ngược chuyển đổi và đưa ra các dữ liệu có thể đọc được. FDE ngăn chặn ổ đĩa trái phép và truy cập dữ liệu [10] .

- **Lợi ích:** Mã hóa toàn bộ dữ liệu, không để lại bất kỳ dữ liệu nào không được mã hóa.

- **Nhược điểm:**

Chỉ có thể hủy mã hóa bằng cách “reset” lại máy về thiết lập mặc định ban đầu khi mới xuất xưởng, thao tác này sẽ xóa toàn bộ dữ liệu lưu trên thiết bị.

Sau khi bị mã hóa, thiết bị của bạn sẽ hoạt động chậm hơn một chút.

File-Based Encryption (FBE)

Mã hóa dựa trên tệp (File-Based Encryption - FBE) là một tính năng bảo mật cho phép các tệp khác nhau trên thiết bị Android được mã hóa bằng các khóa riêng biệt và có thể được mở khóa độc lập. Điều này cho phép bảo vệ dữ liệu ngay cả khi thiết bị bị mất hoặc bị đánh cắp, đồng thời giúp bảo mật các tệp cá nhân của người dùng.

- **Lợi ích:** Linh hoạt hơn FDE, giúp cải thiện hiệu suất và bảo mật.
- **Nhược điểm:** Phức tạp hơn trong việc quản lý và triển khai.

5.2.4.2 Thiết lập hệ thống tập tin đặc biệt

Trong phần này, chúng ta sẽ thảo luận về cách xây dựng một hệ thống quản lý tập tin chuyên biệt để lưu trữ ảnh trên nền tảng Android áp dụng phương pháp mã hóa AES trong chế độ ECB (Electronic Codebook). Việc tự thiết lập hệ thống lưu trữ ảnh mã hóa này giúp bảo vệ dữ liệu hình ảnh quan trọng khỏi các truy cập trái phép và nâng cao bảo mật cho ứng dụng.

Giới thiệu về AES và chế độ ECB

AES (Advanced Encryption Standard) là một thuật toán mã hóa khối đối xứng được sử dụng rộng rãi trong bảo mật dữ liệu. AES có thể hoạt động với các kích thước khóa 128, 192 và 256 bit. Chế độ ECB là một trong những phương thức vận hành của AES, trong đó mỗi khối dữ liệu đầu vào được mã hóa độc lập với các khối khác.

- **Ưu điểm của ECB:** Đơn giản, dễ triển khai.
- **Nhược điểm của ECB:** Nếu cùng một khối dữ liệu xuất hiện nhiều lần, chúng sẽ được mã hóa thành cùng một khối mã hóa, gây ra nguy cơ lộ thông tin.

Thiết lập hệ thống tập tin đặc biệt

Đề tài này sẽ giới thiệu một thiết kế hệ thống quản lý tập tin chuyên biệt, tích hợp các tính năng để đảm bảo quản lý và bảo mật dữ liệu hình ảnh. Hệ thống này sẽ bao gồm hai phần chính: phục hồi dữ liệu ảnh và quản lý tập tin riêng biệt, cùng với phương pháp ẩn thông tin mật vào trong ảnh. Với khả năng lưu trữ, thêm, xóa và phục hồi ảnh một cách an toàn và hiệu quả, hệ thống sẽ sử dụng các kỹ thuật mã hóa để đảm bảo tính toàn vẹn và bảo mật của dữ liệu. Người dùng có thể yên tâm rằng hình ảnh của họ sẽ luôn được bảo vệ và có thể phục hồi khi cần thiết.

5.2.5 Ẩn giấu tập tin hình ảnh

5.2.5.1 Các ứng dụng phổ biến

Dưới đây là ba ứng dụng phổ biến giúp ẩn hình ảnh trên điện thoại Android cùng với cơ chế hoạt động chung của chúng:

- **Keepsafe Photo Vault**

Cơ chế hoạt động:

Mã hóa: Hình ảnh và video được mã hóa để bảo vệ nội dung.

Bảo mật bằng mã PIN/Vân tay: Người dùng có thể thiết lập mã PIN hoặc sử dụng vân tay để truy cập ứng dụng.

Sao lưu đám mây: Hình ảnh và video có thể được sao lưu trên đám mây để bảo vệ khỏi mất mát dữ liệu.

- **Gallery Vault**

Cơ chế hoạt động:

Ẩn tập tin: Ứng dụng tạo một khu vực ẩn để lưu trữ hình ảnh và video, các tập tin này sẽ không xuất hiện trong thư viện chính của điện thoại.

Bảo mật bằng mã PIN/Vân tay/Mật khẩu: Người dùng cần nhập mã PIN, vân tay hoặc mật khẩu để truy cập nội dung ẩn.

Chế độ giả mạo: Cung cấp mật khẩu giả để đánh lạc hướng người truy cập không mong muốn.

- **Hide Something**

Cơ chế hoạt động:

Ẩn nhanh: Cho phép người dùng chọn và ẩn hình ảnh/video một cách nhanh chóng từ thư viện.

Bảo mật bằng mã PIN/Mẫu hình/Vân tay: Thiết lập mã PIN, mẫu hình hoặc vân tay để bảo vệ nội dung ẩn.

Sao lưu Google Drive: Hỗ trợ sao lưu tập tin ẩn lên Google Drive để đảm bảo dữ liệu không bị mất.

5.2.5.2 Cơ chế hoạt động chung của các ứng dụng ẩn hình ảnh:

Mã hóa và Bảo mật: Hình ảnh và video được mã hóa để tránh truy cập trái phép. Các ứng dụng này thường yêu cầu người dùng thiết lập mã PIN, mật khẩu, mẫu hình hoặc sử dụng vân tay để mở khóa.

Ẩn tập tin: Các ứng dụng này sẽ di chuyển các tập tin được chọn vào một thư mục ẩn mà chỉ có thể truy cập thông qua ứng dụng. Những tệp này sẽ không xuất hiện trong thư viện ảnh hoặc trình quản lý tệp thông thường của điện thoại.

Sao lưu đám mây: Nhiều ứng dụng cung cấp tùy chọn sao lưu hình ảnh và video lên đám mây để bảo vệ dữ liệu khỏi mất mát.

5.3 Phục hồi dữ liệu hình ảnh trên Android

5.3.1 Phục hồi từ sao lưu

Phương pháp phục hồi từ sao lưu tận dụng các bản sao lưu đã được tạo ra trước đó để khôi phục dữ liệu hình ảnh. Điều này đảm bảo rằng có thể dễ dàng khôi phục lại các bức ảnh quý giá mà không gặp nhiều rủi ro. Dưới đây là chi tiết về cách sử dụng các công cụ sao lưu phổ biến như Google Photos, Google Drive, và các bản sao lưu cục bộ trên thiết bị để khôi phục dữ liệu.

Phục hồi từ Google Photos

Google Photos là một dịch vụ sao lưu ảnh phổ biến cho người dùng Android. Khi bạn chụp ảnh, ứng dụng Google Photos sẽ tự động sao lưu các ảnh này lên đám mây nếu tính năng sao lưu được bật. Để phục hồi ảnh từ Google Photos, bạn chỉ cần đăng nhập vào tài khoản Google và tải về các ảnh đã sao lưu từ ứng dụng Google Photos.

Phục hồi từ Google Drive

Google Drive cũng là một công cụ hữu ích để sao lưu dữ liệu. Các tệp hình ảnh có thể được tải lên và lưu trữ trên đám mây. Bạn có thể tìm và tải về các ảnh đã sao lưu từ Google Drive bằng cách đăng nhập vào tài khoản Google của mình và truy cập vào thư mục chứa các tệp hình ảnh.

Phục hồi từ sao lưu thiết bị (Local Backup)

Nhiều thiết bị Android cung cấp tính năng sao lưu cục bộ, thường được thực hiện thông qua các ứng dụng hệ thống hoặc các dịch vụ của nhà sản xuất. Bản sao lưu này chứa toàn bộ dữ liệu của thiết bị, bao gồm cả hình ảnh. Bạn có thể phục hồi ảnh bằng cách truy cập vào cài đặt hệ thống và chọn bản sao lưu gần nhất để phục hồi.

5.3.2 Sử dụng phần mềm phục hồi dữ liệu có sẵn

Trong trường hợp không có bản sao lưu, việc sử dụng các phần mềm phục hồi dữ liệu chuyên dụng là lựa chọn tối ưu. Các phần mềm này có thể giúp bạn khôi phục các hình ảnh đã bị xóa hoặc mất do nhiều nguyên nhân khác nhau. Dưới đây là tổng quan về cơ chế hoạt động của một số phần mềm phục hồi dữ liệu phổ biến:

DiskDigger Photo Recovery

DiskDigger hoạt động bằng cách quét bộ nhớ trong của thiết bị để tìm kiếm các dữ liệu hình ảnh đã bị xóa nhưng chưa bị ghi đè. Ứng dụng này sẽ hiển thị danh sách các hình ảnh có thể phục hồi, cho phép bạn chọn và lưu trữ lại những hình ảnh cần thiết.

Dr.Fone - Data Recovery

Dr.Fone cung cấp một giao diện thân thiện, cho phép người dùng kết nối thiết bị Android với máy tính và thực hiện quét sâu để tìm các tệp hình ảnh bị mất. Phần mềm này sử dụng các thuật toán tiên tiến để phát hiện và phục hồi các tệp đã bị xóa hoặc bị hỏng.

EaseUS MobiSaver

EaseUS MobiSaver sử dụng các phương pháp quét cơ bản và nâng cao để tìm kiếm các tệp hình ảnh đã bị xóa trên thiết bị Android. Sau khi quá trình quét hoàn tất, người dùng có thể xem trước và chọn các tệp cần phục hồi. EaseUS MobiSaver cũng hỗ trợ phục hồi dữ liệu từ thẻ nhớ ngoài.

Chương 6. XÂY DỰNG HỆ THỐNG QUẢN LÝ TẬP TIN CHUYÊN BIỆT PHỤC VỤ CHO AN TOÀN VÀ PHỤC HỒI DỮ LIỆU HÌNH ẢNH

6.1 Giới thiệu

Trong thời đại kỹ thuật số ngày nay, việc bảo vệ và phục hồi dữ liệu hình ảnh trở nên cực kỳ quan trọng. Hình ảnh không chỉ là kỷ niệm quý giá mà còn là tài liệu quan trọng trong nhiều lĩnh vực như y tế, nghiên cứu, và giải trí. Với mục tiêu đảm bảo an toàn và khả năng phục hồi dữ liệu hình ảnh, chúng ta cần thiết lập một hệ thống toàn diện bao gồm các mô hình và phương pháp hiệu quả. Hệ thống được đề xuất gồm hai phần chính: phục hồi dữ liệu ảnh, quản lý tập tin riêng biệt và ẩn thông tin mật vào trong ảnh. Hệ thống này cho phép lưu trữ, thêm, xóa và phục hồi ảnh một cách an toàn và hiệu quả. Với các tính năng như mã hóa dữ liệu hệ thống đảm bảo tính toàn vẹn và an toàn cho dữ liệu hình ảnh. Người dùng có thể yên tâm rằng hình ảnh của họ luôn được bảo vệ và có thể phục hồi khi cần thiết.

6.2 Phân tích yêu cầu

6.2.1 Yêu cầu kỹ thuật

Khả năng phục hồi dữ liệu: Hệ thống phải có khả năng phục hồi dữ liệu hình ảnh từ nhiều loại thiết bị lưu trữ khác nhau như ổ cứng, thẻ nhớ, và thiết bị di động.

Khả năng lưu trữ dữ liệu hệ thống phục hồi và bảo vệ hình ảnh cần đáp ứng các yêu cầu về dung lượng lớn và khả năng mở rộng linh hoạt, đảm bảo tốc độ truy xuất nhanh chóng và quản lý tài nguyên hiệu quả.

Tích hợp mã hóa dữ liệu: Đảm bảo mọi dữ liệu hình ảnh được mã hóa khi lưu trữ và truyền tải để bảo vệ trước các mối đe dọa từ bên ngoài.

Khả năng mở rộng: Hệ thống cần có khả năng mở rộng và nâng cấp để tích hợp thêm các tính năng mới trong tương lai mà không ảnh hưởng đến hoạt động hiện tại.

6.2.2 Yêu cầu phi kỹ thuật

Dễ sử dụng: Giao diện người dùng của hệ thống phải thân thiện, dễ sử dụng để người dùng có thể dễ dàng thêm, xóa và phục hồi hình ảnh.

Hỗ trợ người dùng: Cần có tài liệu hướng dẫn chi tiết và hướng dẫn sử dụng bên trong ứng dụng để giúp người dùng giải quyết các vấn đề kỹ thuật.

Tính bảo mật và riêng tư: Đảm bảo dữ liệu người dùng được bảo mật tuyệt đối, không bị truy cập trái phép và có sự riêng biệt giữa các hệ thống trên các điện thoại khác nhau.

Hiệu quả chi phí: Hệ thống cần được thiết kế để tối ưu hóa chi phí vận hành và bảo trì, phù hợp với ngân sách của các cá nhân.

6.3 Thiết kế hệ thống tập tin chuyên biệt

6.3.1 Mô hình quản lý tập tin

Đề tài đưa ra một mô hình quản lý tập tin, là một cấu trúc tổ chức và quản lý dữ liệu hình ảnh cho phép người dùng lưu trữ, truy xuất và thao tác với các tập tin một cách hiệu quả và an toàn. Trong một hệ thống tập tin chuyên biệt, mô hình quản lý tập tin được thiết kế để đáp ứng các yêu cầu cụ thể của từng ứng dụng, đảm bảo khả năng mở rộng, tính toàn vẹn và an ninh của dữ liệu.

Mô hình quản lý tập tin được hiển thị như sau



Hình 6.1 Mô hình cấu trúc chung của hệ thống quản lý tập tin

Vùng Header

Header của mô hình chứa các thông tin liên quan về cá nhân chủ sở hữu như tên người tạo, ngày tạo, ngày chỉnh sửa,... nhằm trường hợp dữ liệu ứng dụng bị thất lạc

có thể biết được chủ sở hữu ban đầu. Cấu trúc chi tiết của Header được miêu tả bằng bản các thành phần sau:

Bảng 6.1 Bảng thông tin dữ liệu hệ thống quản lý tập tin

Offset	Độ lớn (byte)	Nội dung
0x0	4	Signature (Mặc định là “.NEW”)
0x4	4	Kích thước volume (KB)
0x8	32	Password đã được hash (Nếu có)
0x28	4	Ngày tạo (Ngày 1 byte, Tháng 1 byte, Năm 2 byte)
0x2C	4	Ngày sửa (Ngày 1 byte, Tháng 1 byte, Năm 2 byte)
0x30	3	Giờ tạo (Giờ 1 byte, Phút 1 byte, Giây 1 byte)
0x33	3	Giờ sửa (Giờ 1 byte, Phút 1 byte, Giây 1 byte)
0x36	10	Tên người tạo (bản quyền)
0x40	8128	Không gian trống dành cho các bản upgrade sau

Ví dụ dưới đây sẽ cho thấy về cấu trúc lưu trữ ngày giờ.

Dữ liệu ngày sẽ được lưu theo cấu trúc:

“08 07 07 E8”h

1 byte lưu Ngày: 08h = 8d = ngày 8

1 byte lưu Tháng: 07h = 7d = tháng 7

2 byte lưu Năm: 07E8 = 2024d = năm 2024

Dữ liệu giờ sẽ được lưu theo cấu trúc:

“0A 16 20”h

1 byte lưu Giờ: 0Ah = 10d = 10 Giờ

1 byte lưu Phút: 16h = 22d = 22 Phút

1 byte lưu Giây: 20h = 32d = 32 Giây

*Ghi chú:

Kiểu dữ liệu số sẽ lưu theo cơ chế big endian (để tương khớp với hệ thống Android).

Entry table đầu tiên sẽ là sector thứ 2 (Sector 1).

Vùng quản lý sector trống

Là 1 bảng quản lý sector trống. Mỗi phần tử bảng gồm 2 thông tin như mô tả sau:

Bảng 6.2 Bảng thông tin dữ liệu phần tử vùng trống

Offset	Độ lớn (byte)	Nội dung
0x0	4	Sector bắt đầu
0x4	4	Kích thước vùng trống (đơn vị sector)

1 phần tử cấu trúc 4 byte chứa thông tin sector bắt đầu và 4 byte chứa kích thước (số sector) ví dụ: “00 00 03 67 00 00 00 05”h -> sector thứ 871 sẽ có 5 sector trống

5 sector liên tiếp cho vùng này (sector 1-5) có khả năng quản lý được $5 * 8192 / 8 = 5120$ vùng trống. có thể đáp ứng cho volume có sức chứa hàng 100 nghìn file ảnh.

Khi xóa ảnh, vùng dữ liệu của ảnh bị xóa sẽ được tính là vùng trống và khi các ảnh kế tiếp nhau xóa thì sẽ hình thành các vùng trong liền kề, nối tiếp nhau. Để giải quyết vấn đề tối ưu không gian thì những vùng trống này sẽ được gộp thành 1 vùng trống duy nhất với kích thước lớn hơn.

Với cách thức quản lý vùng trống này, hệ thống có thể tiết kiệm không gian lưu trữ khi có thể sáp nhập các vùng trống liền kề lại thành một và thông báo cho hệ thống biết là khu vực đó đủ trống để có thể lưu tập tin.

Vùng Entry table

Sau vùng trống sẽ là vùng entry. Có 320 sector chứa Directory entry. Mỗi entry sẽ có cấu trúc như bảng sau:

Bảng 6.3 Bảng thông tin dữ liệu vùng entry table của hệ thống tập tin

Offset	Độ lớn (byte)	Nội dung
0x0	160	Tên file
0xA0	5	Tên mở rộng
0xA5	4	Ngày tạo (Ngày 1 byte, Tháng 1 byte, Năm 2 byte)
0xA9	4	Sector bắt đầu dữ liệu
0xAD	4	Kích thước tập tin (byte)
0xB1	1	File đã xóa hay chưa (30h là chưa, 31h là đã xóa)
0xB2	32	Password để đọc file đó (Nếu có)
0xD2	1	File có mã hoá hay không (30h là chưa, 31h là đã mã hóa)
0xD3	45	Không gian trống dành cho các bản upgrade sau

Mỗi Entry sẽ lưu các thông tin lưu trữ về từng bức ảnh và vị trí của bức ảnh trong hệ thống quản lý tập tin này. Mỗi Directory Entry sẽ có kích thước

Ghi chú:

- Nếu chưa có password thì phần này sẽ toàn bộ là 00h

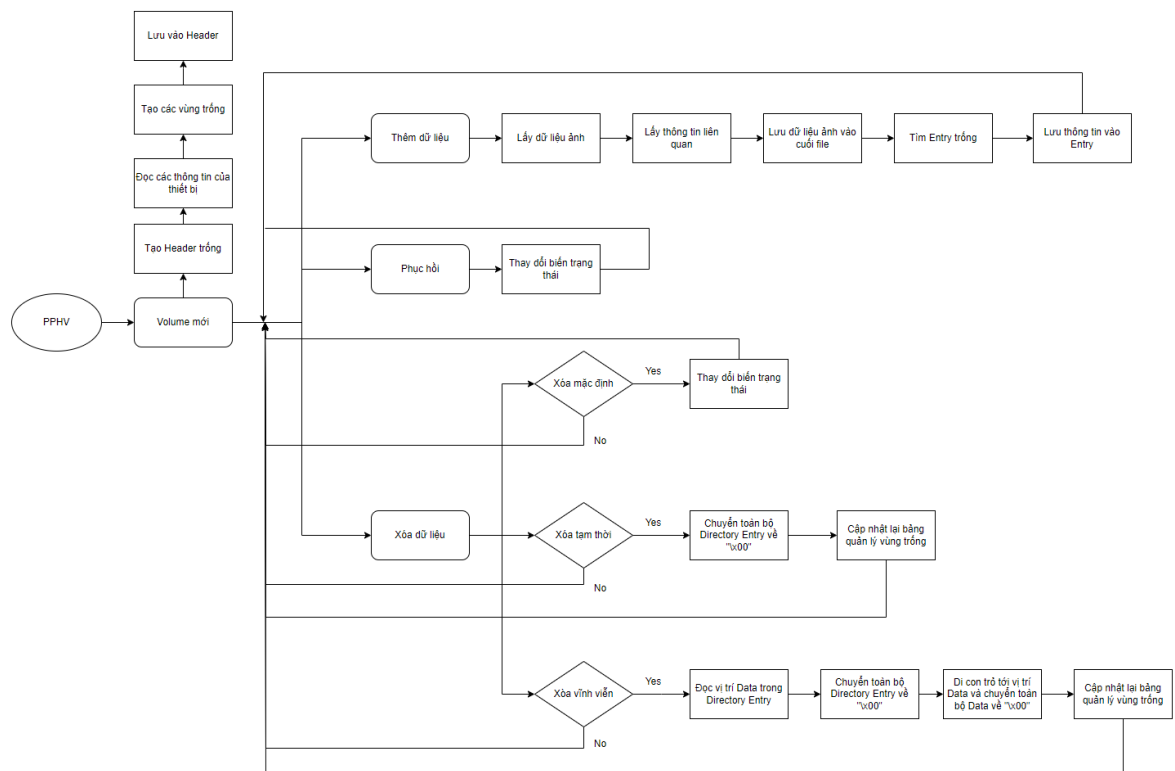
Hệ thống sẽ lưu trữ phần hash của password (sử dụng SHA-256). Khi người dùng muốn mã hóa dữ liệu hình ảnh của mình để an toàn, hệ thống sẽ dùng mật khẩu của người dùng để làm Key mã hóa cho thuật toán AES chế độ ECB. Và hash của password sẽ được lưu lại để khi giải mã, hệ thống sẽ kiểm tra lại chuỗi hash đó sau đó mới giải mã.

Với thiết kế này hệ thống sẽ dễ dàng quản lý các thông tin về dữ liệu để thuận tiện cho việc xác định nguồn gốc và tính cá nhân của tập tin.

Vùng data

Vùng data bắt đầu từ sector 326 (1+5+320). Mỗi tấm ảnh sẽ được lưu nguyên vẹn không phân mảnh vì được sử dụng cơ chế thêm vào cuối file. Với cơ chế này sẽ đảm bảo được việc những dữ liệu ghi trước sẽ không thể bị ghi đè. Điều này giúp ích rất nhiều cho việc phục hồi dữ liệu nếu người dùng vô tình xóa.

6.3.2 Sơ đồ hoạt động



Hình 6.2 Sơ đồ hoạt động của hệ thống quản lý tập tin PPHV

6.3.3 Mô hình chức năng chính

Tạo volume

Khi người dùng cài đặt chương trình, hệ thống sẽ tự động tạo ra một volume được đặt tên theo tên của người dùng. Volume này sẽ chứa thông tin về ngày tạo và có dung lượng lên tới 24GB, đủ lớn để lưu trữ một lượng lớn dữ liệu. Điều này không chỉ đảm bảo tính cá nhân hóa mà còn tạo ra một không gian riêng biệt và an toàn cho từng người dùng. Người dùng còn có thể thiết lập mật khẩu cho volume của mình để tăng cường tính bảo mật, giúp bảo vệ dữ liệu cá nhân khỏi những truy cập trái phép.

B1: Tạo 1 class Header Volume rỗng

B2: Đọc dữ liệu thời gian, tên thiết bị khởi tạo, ... từ thiết bị

B3: Tạo các vùng của hệ thống, lấy kích thước của hệ thống

B4: Cập nhật những dữ liệu có được vào Header Volume

Thêm dữ liệu

Người dùng có thể chọn các tấm ảnh yêu thích hoặc những tấm ảnh cần được bảo vệ từ thư viện và đưa vào một thư mục quản lý trong volume. Việc này không chỉ giúp người dùng dễ dàng quản lý và tìm kiếm ảnh mà còn đảm bảo các tấm ảnh được bảo mật một cách tốt nhất. Hệ thống sẽ duy trì thông tin nguyên vẹn của ảnh, tránh mất mát dữ liệu trong quá trình lưu trữ. Điều này đặc biệt hữu ích cho những người dùng có nhu cầu bảo vệ những kỷ niệm quý giá hoặc các tài liệu quan trọng

Input: Tập tin hình ảnh

Output: Lưu được tập tin hình ảnh nằm trong PPHV

B1: Đọc tập tin hình ảnh dưới dạng byte

B2: Đọc thông tin ngày tạo, giờ tạo, tên,... và vị trí cuối file

B4: Lưu dữ liệu ảnh vào cuối file

B3: Tìm Entry trống

B4: Lưu thông tin vào Entry

B5: Cập nhật thời gian chỉnh sửa ở Header

Xóa dữ liệu

Để người dùng có thể linh hoạt trong việc xóa dữ liệu, chương trình sẽ cung cấp ba tùy chọn xóa khác nhau:

1. **Xóa Thông Thường:** Với tùy chọn này, người dùng có thể xóa ảnh như bình thường, nhưng vẫn có khả năng phục hồi lại thông qua chức năng phục hồi dữ liệu của chương trình. Đây là lựa chọn phù hợp khi người dùng muốn loại bỏ tạm thời các tấm ảnh mà vẫn giữ lại khả năng khôi phục nếu cần thiết.

Input: Tên tập tin hình ảnh

Output: Thay đổi dữ liệu Entry của ảnh trong PPHV

B1: Tìm Directory Entry của ảnh đó trong vùng Entry table

B2: Thay đổi thông tin để hệ thống nhận biết rằng đó là file đã xóa và không hiển thị được

B3: Cập nhật lại thời gian chỉnh sửa trong header

2. **Xóa Có Thể Phục Hồi:** Khi chọn xóa theo phương pháp này, thông tin của ảnh sẽ được giữ lại trong trường hợp ảnh chưa bị ghi đè. Tuy nhiên, các ảnh bị xóa theo cách này sẽ không hiển thị trong tính năng phục hồi của chương trình. Người dùng sẽ cần tìm lại thông tin ảnh một cách thủ công nếu muốn phục hồi. Đây là lựa chọn tốt khi người dùng muốn xóa ảnh nhưng vẫn để lại một cơ hội khôi phục trong trường hợp đặc biệt.

Input: Tên tập tin hình ảnh

Output: Xóa dữ liệu Entry của ảnh trong PPHV

B1: Tìm Directory Entry của ảnh đó trong vùng Entry table

B2: Chuyển toàn bộ dữ liệu Entry về 00h

B3: Cập nhật lại vùng trống

B4: Cập nhật lại thời gian chỉnh sửa trong Header

3. **Xóa Vĩnh Viễn:** Lựa chọn này sẽ xóa tất cả thông tin về ảnh một cách vĩnh viễn, đảm bảo rằng dữ liệu hình ảnh không thể được phục hồi bằng bất kỳ cách nào. Điều này rất hữu ích khi người dùng muốn đảm bảo rằng dữ liệu đã xóa không bao giờ bị khôi phục lại, tăng cường tính bảo mật và quyền riêng tư cho các thông tin nhạy cảm hoặc không còn cần thiết.

Input: Tên tập tin hình ảnh

Output: Xóa toàn bộ dữ liệu gồm Entry và Data của ảnh trong PPHV

B1: Tìm Directory Entry của ảnh đó trong vùng Entry table

B2: Lưu vị trí lưu trữ của dữ liệu của ảnh đọc được từ Directory Entry

B3: Chuyển toàn bộ Directory Entry đó về 00h

B4: Tìm tới vị trí lưu trữ dữ liệu đã được đọc ở B2

B5: Chuyển toàn bộ dữ liệu về 00h

B6: Cập nhật lại vùng trống

B7: Cập nhật lại thời gian chỉnh sửa trong Header

Phục hồi dữ liệu thông thường của volume

Chương trình hỗ trợ chức năng phục hồi dữ liệu bằng cách hiển thị các tấm ảnh đã bị xóa mặc định. Người dùng có thể dễ dàng duyệt qua danh sách các tấm ảnh này và chọn phục hồi lại những bức ảnh cần thiết. Giao diện của chương trình được thiết kế trực quan, giúp người dùng thao tác phục hồi nhanh chóng và hiệu quả. Điều này đảm bảo rằng những dữ liệu quan trọng không bị mất mát vĩnh viễn và người dùng luôn có khả năng khôi phục lại các kỷ niệm quý giá hoặc thông tin cần thiết.

Input: Tên tập tin hình ảnh

Output: Chuyển tập tin hình ảnh đó về lại PPHV

B1: Tìm Directory Entry của ảnh đó trong vùng Entry table

B2: Thay đổi thông tin để hệ thống nhận biết file đó chưa xóa và có thể hiển thị được

B3: Cập nhật lại thời gian chỉnh sửa trong Header

Phục hồi dữ liệu thủ công

Khi dữ liệu đã xóa phần Directory Entry thì hệ thống sẽ không thể hỗ trợ trong việc phục hồi lại dữ liệu đó. Các duy nhất để có thể phục hồi lại là đọc trực tiếp thông tin của tập tin hệ thống quản lý tập tin này dưới dạng sector và tìm dữ liệu bị xóa trong vùng data.

Input: Tập tin chứa hệ thống quản lý tập tin PPHV

Output: Trích xuất các tập tin hình ảnh cần phục hồi

B1: Đọc tập tin dưới dạng binary

B2: Dịch chuyển tới sector 326

B3: Tìm phần dữ liệu của tập tin ảnh cần được phục hồi

B4: Đọc dữ liệu tới khi gặp chữ ký của loại ảnh đó

B5: Lưu tập tin ảnh với dữ liệu vừa đọc được

Mã hóa/Giải mã dữ liệu

Để đảm bảo tính bảo mật của dữ liệu, chương trình sử dụng thuật toán AES (Advanced Encryption Standard) với chế độ ECB (Electronic Codebook) để mã hóa các tấm ảnh. Khi người dùng thêm ảnh vào volume, ảnh sẽ được mã hóa để bảo vệ khỏi các truy cập trái phép. Quá trình mã hóa đảm bảo rằng chỉ những người dùng có mật khẩu hợp lệ mới có thể truy cập và xem các tấm ảnh.

Input: Tên tập tin hình ảnh

Output: Dữ liệu hình ảnh đã được mã hóa

B1: Tìm Directory Entry dựa theo tên tập tin hình ảnh

B2: Hỏi người dùng mật khẩu

B3: Kiểm tra hash của mật khẩu với mã hash được lưu trong Directory Entry

B4: Đọc thông tin vị trí dữ liệu ảnh trong vùng Data

B5: Đọc toàn bộ dữ liệu ảnh dựa theo vị trí và số sector chứa ảnh

B6: Mã hóa hoặc Giải mã dựa trên biến cờ

B7: Cập nhật lại thời gian chỉnh sửa trong Header

Tái cấu trúc Volume PPHV

Khi sử dụng một thời gian dài sẽ không tránh được việc xóa những tập tin ảnh, với cơ chế lưu đặt biệt hỗ trợ cho việc phục hồi nhưng lại không tối ưu về mặt không gian. Làm cho kích thước hệ thống ngày càng lớn. Tính năng này với mục đích “dọn dẹp” những chỗ trống không dùng đến trong PPHV

Input: Tập tin chứa hệ thống quản lý tập tin PPHV

Output: Tập tin chứa hệ thống quản lý tập tin PPHV đã được tối ưu không gian

B1: Đọc dữ liệu Header

B2: Tạo tập tin Volume PPHV mới (A) với thông tin Header ban đầu

B3: Đi tới sector 6

B4: Đọc và lưu thông tin Entry

B5: Tìm các sector chứa dữ liệu dựa trên thông tin Entry đọc được

B6: Đọc dữ liệu hình ảnh

B7: Viết Entry vào vùng Entry của (A)

B8: Viết dữ liệu hình ảnh vào cuối file

B9: Cập nhật lại vị trí lưu dữ liệu trong Entry

B10: Quay lại bước 4 tới khi hết vùng Entry

B11: Xóa tập tin PPHV cũ

6.3.4 Mô hình chức năng phụ

Cập nhật lại bảng quản lý vùng trống

Khi sử dụng tính năng xóa bảng quản lý vùng trống sẽ được chỉnh sửa để hệ thống PPHV nhận biết được vùng đó trống và có thể ghi dữ liệu ảnh vào được.

Input: Vị trí bắt đầu của vùng dữ liệu cần xóa

Output: Vùng trống được cập nhật

B1: Tạo phần tử trong bảng quản lý vùng trống dựa vào vị trí và độ lớn của tập tin ảnh cần xóa

B2: Thêm phần tử vào bảng quản lý vùng trống ở cuối bảng

B3: Sắp xếp lại bảng dựa theo vị trí

B4: Xử lý những phần tử liên tiếp nhau (sát nhập)

Hiển thị ảnh

Khi ảnh được lưu trong PPHV, tính năng hiển thị ảnh giúp người dùng xem được các bức ảnh mà mình đã lưu

Input: Tập tin PPHV

Output: Hiển thị ảnh ra màn hình

B1: Đọc tập tin dưới dạng binary

B2: Đi đến sector 6

B3: Đọc và lưu tất cả Directory Entry vào trong mảng

B4: Hiển thị những bức ảnh nào có cờ là chưa xóa và cờ chưa bị mã hóa

B5: Hiển thị những bức ảnh có cờ đã bị mã hóa bằng ảnh của hệ thống

Tìm Directory Entry trống

Khi thêm ảnh vào hệ thống PPHV, hệ thống sẽ chạy tính năng này để tìm chỗ để lưu Entry của ảnh

Input: Tập tin PPHV

Output: Vị trí Entry trống

B1: Đọc tập tin dưới dạng binary

B2: Đi đến sector 6

B3: Duyệt từng khối Entry và kiểm tra toàn 00h

B4: Đọc vị trí đầu khối

Chương 7. CÁC GIẢI PHÁP MỞ RỘNG

7.1 Thiết kế các phương pháp an toàn dữ liệu hình ảnh

7.1.1 Sao lưu, mã hóa và ẩn dữ liệu dưới hình thức đặc biệt

Dữ liệu hình ảnh sẽ được người dùng lựa chọn và lưu vào trong hệ thống quản lý tập tin chuyên biệt để có thể được quản lý. Điều này giúp cho người dùng có thể có một bản sao lưu an toàn của những tấm ảnh quý giá. Nếu thư viện ảnh của Android bị lỗi, vô tình bị xóa hoặc bị mã hóa có chủ đích nhằm mục đích tống tiền bởi kẻ xấu thì những dữ liệu hình ảnh được lưu trong quản lý tập tin chuyên biệt này vẫn sẽ an toàn. Đây cũng là 1 hình thức ẩn dữ liệu khi mà dữ liệu ảnh được lưu theo một hệ thống quản lý tập tin chuyên biệt khác với hệ thống Android. Bên cạnh đó hệ thống còn cung cấp tính năng mã hóa ảnh để giúp nâng cao tính bảo mật cho những tấm ảnh vô cùng quan trọng với người dùng.

7.1.2 Xóa thông tin siêu dữ liệu trong tập tin hình ảnh

Như đã được nhắc đến ở Chương 3, dữ liệu hình ảnh bao gồm cả những dữ liệu cá nhân của người dùng điện thoại bao gồm tên thiết bị chụp, vị trí chụp, ngày giờ chụp,... Điều này có thể gây ra rò rỉ thông tin cá nhân nếu những thông tin này chưa được xử lý khi gửi ảnh. Để giải quyết điều này thì đề tài sẽ cung cấp một tính năng để lọc và xóa những dữ liệu này ra khỏi tấm ảnh và chỉ để lại nội dung hình ảnh thực tế hiển thị.

7.1.3 Nhúng và ẩn thông tin vào dữ liệu hình ảnh

Để giải quyết nhu cầu ẩn giấu những thông tin, nội dung quan trọng ví dụ như một đoạn văn bản quan trọng, đề tài cung cấp tính năng ẩn dữ liệu dựa theo phương pháp LSB đã được nhắc đến ở mục 5.4.2.

7.1.4 Làm “sạch” ảnh bằng kỹ thuật CDR được thiết kế riêng

Kỹ thuật Content Disarm and Reconstruction (CDR) thiết kế riêng này được áp dụng để làm "sạch" hình ảnh, loại bỏ các mã độc và các yếu tố tiềm ẩn nguy cơ bảo

mật mà không . Đề tài đã cải tiến dành riêng cho ảnh bằng cách đọc các pixel hình ảnh và thay đổi các bit thấp. Sau đó tạo lại một file ảnh mới với nội dung những pixel ảnh thuần túy.

7.2 Thiết kế các phương pháp phục hồi dữ liệu hình ảnh

7.2.1 Phục hồi trên hệ thống tập tin chuyên biệt

Dựa vào thiết kế của đề tài đưa ra, sẽ có 2 kiểu phục hồi dữ liệu ảnh trên hệ thống tập tin chuyên biệt như sau:

Phục hồi bằng tính năng của ứng dụng: Khi người dùng “xóa thông thường” thì ảnh sẽ không bị xóa mà chỉ tạm thời ẩn không hiển thị cho người dùng và người dùng phục hồi ảnh bằng cách sử dụng tính năng “phục hồi ảnh” bên được cung cấp bởi hệ thống này.

Phục hồi thủ công: Khi ứng dụng bị lỗi hoặc người dùng sử dụng tính năng “xóa tạm thời” thì lúc này phải đọc dữ liệu bên dưới của ứng dụng (đọc binary file) theo cấu trúc. Chúng ta vẫn có thể tìm được phần dữ liệu hình ảnh nếu nó chưa bị ghi đè.

7.2.2 Phục hồi ảnh trên điện thoại Android thông qua thư mục Emulated

Thư mục Emulated là một thư mục ảo trên thiết bị Android, được tạo ra để giả lập bộ nhớ trong của thiết bị. Nó là một phần của hệ thống tập tin, nơi mà các ứng dụng và người dùng lưu trữ dữ liệu như ảnh, video, tài liệu và các loại tệp khác.

Thư mục này thường có đường dẫn như: /storage/emulated/0 hoặc /sdcard.

Thư mục Emulated là nơi chứa hầu hết các dữ liệu cá nhân và ứng dụng trên thiết bị Android, bao gồm ảnh, video, tài liệu, nhạc và các tệp khác. Do đó, khi dữ liệu bị mất hoặc xóa, đây là nơi đầu tiên cần kiểm tra để phục hồi.

Vì thư mục Emulated là một lớp ảo hóa của bộ nhớ trong, nó cung cấp một cái nhìn nhất quán và đơn giản hơn về hệ thống tập tin, giúp các ứng dụng phục hồi dữ liệu dễ dàng quét và truy xuất dữ liệu.

Khi dữ liệu bị xóa trên Android, chúng thường không bị xóa hoàn toàn ngay lập tức mà chỉ đánh dấu là đã xóa trong hệ thống tập tin. Do đó, quét thư mục Emulated có thể giúp phát hiện và phục hồi các tệp này trước khi chúng bị ghi đè.

7.2.3 Truy xuất và phục hồi dữ liệu trực tiếp trên hệ thống tập tin

7.2.3.1 Image (.img) file của điện thoại là gì và cách trích xuất

Định nghĩa

Tệp .img là một định dạng tập tin chứa hình ảnh (image) của một phân vùng hoặc toàn bộ ổ đĩa. Trong ngữ cảnh của hệ điều hành Android và các thiết bị điện tử, tệp .img thường được sử dụng để lưu trữ một bản sao hoàn chỉnh của một phân vùng cụ thể trên thiết bị. Những tệp này chứa toàn bộ nội dung của một phân vùng nhất định và được sử dụng trong nhiều tình huống, chẳng hạn như khôi phục hệ điều hành, cập nhật phần mềm, hoặc tùy chỉnh hệ thống.

Cách trích xuất

Sử dụng công cụ adb (Android Debug Bridge) của Android Studio, được lưu tại đường dẫn C:\Your-SDK-Folder\Android\Android-sdk\platform-tools\ sau đó ta sẽ kết nối thiết bị với máy, để kiểm tra ta sẽ sử dụng “adb devices”. Khi đủ điều kiện trên ta sẽ “adb shell” để có thể chạy lệnh Linux trực tiếp trên thiết bị, sau đó sẽ dùng lệnh “su” để lấy quyền tiếp tục dùng “dd if=/dev/Block/sda of=/sdcard/sda_dump.img” và adb pull /sdcard/sda_dump.img /path để lưu vào đường dẫn trên máy.

7.2.3.2 Truy xuất dữ liệu trên hệ thống tập tin EXT4

Dưới đây là phân tích trên cùng một tập tin .img đã được trích xuất ra từ một hệ thống tập tin EXT4 nhưng do dữ liệu rất dài nên chỉ phân tích cấu trúc để có thể tìm được nơi lưu trữ dữ liệu và sau đó có những biện pháp phục hồi dữ liệu.

Khi muốn phân tích một hệ thống tập tin EXT4. Mở đầu phần cấu trúc hệ thống quản lý tập tin, ta phải phân tích Superblock, giả sử ta có một Superblock như sau:

```

Offset(h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F Decoded text
00000400 30 DD 01 00 59 73 07 00 5E 5F 00 00 C2 2F 07 00 0Ý..Ys..^_..Ã/..
00000410 25 DD 01 00 00 00 00 00 02 00 00 00 02 00 00 00 %Ý.....
00000420 00 80 00 00 00 80 00 00 D0 1F 00 00 6A AD 07 66 .€....€...Đ...j..f
00000430 6A AD 07 66 01 00 FF FF 53 EF 01 00 01 00 00 00 j..f..ÿÿSi.....
00000440 6A AD 07 66 00 00 00 00 00 00 00 00 01 00 00 00 j..f.....
00000450 00 00 00 00 0B 00 00 00 00 01 00 00 3C 00 00 00 .....<...
00000460 C6 02 00 00 6B 04 00 00 0F 0E 56 4E 3E 01 49 30 E...k.....VN>.IO
00000470 80 FA 43 0D 4A 46 75 B9 00 00 00 00 00 00 00 00 €úC.JFu¹.....
00000480 00 00 00 00 00 00 00 00 2F 6D 65 64 69 61 2F 76 ...../media/v
00000490 69 6E 68 2F 30 66 30 65 35 36 34 65 2D 33 65 30 inh/0f0e564e-3e0
000004A0 31 2D 34 39 33 30 2D 38 30 66 61 2D 34 33 30 64 1-4930-80fa-430d
000004B0 34 61 34 36 37 35 62 39 00 00 00 00 00 00 00 00 4a4675b9.....

```

Hình 7.1 Bản mẫu Superblock của EXT4(2)

Trong trường hợp ta có một hệ thống tập tin EXT4 với cấu trúc cơ bản thì ta có một số các thông tin quan trọng cần biết trong Superblock là:

Bảng 7.1 Bảng thông tin dữ liệu thu thập từ Superblock mẫu

Offset	kích thước	Tên	Hex	Dec
0x400	4	Tổng số Inode	0x0001DD30	122160
0x404	4	Tổng số Block	0x00077359	488281
0x40C	4	Số lượng Block còn trống	0x00072BAE	469934
0x410	4	số lượng Inode còn trống	0x0001DD17	122135
0x414	4	Block dữ liệu đầu tiên	0x00000000	0
0x418	4	Kích thước của Block [2 ^{^(10+X)}]	0x00000002	2 ^{^(10+2)} = 4096
0x438	4	Magic signature	0xEF53	

Thông qua việc biết trước kích thước Block, ta có thể tìm được Block Group Descriptor do vị trí này nằm ở Block ngay sau Superblock, từ đó ta có bảng Block Group Descriptor ở vị trí Offset là 4096 = 0x1000 như sau:

```

00001000 F0 00 00 00 FF 00 00 00 0E 01 00 00 15 61 BB 1F 8...ÿ.....a».
00001010 02 00 04 00 00 00 00 00 00 50 A7 36 BB 1F 0D C5 .....P$6»...Ă
00001020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001030 00 00 00 00 00 00 00 00 77 87 66 20 00 00 00 00 .....w+f ....
00001040 F1 00 00 00 00 01 00 00 0B 03 00 00 00 7B CE 1F ñ.....{İ.
00001050 02 00 04 00 00 00 00 00 79 13 24 F3 CE 1F 70 EC .....y.óİ.pì
00001060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001070 00 00 00 00 00 00 00 00 C3 2F 80 E1 00 00 00 00 .....Ă/€á....

```

Hình 7.2 Bản mẫu Block Group Descriptor của EXT4 (2)

Phân tích Block Group Descriptor cho Block Group đầu tiên ta có bảng giá trị sau:

Bảng 7.2 Bảng thông tin dữ liệu thu thập từ Block Group Descriptor mẫu

Offset	Kích thước (bytes)	Nội dung	Hex
0x1000	4	32-bits thấp vị trí của Block Bitmap	0x000000F0
0x1004	4	32-bits thấp vị trí của Inode Bitmap	0x000000FF
0x1008	4	32-bits thấp vị trí của Inode Table	0x0000010E
0x100C	2	16-bits thấp số lượng Block còn trống	0x6112
0x100E	2	16-bits thấp số lượng Inode còn trống	0x1FB6
0x1010	2	16-bits thấp số lượng Directory	0x0005
0x1012	2	Cờ dùng để hỗ trợ hệ thống quản lý	0x0004
0x101C	2	16-bits thấp số lượng Inode chưa sử dụng	0x1FB5
0x1020	4	32-bits cao vị trí của Block Bitmap	0x00000000
0x1024	4	32-bits cao vị trí của Inode Bitmap	0x00000000
0x1028	4	32-bits cao vị trí của Inode Table	0x00000000

0x102C	2	16-bits cao số lượng Block còn trống	0x0000
0x102E	2	16-bits cao số lượng Inode còn trống	0x0000
0x1030	2	16-bits cao số lượng Directory	0x0000
0x1032	2	16-bits cao số lượng Inode chưa sử dụng	0x0000

Qua bảng giá trị trên ta sẽ có được vị trí Block của bảng Inode Table là:
 $0x00000000 + 0x0000010E = 0x000000000000010E$.

Vậy vị trí Offset cụ thể là: $0x10E \times 0x1000$ (kích thước Block) = $0x10E000$

Từ vị trí này ta sẽ có phần đầu Inode Table sau:

```

Offset(h) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F Decoded text
0010E000 00 00 00 00 00 00 00 00 6A AD 07 66 6A AD 07 66 .....j..fj..f
0010E010 6A AD 07 66 00 00 00 00 00 00 00 00 00 00 00 00 j..f.....
0010E020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E040 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E070 00 00 00 00 00 00 00 00 00 00 00 00 00 DE FC 00 00 .....Bü..
0010E080 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E0A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E0B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E0C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E0D0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E0E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E0F0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E100 FF 41 00 00 00 10 00 00 AB AE 07 66 A4 AE 07 66 ŸA.....«®.f«®.f
0010E110 A4 AE 07 66 00 00 00 00 00 00 00 07 00 08 00 00 00 «®.f.....
0010E120 00 00 08 00 09 00 00 00 0A F3 01 00 04 00 00 00 .....ó.....
0010E130 00 00 00 00 00 00 00 00 01 00 00 00 E1 1E 00 00 .....á...
0010E140 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E150 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E160 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E170 00 00 00 00 00 00 00 00 00 00 00 00 D3 66 00 00 .....Óf..
0010E180 20 00 82 B7 3C 37 34 AB 3C 37 34 AB 90 3C 76 92 .,·<74«<74«.·<v'
0010E190 6A AD 07 66 00 00 00 00 00 00 00 00 00 00 00 00 j..f.....
0010E1A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E1B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E1C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E1D0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E1E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010E1F0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

Hình 7.3 Bản mẫu Inode Table của EXT4

Do Inode đầu tiên của hệ thống sẽ không được dùng cho các loại tệp thông thường nên chúng ta sẽ bỏ qua, Inode thứ 2 mặc định sẽ luôn chứa root Directory, từ đó ta phân tích I_block của Inode ta sẽ có được trị trí Block của root Directory là 0x1EE1 tức Offset 0x1EE1000.

Qua đó ta sẽ có Directory sau:

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
01EE1000	02	00	00	00	0C	00	01	02	2E	00	00	00	02	00	00	00	
01EE1010	0C	00	02	02	2E	2E	00	00	0B	00	00	00	14	00	0A	02	
01EE1020	6C	6F	73	74	2B	66	6F	75	6E	64	00	00	0C	00	00	00	lost+found.....
01EE1030	14	00	0B	01	6B	61	6E	65	6B	69	2E	6A	70	65	67	00	kaneki.jpeg.
01EE1040	0D	00	00	00	14	00	09	01	79	75	74	61	2E	6A	70	65	yuta.jpe
01EE1050	67	00	00	00	0E	00	00	00	14	00	0C	01	67	6F	6A	6F	g.....gojo
01EE1060	31	2D	32	2E	6A	70	65	67	0F	00	00	00	14	00	0A	01	l-2.jpeg.....
01EE1070	6E	61	67	75	6D	6F	2E	6A	70	67	00	00	D1	1F	00	00	nagumo.jpg..Ñ...
01EE1080	0C	00	04	02	66	69	6C	65	D2	1F	00	00	10	00	08	02	fileỒ.....
01EE1090	66	69	6C	65	54	68	75	32	A1	3F	00	00	18	00	10	02	fileThu2;?.....
01EE10A0	46	69	6C	65	74	65	6E	64	61	69	31	36	6B	79	74	75	Filetendail6kytu
01EE10B0	A2	3F	00	00	44	0F	01	02	31	00	00	00	00	00	00	00	¿?...D...l.....

Hình 7.4 Bản mẫu root Directory của EXT4

Để có xác định được vị trí bắt đầu của mỗi entry ta buộc phải xác định từ đầu, căn cứ vào trường độ dài của entry tại Offset 0x5 trong mỗi entry, ví dụ với entry đầu tiên ta có độ dài là 0xC = 12, trừ đi 4 bytes đầu vậy phần còn lại sẽ có kích thước là 8, trực tiếp nhảy qua 8 bytes này thì ta sẽ tới entry tiếp theo, cứ tiếp tục như vậy cho tới khi tới entry muốn tìm.

Sau khi nắm bắt được từng entry, giả sử ta muốn truy xuất entry: "0D 00 00 00 14 00 09 01 79 75 74 61 2E 6A 70 65 67 00 00 00" có tên file là "yuta.jpeg"

Phân tích entry này ta biết được rằng vị trí Inode mà entry này chỉ tới là 0x0D, mà mỗi Inode có kích thước 0x100 và Inode Table đầu tiên bắt đầu ở vị trí. 0x10E000. Vậy Inode thứ 0x0D sẽ bắt đầu tại vị trí 0x10EC00.

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0010EC00	B4	81	E8	03	08	1C	00	00	2C	AF	07	66	6A	AE	07	66	‘.è.....,~.fj@.f
0010EC10	8F	D2	E9	65	00	00	00	00	E8	03	01	00	10	00	00	00	.Ôée.....è.....
0010EC20	00	00	08	00	04	00	00	00	0A	F3	01	00	04	00	00	00	ó.....
0010EC30	00	00	00	00	00	00	00	00	02	00	00	00	02	82	00	00	,...
0010EC40	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010EC50	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010EC60	00	00	00	00	3B	53	91	96	00	00	00	00	00	00	00	00	;S`-.....
0010EC70	00	00	00	00	00	00	00	00	00	00	00	00	59	09	00	00	Y...
0010EC80	20	00	DE	5D	80	F3	A4	58	20	13	7B	3D	E8	72	5D	93	.P]€ó×X .{=èr]”
0010EC90	6A	AE	07	66	80	F3	A4	58	00	00	00	00	00	00	00	00	j@.f€ó×X.....
0010ECA0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010ECB0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010ECC0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010ECD0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010ECE0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010ECF0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Hình 7.5 Bản mẫu Inode do file “yuta.jpeg” chỉ tới

Phân tích I_block của Inode ta có:

- Extent Header: “0A F3 01 00 04 00 00 00 00 00 00 00”. Là Extent tree, độ sâu cây bằng 0, các node sau là leaf node.
- Extent leaf:”00 00 00 00 02 00 00 00 02 82 00 00”. Bắt đầu tại vị trí Block 0x8202, nằm trong 2 Block(liên tiếp).

Tại Block thứ 0x8202 tức Offset 0x8202000 ta sẽ có dữ liệu của Inode này:

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
08202000	FF	D8	FF	E0	00	10	4A	46	49	46	00	01	01	00	00	01	yvya..JFIF.....
08202010	00	01	00	00	FF	DB	00	84	00	0A	07	08	16	15	12	18	yũ.....
08202020	15	16	15	18	18	12	18	18	18	12	11	18	11	12	11	11	
08202030	12	11	12	18	18	19	19	18	18	18	1C	21	2E	25	1C		!.%.
08202040	1E	2C	21	18	18	26	38	26	2B	2F	31	35	35	35	1A	24	.,!...&8&+/1555.\$
08202050	3B	40	3B	33	3F	2E	34	35	31	01	0C	0C	0C	10	0F	10	;@;3?.451.....
08202060	1C	12	12	1E	34	21	24	24	34	34	34	34	34	34	34	34	4!\$\$44444444
08202070	34	34	34	34	31	34	34	31	31	34	34	34	34	34	34	34	4444144114444444

Hình 7.6 Dữ liệu do Inode của file “yuta.jpeg” chỉ tới

Giả sử nếu file “yuta.jpeg” bị xóa, ta sẽ lần lượt có nhận thấy được 2 điểm khác biệt rõ ràng là directory entry của file này sẽ bị xóa và sự thay đổi đáng kể trong inode của file này bao gồm việc hardlink count bị giảm, thời gian xóa (deleted time) bắt đầu đếm (khác 0) và extent tree của file này sẽ bị xóa.

Offset (h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
01EE1000	02	00	00	00	0C	00	01	02	2E	00	00	00	02	00	00	00	
01EE1010	0C	00	02	02	2E	2E	00	00	0B	00	00	00	14	00	0A	02	
01EE1020	6C	6F	73	74	2B	66	6F	75	6E	64	00	00	0C	00	00	00	lost+found.....
01EE1030	3C	00	0B	01	6B	61	6E	65	6B	69	2E	6A	70	65	67	00	<...kaneki.jpeg.
01EE1040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
01EE1050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
01EE1060	00	00	00	00	00	00	00	00	0F	00	00	00	14	00	0A	01	
01EE1070	6E	61	67	75	6D	6F	2E	6A	70	67	00	00	D1	1F	00	00	nagumo.jpg..Ñ...
01EE1080	0C	00	04	02	66	69	6C	65	D2	1F	00	00	10	00	08	02	fileỒ.....
01EE1090	66	69	6C	65	54	68	75	32	A1	3F	00	00	18	00	10	02	fileThu2;?.....
01EE10A0	46	69	6C	65	74	65	6E	64	61	69	31	36	6B	79	74	75	Filetendail6kytu
01EE10B0	A2	3F	00	00	0C	00	01	02	31	00	00	00	16	00	00	00	ø?.....1.....

Hình 7.7 Dữ liệu directory sau khi xóa file “yuta.jpeg” và “gojo1-2.jpeg” bị xóa

Offset (h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0010EC00	B4	81	E8	03	08	1C	00	00	2C	AF	07	66	5E	B4	0E	66	^..è.....,^..f^..f
0010EC10	8F	D2	E9	65	6F	3A	76	66	E8	03	00	00	00	00	00	00	.Ôéeo:vfè.....
0010EC20	00	00	08	00	04	00	00	00	0A	F3	01	00	04	00	00	00	ó.....
0010EC30	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010EC40	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010EC50	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010EC60	00	00	00	00	3B	53	91	96	00	00	00	00	00	00	00	00	;S^-.....
0010EC70	00	00	00	00	00	00	00	00	00	00	00	00	5B	CB	00	00	[È..
0010EC80	20	00	52	01	A4	83	CC	A5	20	13	7B	3D	E8	72	5D	93	.R.¼fİ¥ .{=èr]"
0010EC90	6A	AE	07	66	80	F3	A4	58	00	00	00	00	00	00	00	00	j@.fÉóX.....
0010ECA0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010ECB0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010ECC0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010ECD0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010ECE0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0010ECF0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Hình 7.8 Dữ liệu inode của file “yuta.jpeg” sau khi file yuta.jpeg đã bị xóa

Ta có thể nhận thấy rằng trong Inode của file này, hardlink count đã trở về 0 (tức là bị xóa) cùng với việc thời gian xóa cũng đã bắt đầu đếm. Không chỉ vậy, thông tin chứa vị trí dữ liệu của file này cũng đã bị xóa theo.

7.2.3.3 Phục hồi dữ liệu trên hệ thống tập tin EXT4

Do khi thực hiện xóa file, hệ thống đã trực tiếp xóa đi directory entry, việc này sẽ khiến ta không thể tìm được Entry này cùng với tên của nó nữa. Nên chúng ta sẽ bắt đầu với inode table, nơi lưu trữ thông tin Metadata của một file. Mặc dù khi xóa file, dữ liệu của inode đã không còn hoàn chỉnh nhưng nó sẽ để lại dấu hiệu rằng file đã bị xóa, từ đó ta có thể lợi dụng tính năng Journaling của EXT4 để tìm lại dữ liệu của file này.

Ví dụ ta có file “yuta.jpeg” trên, khi này ta đã không còn biết tên của nó nhưng thông qua vị trí bắt đầu của Inode table, ta biết được rằng đây là inode này là inode thứ 13, thuộc block 0x10E.

Sau đó ta tìm vị trí bắt đầu của Journal, thông qua thông tin trong Superblock tìm được vị trí bắt đầu này là Block thứ 0x30000, từ đó ta đến được Journal Superblock, khởi điểm của Journal

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
30000000	C0	3B	39	98	00	00	00	04	00	00	00	00	00	00	10	00	9~.....
30000010	00	00	20	00	00	00	00	01	00	00	00	17	00	00	00	75	u
30000020	00	00	00	00	00	00	00	00	00	00	00	12	00	00	00	00	
30000030	0F	0E	56	4E	3E	01	49	30	80	FA	43	0D	4A	46	75	B9	..VN>.IOúC.JFu¹
30000040	00	00	00	01	00	00	00	00	00	00	00	00	00	00	00	00	
30000050	04	00	00	00	00	00	00	00	00	00	00	75	00	00	00	00	u...

Hình 7.9 Dữ liệu Journal Superblock của EXT4

Từ đó ta có được vị trí Transaction đầu tiên là Block thứ 0x30001

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
30001000	C0	3B	39	98	00	00	00	01	00	00	00	02	00	00	01	0E	9~.....
30001010	00	00	00	00	00	00	00	00	74	A3	D2	D8	00	00	00	00	tŁ00....
30001020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	01	
30001030	00	00	00	0A	00	00	00	00	19	50	63	43	00	00	00	00	PcC....

Hình 7.10 Dữ liệu Journal Descriptor Block của Transaction đầu tiên

Phân tích Transaction này ta có được:

Journal Header (12 bytes đầu tiên) “C0 3B 39 98 00 00 00 01 00 00 00 02”:

Header này cho biết rằng đây là Block thuộc Journal (với signature “C0 3B 39 98”), Block này là Journal Descriptor Block (với flag là “00 00 00 01”)

Descriptor Tag đầu tiên (12 bytes sau Header): vị trí Block mà dữ liệu được lưu lại trong Journal là Block thứ 0x10E

12 bytes padding: vùng trống

Descriptor Tag tiếp theo (12 bytes tiếp theo): vị trí Block mà dữ liệu được lưu lại trong Journal là Block thứ 0x1, Tag này là Tag cuối cùng trong Journal Descriptor Block (phân tích cờ 0x0A tại Offset 0x30001033 có được 0x8 tức là end Tag)

Nhận thấy rằng có dữ liệu của Block thứ 0x10E tức Block có thể chứa dữ liệu của Inode cần phục hồi, do nó là Tag đầu tiên nên nó sẽ là Block đầu tiên sau

Journal Descriptor Block, kiểm tra bản Inode Table này, ta nhận thấy rằng thời điểm Transaction được thực hiện, dữ liệu của Inode cần phục hồi vẫn chưa được phân bổ

Vậy ta sẽ đến vị trí của Journal Descriptor Block tiếp theo và tuân tự như vậy cho đến khi tìm được Transaction chứa dữ liệu Block cần tìm, sau đó sẽ tìm vào trong Inode Table tìm kiếm Inode cần tìm.

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
3000F000	C0	3B	39	98	00	00	00	01	00	00	00	05	00	00	00	F1	À;9~.....ñ
3000F010	00	00	00	00	00	00	00	00	41	87	5D	D4	00	00	00	00	A+]Ô....
3000F020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	01	
3000F030	00	00	00	02	00	00	00	00	D8	19	B4	6D	00	00	01	0E	Ø.'m....
3000F040	00	00	00	02	00	00	00	00	00	9D	67	64	00	00	01	00	gd....
3000F050	00	00	00	02	00	00	00	00	67	73	06	9A	00	00	03	0B	gs.ă....
3000F060	00	00	00	02	00	00	00	00	8D	7D	7F	29	00	00	00	F0	}.)...đ
3000F070	00	00	00	02	00	00	00	00	E1	62	F4	80	00	00	1E	E7	ábô€...ç
3000F080	00	00	00	02	00	00	00	00	60	BD	24	CC	00	00	1E	E1	`%\$İ...á
3000F090	00	00	00	0A	00	00	00	00	DB	12	D9	82	00	00	00	00	Û.Û,....

Hình 7.11 Dữ liệu Journal Descriptor Block cần tìm

Nhận thấy có sự xuất hiện của dữ liệu Block thứ 0x10E, tiến hành kiểm tra nhận thấy có dữ liệu của Inode cần phục hồi

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
30012C00	B4	81	E8	03	08	1C	00	00	22	6A	EC	65	6A	AE	07	66	'.è....."jìej@.f
30012C10	8F	D2	E9	65	00	00	00	00	E8	03	01	00	10	00	00	00	.Ôée.....è.....
30012C20	00	00	08	00	04	00	00	00	0A	F3	01	00	04	00	00	00	ó.....
30012C30	00	00	00	00	00	00	00	00	02	00	00	00	02	82	00	00	,...
30012C40	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
30012C50	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
30012C60	00	00	00	00	3B	53	91	96	00	00	00	00	00	00	00	00	;S`-.....
30012C70	00	00	00	00	00	00	00	00	00	00	00	00	B0	95	00	00	°*...
30012C80	20	00	F4	2C	80	F3	A4	58	20	13	7B	3D	E0	A2	2F	6B	.ô,€ó×X .{=àç/k
30012C90	6A	AE	07	66	80	F3	A4	58	00	00	00	00	00	00	00	00	j@.f€ó×X.....
30012CA0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
30012CB0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
30012CC0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
30012CD0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
30012CE0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
30012CF0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

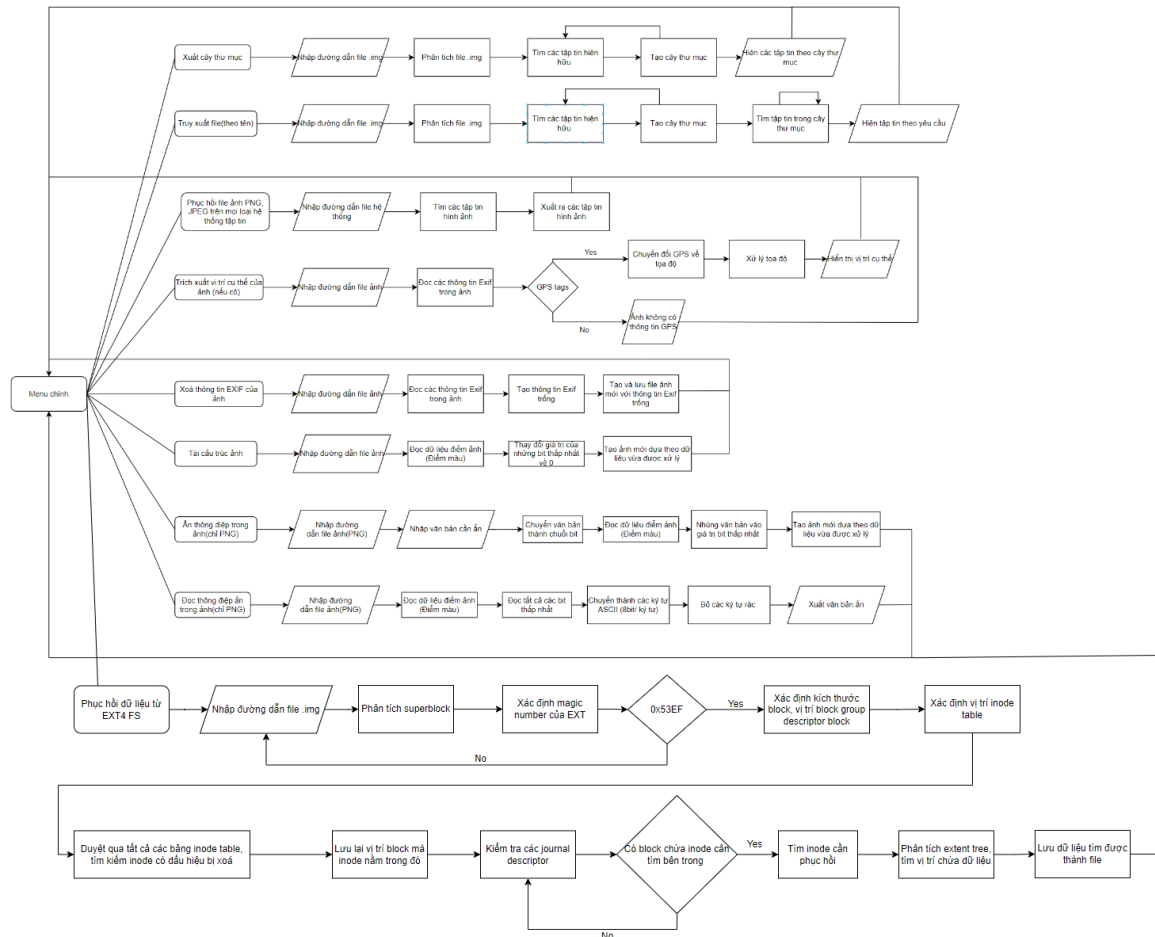
Hình 7.12 Dữ liệu Inode cần phục hồi tìm được trong Journal

Từ đó ta có dữ liệu Inode của file cần tìm, từ đây ta phân tích extent tree có được vị trí dữ liệu của file là Block thứ 0x8202, độ dài 2 Block. Từ đó ta có được dữ liệu của file là:

Chương 8. XÂY DỰNG ỨNG DỤNG

8.1 Ứng dụng chạy trên Windows

8.1.1 Sơ đồ hoạt động



Hình 8.1 Sơ đồ hoạt động của ứng dụng chạy trên Windows

8.1.2 Các tính năng của ứng dụng

8.1.2.1 Phân tích image file của EXT4

Người dùng sẽ được chọn tập tin image EXT4 mà mình muốn phân tích, hệ thống sẽ phân tích và trích xuất ra những thông tin quan trọng liên quan đến việc truy xuất và phục hồi dữ liệu(Block size, total Block, total Inode, vị trí bắt đầu của các Inode Table, ...). Các bước hoạt động của tính năng này như sau:

B1: Người dùng nhập đường dẫn đến file image của EXT4

B2: Dùng hàm của thư viện pytsk3 để phân tích Superblock

B3: Tìm Block size, total Block, total Inode

B4: Dùng thông tin vừa phân tích được từ Superblock tìm vị trí của các Block Group descriptor

B5: Phân tích các Block Group descriptor

B6: In ra Block size, total Block, total Inode

8.1.2.2 Xây dựng cây thư mục

Thông qua các thông tin mà ta đã thu thập được từ việc phân tích file image của EXT4, tiến hành tìm vị trí của thư mục root, từ đó phân tích nhằm xây dựng cây thư mục, cho người dùng cái nhìn tổng quan về các file cũng như thư mục hiện đang tồn tại trong hệ thống tập tin EXT4 này. Các bước hoạt động như sau:

B1: Lấy dữ liệu từ phân tích file image của EXT4

B2: Tìm vị trí Inode Table

B3: Tìm vị trí Inode thứ 2 (root Directory)

B4: Duyệt qua danh sách các Directory entry (bỏ qua “.” và “..”) nhằm xây dựng cây thư mục

B5: In ra cây thư mục cho người dùng

8.1.2.3 Truy xuất dữ liệu của file (theo tên)

Từ những thông tin có được từ phân tích EXT4 cùng với cây thư mục, tính năng này cho phép người dùng nhập tên để qua đó truy xuất dữ liệu thực tế của file này. Các bước hoạt động như sau:

B1: Người dùng nhập tên file muốn truy xuất

B2: Hệ thống sẽ tìm kiếm tên file trong cây thư mục. Nếu:

- Không tìm thấy tên file trong cây thư mục. Trở về B1
- Tìm thấy tên file trong cây thư mục. Đến B3

B3: Phân tích Directory có tên file người dùng muốn truy xuất để tìm kiếm Inode mà nó chỉ đến

B4: Phân tích Inode, tìm kiếm vị trí lưu trữ thông tin (phân tích I_Block)

B5: Đến vị trí chứa thông tin

B6: Tạo ra 1 file mới cùng tên và copy dữ liệu qua đó.

8.1.2.4 Phục hồi dữ liệu từ hệ thống tập tin EXT4

Người dùng sẽ chọn tập tin EXT4 chứa ảnh mình muốn phục hồi ảnh, hệ thống sẽ phân tích Superblock tập tin, xác định magic number để chắc chắn rằng đây là một hệ thống tập tin EXT4, sau đó hệ thống xác định kích thước Block và các thành phần quan trọng khác như vị trí Block Group Descriptor, từ đó có được vị trí của Inode Table. Duyệt qua toàn bộ các inode, ta xác định những inode mang dấu hiệu đã bị xóa. Với vị trí Block chứa Inode này, ta đem nó truy xuất qua dữ liệu của Journal để tìm lại bản ghi của Inode, từ đó tìm lại được dữ liệu của Inode, lưu xuống file cho người dùng. Các bước hoạt động chi tiết sẽ như sau:

B1: Người dùng nhập đường dẫn đến tập tin EXT4 muốn phục hồi

B2: Phân tích Superblock(bắt đầu tại Offset 1024), kiểm tra giá trị offset 0x38 lấy 2 bytes Little-endian có phải là 0x53EF hay không, nếu không đúng, hệ thống tập tin này không phải EXT4.

B3: Xác định giá trị kích thước Block tại offset 0x18 lấy 2 bytes Little-endian, được tính theo công thức $[2^{(10 + \text{giá trị của trường này})}]$

B4: Dựa vào kích thước Block, xác định vị trí Block Group Descriptor, do nó luôn nằm tại Block sau Superblock nên offset bắt đầu của nó sẽ là [vị trí bắt đầu của Superblock + kích thước Block]

B5: Phân tích các Block Group Descriptor, xác định vị trí Inode Table

B6: Phân tích Inode Table, xác định các Inode có dấu hiệu bị xóa(hardlink Count bằng 0, deleted time bắt đầu đếm cùng với dữ liệu extent tree đã bị xóa)

B7: Xác định Block mà Inode này đang nằm trong

B8: Tìm kiếm trong Journal về sự thay đổi của Block này, kiểm tra dữ liệu mà Journal ghi lại có dữ liệu gốc Inode mà ta tìm kiếm hay không

B9: Phân tích Inode gốc, xác định vị trí dữ liệu thông qua phân tích Extent Tree

B10: Đọc dữ liệu dựa theo vị trí vừa tìm được

B11: Dùng thư viện “filetype” để xác định dữ liệu đó là tập tin gì [17] và xuất ra tập tin hoàn chỉnh

8.1.2.5 Trích xuất dữ liệu hình ảnh PNG và JPEG/JPG từ một hệ thống tập tin không xác định

Người dùng sẽ chọn tập tin chứa ảnh mình muốn phục hồi ảnh, hệ thống sẽ yêu cầu người dùng chọn kích thước sector để thuận tiện cho việc tìm kiếm và trích xuất (mặc định là 512). Sau đó hệ thống sẽ trích xuất ra những ảnh PNG và JPEG theo thứ tự tìm được trong tập tin đó. Các bước hoạt động của tính năng này như sau:

B1: Chọn kích thước bước nhảy (256/512/2048/4096/ hoặc được nhập vào)

B2: Đọc file dưới dạng binary cho đến khi hết file với bước nhảy được chọn:

B3: Tìm chữ ký bắt đầu của tập tin ảnh JPEG và PNG (đã được giới thiệu ở mục 3.3 và 3.4)

B4: Đọc dữ liệu tới khi gặp chữ ký kết thúc tập tin ảnh JPEG và PNG

B5: Xuất ảnh ra

8.1.2.6 Trích xuất dữ liệu hình ảnh PNG và JPEG/JPG từ một hệ thống tập tin EXT4

Người dùng sẽ chọn tập tin .img mình muốn phục hồi. Hệ thống sẽ dựa theo cấu trúc EXT4 đã được nghiên cứu và đọc trực tiếp vào phần dữ liệu cho người dùng. Điều này giúp cải thiện tốc độ hơn thay vì tìm kiếm tuần tự. Các bước hoạt động của tính năng này như sau:

B1: Người dùng nhập đường dẫn đến file .img của EXT4 muốn phục hồi

B2: Dùng thư viện pytsk3 để phân tích, tìm vị trí các thành phần quan trọng như Inode Table, journal.

B3: Duyệt qua và tìm kiếm toàn bộ các Inode bị đánh dấu đã xóa(deleted time khác 0, hard link count bằng 0)

B4: Thông qua vị trí Inode tra ngược journal tìm Inode(thời điểm chưa bị xóa)

B5: Phân tích thông tin Inode. tìm kiếm vị trí dữ liệu.

B6: Kiểm tra signature của JPEG hoặc PNG, Nếu:

- Sai, bỏ qua Inode này
- Đúng, Xuất ảnh này ra thành file bên ngoài thư mục

8.1.2.7 Đọc dữ liệu GPS của dữ liệu hình ảnh nếu có

Người dùng sẽ chọn tập tin ảnh mà mình muốn trích xuất thông tin GPS. Hệ thống dùng thư viện có sẵn “Pillow” của python để đọc và dùng thư viện “geopy” để hiển thị vị trí cụ thể như Đất nước, Thành Phố, Quận,... nếu có. Các bước hoạt động của tính năng này như sau:

B1: Chọn ảnh

B2: Dùng thư viện “Pillow” để tìm tag GPS

B3: Đọc và lưu vị trí Kinh độ, Vĩ độ của ảnh (dạng giờ, phút, giây)

B4: Chuyển Kinh độ, Vĩ độ thành dạng tọa độ (X, Y)

B5: Dùng thư viện “Geopy” để đọc tọa độ (X, Y)

B6: Dùng định dạng dữ liệu từ điển (dictionary) để lưu thông tin đọc được từ bước trên

B7: Xuất ra vị trí cụ thể của bức ảnh

8.1.2.8 Xóa thông tin nhạy cảm trong dữ liệu hình ảnh nếu có

Người dùng sẽ chọn tập tin ảnh JPEG mà mình muốn xóa thông tin EXIF. Hệ thống dùng thư viện có sẵn “Pillow” của python để đọc tất cả các thông tin EXIF có trong ảnh. Sau đó tạo dữ liệu EXIF trống và lưu ảnh mới chỉ chứa dữ liệu EXIF trống này. Các bước hoạt động của tính năng này như sau:

B1: Chọn ảnh JPEG

B2: Dùng thư viện “Pillow” để tìm tất cả các EXIF tag có trong ảnh

B3: Tạo ra dữ liệu EXIF trống

B4: Lưu ảnh mới dựa theo EXIF trống đó

8.1.2.9 Xóa các thông tin bất thường bên trong dữ liệu hình ảnh (biến thể của CDR)

Người dùng sẽ chọn tập tin ảnh mà mình nghi ngờ. Hệ thống dùng thư viện có sẵn “Pillow” của python để đọc dữ liệu điểm màu trong ảnh, làm mờ những bit màu thấp nhất. Sau đó tạo một bức ảnh mới với dữ liệu là điểm màu vừa xử lý. Các bước hoạt động của tính năng này như sau:

B1: Chọn ảnh

B2: Thư viện “Pillow” chuyển ảnh về RGB

B3: Chuyển các bit thấp nhất của màu về 0

B4: Lưu dữ liệu RGB vừa được xử lý

B5: Lưu ảnh mới dựa theo dữ liệu đó

8.1.2.10 Ấn văn bản vào trong dữ liệu hình ảnh

Người dùng sẽ chọn tập tin ảnh mà mình muốn ấn dữ liệu văn bản. Nhập văn bản cần ấn. Hệ thống dùng thư viện có sẵn “Pillow” của python để đọc dữ liệu điểm màu trong ảnh. Nhúng nội dung văn bản được nhập vào trong những dữ liệu màu đó. Sau đó tạo một bức ảnh mới với dữ liệu là điểm màu vừa xử lý. Các bước hoạt động của tính năng này như sau:

B1: Chọn ảnh

B2: Thư viện “Pillow” chuyển ảnh về RGB

B3: Nhập văn bản cần ấn

B4: Văn bản sẽ được chuyển thành dãy nhị phân dựa trên base64

B5: Nhúng từng bit của dãy nhị phân trên vào từng bit thấp nhất của màu (mục 5.4.2)

B6: Lưu dữ liệu RGB vừa được xử lý

B7: Lưu ảnh mới dựa theo dữ liệu đó

8.1.2.11 Trích xuất văn bản ẩn bên trong dữ liệu hình ảnh

Người dùng sẽ chọn tập tin ảnh mà mình muốn trích xuất dữ liệu văn bản. Hệ thống dùng thư viện có sẵn “Pillow” của python để đọc dữ liệu điểm màu trong ảnh,. Sau đó tạo một bức ảnh mới với dữ liệu là điểm màu vừa xử lý. Các bước hoạt động của tính năng này như sau:

B1: Chọn ảnh

B2: Thư viện “Pillow” chuyển ảnh về RGB

B3: Hệ thống sẽ đọc và trích xuất tất cả các bit cuối của điểm màu (thao tác ngược lại ấn dữ liệu)

B4: Chuyển chuỗi bit đó thành chuỗi byte (1 byte ký tự = 8 bit)

B5: Lọc bỏ những ký tự nào không thuộc bảng mã ASCII

B6: Dùng base64 để giải mã thông điệp

B7: Hiển thị thông tin ẩn

8.1.3 Các kết quả đạt được

Trích xuất DLHA PNG và JPEG/JPG từ nhiều loại hệ thống tập tin:

Ứng dụng đã thành công trong việc trích xuất các file DLHA định dạng PNG và JPEG/JPG từ hệ thống tập tin không xác định, chứng minh tính linh hoạt và khả năng tương thích cao với nhiều môi trường khác nhau.

Đặc biệt, đối với hệ thống tập tin EXT4, ứng dụng không chỉ trích xuất được DLHA mà còn đảm bảo tính toàn vẹn của dữ liệu, giúp phục hồi chính xác các thông tin cần thiết.

Phục hồi và xử lý thông tin trên hệ thống tập tin EXT4:

Ứng dụng đã thực hiện trích xuất các thông tin quan trọng cho việc phục hồi trên hệ thống tập tin EXT4 một cách hiệu quả. Điều này bao gồm việc xác định và lấy lại các dữ liệu bị mất hoặc bị hỏng.

Thông tin về vị trí tạo của DLHA cũng đã được đọc và lưu trữ thành công, phục vụ cho các mục đích điều tra và phân tích dữ liệu.

Xử lý và bảo mật thông tin EXIF:

Ứng dụng đã xóa thành công các thông tin EXIF nếu có, giúp bảo vệ quyền riêng tư và bảo mật thông tin của người dùng. Đây là một bước quan trọng trong việc đảm bảo rằng các dữ liệu nhạy cảm không bị rò rỉ.

Xóa các thông tin bất thường bên trong DLHA:

Đề tài đã điều chỉnh phương pháp CDR truyền thống để có thể lọc bỏ các thông tin bất thường sâu bên trong dữ liệu hình ảnh, từ đó tái cấu trúc lại DLHA dựa theo hình ảnh thật sự mà người dùng thấy được.

Kỹ thuật giấu văn bản và trích xuất văn bản ẩn:

Ứng dụng đã triển khai thành công kỹ thuật giấu văn bản vào trong DLHA, cho phép bảo mật thông tin quan trọng một cách hiệu quả.

Quá trình trích xuất văn bản ẩn từ DLHA cũng đã được thực hiện thành công, chứng minh tính năng bảo mật và phục hồi dữ liệu ẩn của ứng dụng.

8.1.4 Giao diện ứng dụng

```
Menu:
1. Xuất cây thư mục
2. Truy xuất file (theo tên)
3. Phục hồi file ảnh PNG, JPEG trên mọi loại hệ thống tập tin
4. Trích xuất vị trí cụ thể của ảnh (nếu có)
5. Xoá thông tin EXIF của ảnh
6. Tải cấu trúc ảnh
7. Ẩn thông điệp trong ảnh(chỉ PNG)
8. Đọc thông điệp ẩn trong ảnh(chỉ PNG)
9. Phục hồi file từ file image EXT4
0. Thoát
Chọn một tùy chọn:
```

Hình 8.2 Menu chính cho ứng dụng windows

```
Chọn một tùy chọn: 1
Nhập đường dẫn đến file image EXT4: D:\Disks\30mar.img
Cây thư mục:
lost+found
kaneki.jpeg (Block: [33280], Length: 2)
yuta.jpeg (Block: [33282], Length: 2)
gojo1-2.jpeg (Block: [33008], Length: 19)
nagumo.jpg (Block: [33284], Length: 11)
file
  meme.jpg (Block: [34304], Length: 8)
  Android.docx (Block: [34816], Length: 624)
fileThu2
  suyenca.jpg (Block: [33047], Length: 22)
Filetendai16kytu
  pia1.jpg (Block: [33027], Length: 20)
  ext4 analysis.docx (Block: [35440], Length: 321)
1
  sasha.jpg (Block: [33792], Length: 11)
$OrphanFiles
```

Hình 8.3 Chức năng xuất cây thư mục

```
Chọn một tùy chọn: 2
Nhập đường dẫn đến file image EXT4: D:\Disks\30mar.img
Nhập tên tệp cần truy xuất: kaneki.jpeg

Đã trích xuất dữ liệu vào 'kaneki.jpeg'
```

Hình 8.4 Chức năng truy xuất file theo tên

```

Menu:
1. Xuất cây thư mục
2. Truy xuất file (theo tên)
3. Phục hồi file ảnh PNG, JPEG trên mọi loại hệ thống tập tin
4. Trích xuất vị trí cụ thể của ảnh (nếu có)
5. Xoá thông tin EXIF của ảnh
6. Tái cấu trúc ảnh
7. Ẩn thông điệp trong ảnh(chỉ PNG)
8. Đọc thông điệp ẩn trong ảnh(chỉ PNG)
0. Thoát
Chọn một tùy chọn: 3
Nhập đường dẫn đến tệp file image EXT4 cần phục hồi: D:\Disks\30mar.img

```

Hình 8.5 Chức năng phục hồi ảnh PNG, JPEG trên các loại hệ thống

```

Chọn một tùy chọn: 4
Nhập đường dẫn đến tệp JPEG: C:\Users\Admin\Documents\GitHub\VMQLTT\TESTJPGwithEXIF.JPG
Latitude: 10.800258333333334
Longitude: 106.64335555555556
Hẻm 4 Bàu Bàng, Phường 13, Quận Tân Bình, Thành phố Hồ Chí Minh, 72106, Việt Nam

```

Hình 8.6 Chức năng trích xuất vị trí cụ thể của ảnh(nếu có)

```

Chọn một tùy chọn: 5
Nhập đường dẫn đến tệp hình ảnh gốc: TESTJPGwithEXIF.JPG
Nhập đường dẫn lưu tệp hình ảnh đã xoá thông tin EXIF: TESTJPGwithoutEXIF.JPG
Đã xoá thông tin EXIF của ảnh và lưu tại 'TESTJPGwithoutEXIF.JPG'

```

Hình 8.7 Chức năng xoá thông tin nhạy cảm trong dữ liệu hình ảnh

```

Chọn một tùy chọn: 6
Nhập đường dẫn đến tệp hình ảnh gốc: TESTJPGwithEXIF.JPG
Nhập đường dẫn lưu tệp hình ảnh đã tái cấu trúc: TESTJPGwithEXIF_CDR.JPG
Đã tái cấu trúc ảnh và lưu tại 'TESTJPGwithEXIF_CDR.JPG'

```

Hình 8.8 Chức năng tái cấu trúc ảnh

```

Chọn một tùy chọn: 7
Nhập đường dẫn đến tệp hình ảnh gốc: images.png
Nhập thông điệp cần ẩn: Phát Phong Vinh Hoa
Nhập đường dẫn lưu tệp hình ảnh đã ẩn thông điệp: images_hidden_message.png
Đã ẩn thông điệp trong ảnh và lưu tại 'images_hidden_message.png'

```

Hình 8.9 Chức năng ẩn thông điệp trong ảnh(chỉ PNG)

```
Chọn một tùy chọn: 8
Nhập đường dẫn đến tệp hình ảnh: images_hidden_message.png
Thông điệp ẩn: Phát Phong Vinh Hoa
```

Hình 8.10 Chức năng đọc thông điệp ẩn trong ảnh(chỉ PNG)

```
Chọn một tùy chọn: 9
Nhập đường dẫn đến tệp file image EXT4 cần phục hồi: D:\Disks\testDel3+4\30mar_4.img
Journal start block: 196608
Deleted inode: [{'inode': 13}, {'inode': 14}]
Searching journal for inode 13 file
file recovery into file0.jpg
Searching journal for inode 14 file
file recovery into file1.jpg
Ấn nút Enter để tiếp tục
```

Hình 8.11 Phục hồi dữ liệu file từ file imgae của EXT4

8.1.5 Các khó khăn hiện hữu

Hệ thống chỉ dừng ở mức công cụ (tool): Hiện tại, hệ thống chỉ hoạt động như một công cụ hỗ trợ và chưa thể tự hoạt động một cách độc lập. Điều này đòi hỏi người dùng phải can thiệp thủ công nhiều trong quá trình sử dụng, làm giảm hiệu quả và tăng thời gian thao tác.

Chưa hoạt động một cách tự động: Hệ thống chưa được trang bị khả năng hoạt động tự động. Việc thiếu tính năng này làm hạn chế khả năng phản ứng nhanh và xử lý kịp thời trong các tình huống khẩn cấp, đồng thời làm giảm tính tiện lợi và hiệu quả của hệ thống.

Chỉ có thể hoạt động tốt nhất cho hệ thống quản lý tập tin EXT4: Hiện tại, hệ thống chỉ tối ưu hóa cho hệ thống tập tin EXT4. Điều này làm hạn chế phạm vi ứng dụng của hệ thống, vì hoạt động không quá hiệu quả trên các hệ thống tập tin khác như NTFS, FAT32, hoặc HFS+.

8.1.6 Hướng phát triển

Cải thiện tốc độ: Tốc độ xử lý của hệ thống cần được cải thiện để đáp ứng yêu cầu ngày càng cao của người dùng. Việc tối ưu hóa mã nguồn và sử dụng các thuật toán hiệu quả hơn sẽ giúp hệ thống hoạt động nhanh hơn và ổn định hơn.

Mở rộng ra cho các định dạng file khác: Hệ thống cần được mở rộng để hỗ trợ nhiều định dạng file khác nhau. Việc này sẽ tăng khả năng ứng dụng của hệ thống và giúp người dùng quản lý, khôi phục dữ liệu trên nhiều loại file khác nhau một cách hiệu quả hơn.

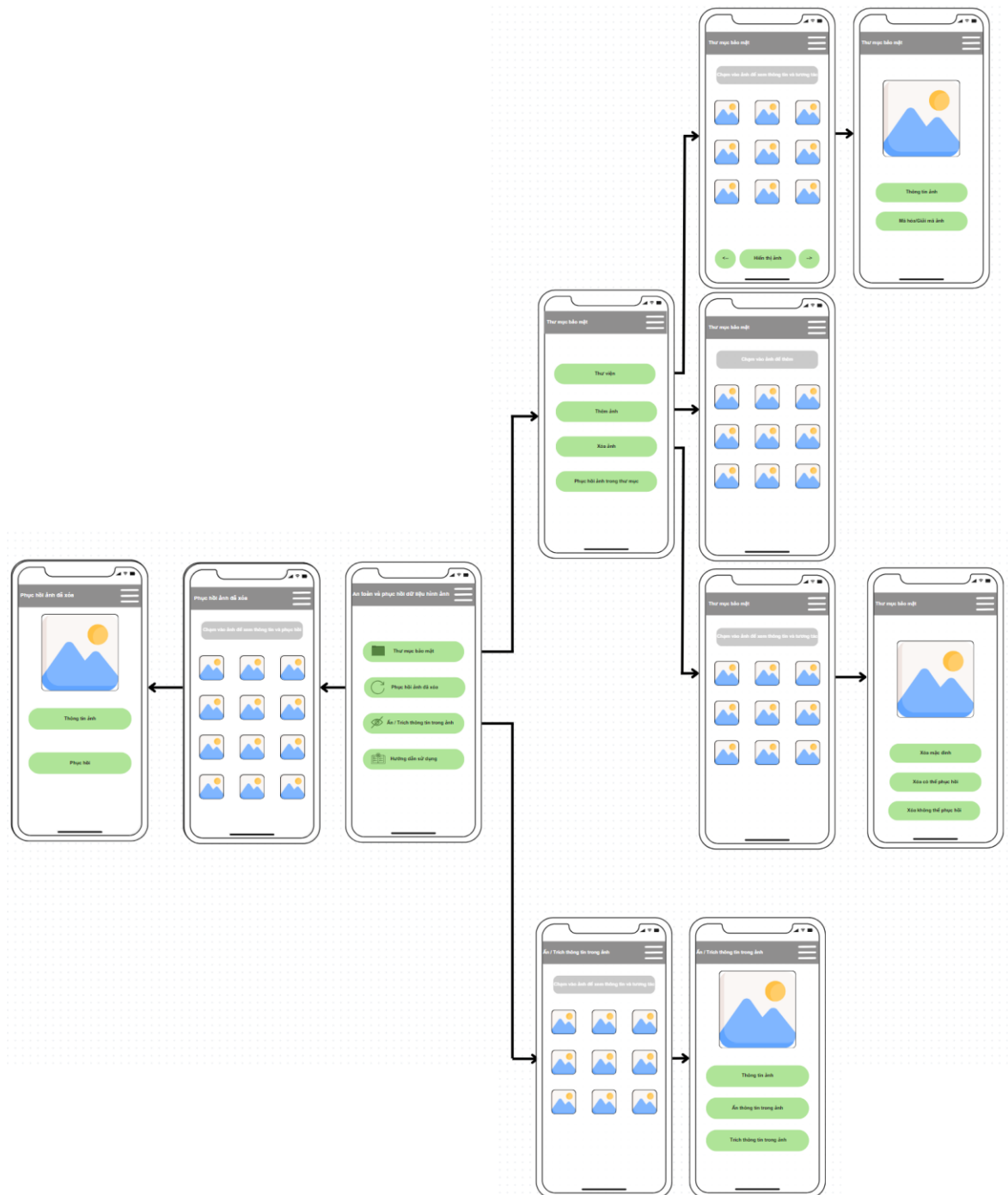
Sử dụng được cho các hệ thống tập tin khác ngoài EXT4: Để tăng tính linh hoạt và ứng dụng của hệ thống, cần phát triển để hệ thống có thể hoạt động trên các hệ thống tập tin khác như NTFS, FAT32, HFS+, v.v. Điều này sẽ giúp hệ thống tiếp cận được với nhiều người dùng hơn và tăng khả năng ứng dụng trong thực tế.

Phát triển thành ứng dụng: Hệ thống nên được phát triển thành một ứng dụng hoàn chỉnh với giao diện người dùng thân thiện và các tính năng tự động hóa. Việc này sẽ giúp người dùng dễ dàng sử dụng và tăng hiệu quả trong việc quản lý và khôi phục dữ liệu.

8.2 Ứng dụng chạy trên Android

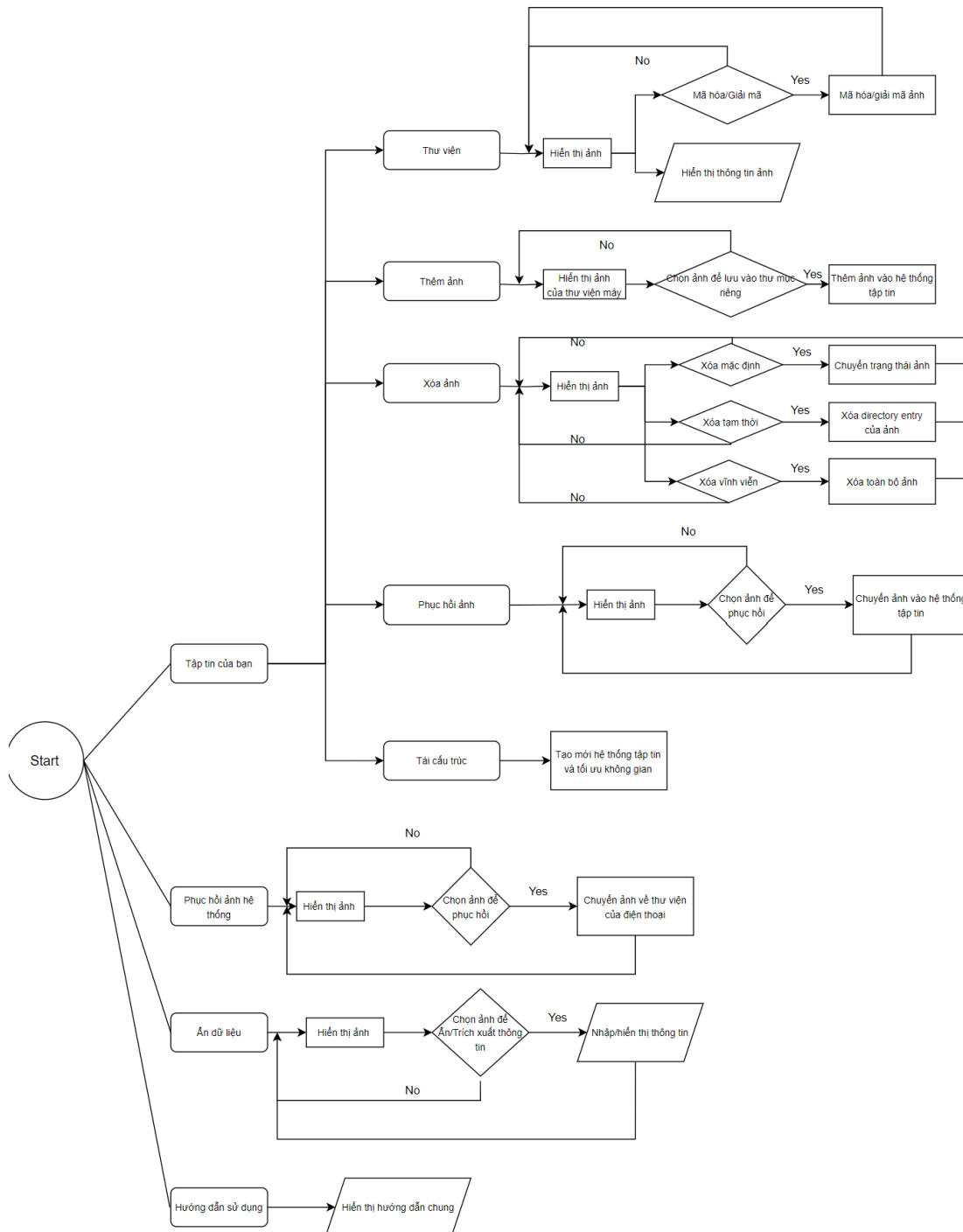
Hệ thống ứng dụng trên Android sẽ hỗ trợ các tính năng an toàn và phục hồi dữ liệu hình ảnh trên hệ thống tập tin có sẵn của thiết bị. Đồng thời để tăng thêm hiệu quả, App có hỗ trợ cho người dùng một hệ thống tập tin mới là PPHV (đề tài tự thiết kế và xây dựng) , kèm theo một loạt các chức năng hữu dụng trên hệ thống tập tin PPHV này

8.2.1 Thiết kế giao diện



Hình 8.12 Giao diện dự kiến của ứng dụng

8.2.2 Sơ đồ hoạt động



Hình 8.13 Flowchart cho ứng dụng Android

8.2.3 Các tính năng của ứng dụng

8.2.3.1 Tạo volume có định dạng chuyên biệt PPHV

Khi người dùng nhấn vào “Tập tin của bạn” chương trình sẽ kiểm tra phần Volume chứa dữ liệu của người dùng, đọc thông tin nếu đã tồn tại hoặc tạo mới nếu chưa tồn tại. Volume được đọc chứa dữ liệu bao gồm: vùng Header, vùng quản lý sector trống, vùng entry, vùng data.

B1: Tạo cấu trúc Header Volume rỗng

B2: Đọc dữ liệu thời gian, tên thiết bị khởi tạo, ...

B3: Tạo các vùng của hệ thống, lấy kích thước của hệ thống

B4: Cập nhật những dữ liệu có được vào Header Volume

8.2.3.2 Thêm ảnh vào PPHV

Chương trình truy cập vào Thư viện ảnh của máy, cho người dùng chọn ảnh để nhập vào volume. Thông tin ảnh sẽ được trích xuất để đưa vào vùng entry, phần dữ liệu ảnh sẽ được đưa vào vùng data.

B1: Chọn ảnh từ thư viện của điện thoại

B2: Ảnh được chọn sẽ được trích xuất thông tin và dữ liệu dưới dạng chuỗi byte

B3: Dữ liệu ảnh sẽ được lưu “append” vào cuối file, vị trí sector bắt đầu của dữ liệu sẽ được đưa vào phần thông tin ảnh.

B4: Tạo Directory entry với thông tin được trích xuất từ ảnh và sector bắt đầu của dữ liệu (ở bước 3)

B5: Lưu Directory entry vào vùng entry trên volume

B6: Cập nhật ngày giờ chỉnh sửa của volume

8.2.3.3 Hiển thị ảnh trong PPHV

Chương trình sẽ hiển thị các trang hiển thị ảnh được lưu ở Volume gồm 15 ảnh mỗi trang. Khi chọn ảnh, sẽ có nút cho người dùng chọn Mã hóa/ Giải mã ảnh và xem thông tin của ảnh.

B1: Chọn ảnh trong thư viện ứng dụng

B2: Đọc thông tin 15 ảnh từ volume, chuyển từ chuỗi byte sang bitmap và lưu vào danh sách.

B3: Hiển thị danh sách.

8.2.3.4 Tìm kiếm theo tháng năm

Tạo dialog cho người dùng chọn, khi người dùng chọn được tháng năm, chương trình sẽ lấy danh sách entry và sau đó lấy chuỗi byte thời gian tạo xử lý và đem so sánh để có được danh sách ảnh thỏa điều kiện.

B1: Chọn tháng năm cần tìm kiếm.

B2: Đọc thông tin ngày tháng tạo từ volume, xử lý chuỗi byte lấy được và đem so sánh với tháng năm người dùng tìm kiếm.

B3: Tạo lại danh sách mới thỏa điều kiện.

B4: Hiển thị ảnh lên thư viện.

8.2.3.5 Sắp xếp theo ngày

Đọc thông tin của toàn bộ entry và lấy được danh sách entry, sau đó trích xuất chuỗi byte ngày tạo, xử lý và so sánh dựa trên chuỗi byte lấy được, sau đó sử dụng thuật toán bubble sort để xếp lại danh sách entry.

B1: Chọn ngày tháng tăng giảm.

B2: Đọc thông tin ngày tháng tạo từ volume, xử lý chuỗi byte lấy được và sử dụng thuật toán Bubble sort để sắp xếp.

B3: Tạo lại danh sách dựa trên lựa chọn tăng giảm.

B4: Hiển thị ảnh lên thư viện.

8.2.3.6 Xóa dữ liệu trong PPHV

Xóa thông thường:

Xóa thông thường người dùng có thể phục hồi từ chức năng phục hồi dữ liệu trong volume. Xóa thông thường sẽ chuyển đổi tình trạng của ảnh từ tồn tại thành đã xóa để tạm thời ẩn ảnh khỏi phần hiển thị ảnh.

B1: Lấy thông tin phần entry của tấm ảnh cần xóa và truy cập đến entry đó

B2: Ghi đè byte State phần Directory entry của ảnh thành “1” để đánh dấu ảnh đã xóa tạm thời

B3: Cập nhật Directory entry vừa xóa tạm thời vào mảng entry hiển thị ảnh để ẩn

B4: Cập nhật ngày giờ chỉnh sửa của volume

Xóa có thể phục hồi:

Xóa có thể phục hồi người dùng cần có kiến thức về hệ thống tập tin và một số công cụ đặc biệt để có thể phục hồi dữ liệu của ảnh. Xóa toàn bộ entry chứa thông tin ảnh nhưng vẫn giữ lại phần data cho việc phục hồi ảnh sau này.

B1: Lấy thông tin phần entry của tấm ảnh cần xóa và truy cập đến entry đó

B2: Lấy thông tin sector bắt đầu và kích thước của ảnh ở entry

B3: Cập nhật thông tin sector bắt đầu và kích thước của ảnh vào vùng quản lý sector trống

B4: Xóa toàn bộ 256 Bytes phần entry của tấm ảnh

B5: Cập nhật ngày giờ chỉnh sửa của volume

Xóa vĩnh viễn:

Xóa vĩnh viễn người dùng sẽ không thể phục hồi dữ liệu. Xóa vĩnh viễn sẽ xóa toàn bộ entry và data của tấm ảnh tránh cho việc phục hồi lại.

B1: Lấy thông tin phần entry của tấm ảnh cần xóa và truy cập đến entry đó

B2: Lấy thông tin sector bắt đầu và kích thước của ảnh ở entry, truy cập đến vùng data của ảnh, tiến hành xóa toàn bộ data của ảnh

B3: Cập nhật thông tin sector bắt đầu và kích thước của ảnh vào vùng quản lý sector trống

B4: Xóa toàn bộ 256 bytes phần Directory entry của tấm ảnh

B5: Cập nhật ngày giờ chỉnh sửa của volume

8.2.3.7 Phục hồi dữ liệu của volume PPHV

Phục hồi dữ liệu của volume PPHV sẽ phục hồi các tấm ảnh bị xóa thông thường. Việc phục hồi sẽ chuyển đổi tình trạng của ảnh từ đã xóa thành tồn tại để khôi phục ảnh về phần hiển thị ảnh.

B1: Lấy thông tin phần entry của tấm ảnh cần phục hồi và truy cập đến entry đó

B2: Ghi đè byte State phần Directory entry của ảnh thành “0” để đánh dấu ảnh đã trở về mặc định

B3: Cập nhật Directory entry vừa khôi phục vào mảng entry hiển thị ảnh để khôi phục hiển thị

B4: Cập nhật ngày giờ chỉnh sửa của volume

8.2.3.8 Phục hồi dữ liệu trên điện thoại Android

Phục hồi dữ liệu trên điện thoại Android sẽ phục hồi các tấm ảnh trên điện thoại người dùng không hiển thị các tấm ảnh bị xóa trong volume. Để có thể phục hồi ta cần xin quyền truy cập bộ nhớ ngoài, và truy cập vào duyệt qua hết tất cả thư mục và kiểm tra nếu thỏa điều kiện phân mở rộng là ảnh để lấy được danh sách đường dẫn của các ảnh, dựa vào việc tạo ảnh mới vào thư viện qua đường dẫn lấy được.

B1: Lấy đường dẫn bộ nhớ ngoài của máy.

B2: Duyệt qua tất cả các thư mục trong đường dẫn, và kiểm tra các ảnh có trong thư mục và tạo danh sách lưu đường dẫn các ảnh.

B3: Hiển thị ảnh từ danh sách

B4: Chọn ảnh cần phục hồi và sau đó dựa trên ảnh lấy được từ đường dẫn tạo tấm ảnh mới trong thư viện.

8.2.3.9 Mã hóa / Giải mã dữ liệu trong PPHV

Các tấm ảnh sẽ được mã hóa thông tin và hiển thị là tấm ảnh khác mã chương trình đã cung cấp để thể hiện rằng tấm ảnh đã mã hóa. Sử dụng thuật toán AES để mã hoá / giải mã phần data của ảnh với key do người dùng nhập vào. Key sẽ được hash bằng thuật toán SHA-256 để so sánh khi giải mã.

B1: Chọn tấm ảnh cần mã hoá / giải mã

B2: Nhập mật khẩu dành cho mã hoá / giải mã ảnh

B3: Truy cập entry của ảnh

- Mã hoá: Đọc vị trí bắt đầu và kích thước phần data của ảnh
- Giải mã: Đọc vị trí bắt đầu và kích thước phần data của ảnh, mật khẩu đã hash của ảnh, so sánh với mật khẩu đã nhập để chấp thuận giải mã

B4: Tiến hành mã hoá / giải mã (nếu mật khẩu đúng). Truy cập đến phần data của ảnh, tiến hành mã hoá / giải mã toàn bộ data bằng AES

B5: Lưu mật khẩu đã hash (mã hoá) / Xóa mật khẩu đã lưu (giải mã) ở phần entry của ảnh

8.2.3.10 Tái cấu trúc PPHV

Khi sử dụng một thời gian dài sẽ không tránh được việc xóa những tập tin ảnh, với cơ chế lưu đặt biệt hỗ trợ cho việc phục hồi nhưng lại không tối ưu về mặt không gian. Làm cho kích thước hệ thống ngày càng lớn. Tính năng này với mục đích “dọn dẹp” những chỗ trống không dùng đến trong PPHV

B1: Đọc dữ liệu Header

B2: Tạo tập tin Volume PPHV mới (A) với thông tin Header ban đầu

B3: Đi tới sector 6

B4: Đọc và lưu thông tin Entry

B5: Tìm các sector chứa dữ liệu dựa trên thông tin Entry đọc được

B6: Đọc dữ liệu hình ảnh

B7: Viết Entry vào vùng Entry của (A)

B8: Viết dữ liệu hình ảnh vào cuối file

B9: Cập nhật lại vị trí lưu dữ liệu trong Entry

B10: Quay lại bước 4 tới khi hết vùng Entry

B11: Xóa tập tin PPHV cũ

8.2.4 Các kết quả đạt được

Chương trình đã đạt được các kết quả đáng kể trong việc cung cấp một hệ thống quản lý ảnh hiệu quả. Trước hết, ứng dụng có khả năng tạo và quản lý volume chứa tới 10240 tấm ảnh, với cấu trúc bao gồm vùng Header, quản lý sector trống, entry, và data. Điều này giúp tối ưu hóa việc lưu trữ và truy xuất dữ liệu ảnh.

Người dùng có thể dễ dàng thêm ảnh từ thư viện của máy vào volume, với quá trình trích xuất thông tin và lưu trữ dữ liệu ảnh được thực hiện tự động. Ứng dụng cũng cho phép hiển thị ảnh theo trang, với mỗi trang chứa 15 ảnh, cùng khả năng duyệt và xem thông tin chi tiết của từng ảnh.

Tính năng tìm kiếm theo tháng và năm tạo giúp người dùng nhanh chóng tìm thấy ảnh dựa trên thông tin ngày tháng. Ngoài ra, chức năng sắp xếp ảnh theo ngày tạo, với thứ tự tăng dần hoặc giảm dần, sử dụng thuật toán bubble sort, giúp tổ chức ảnh một cách khoa học.

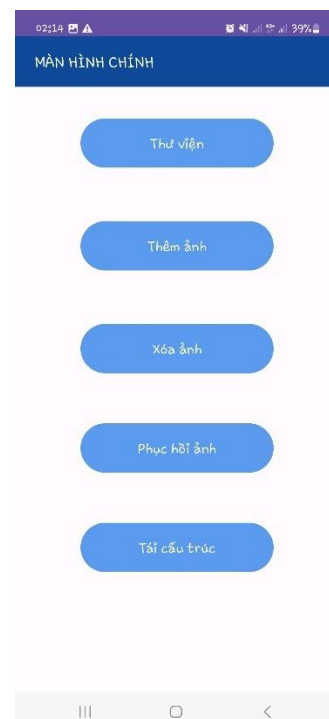
Chương trình cung cấp ba phương thức xóa dữ liệu: xóa thông thường để tạm thời ẩn ảnh, xóa có thể phục hồi để giữ lại dữ liệu ảnh cho việc phục hồi sau này, và xóa vĩnh viễn để loại bỏ hoàn toàn dữ liệu. Khả năng phục hồi dữ liệu của volume cho phép khôi phục các ảnh đã bị xóa tạm thời, đưa chúng trở lại trạng thái hiển thị.

Cuối cùng, chương trình hỗ trợ mã hóa và giải mã dữ liệu ảnh bằng thuật toán AES, bảo vệ ảnh bằng mật khẩu do người dùng nhập vào. Mật khẩu được hash bằng thuật toán SHA-256 để đảm bảo tính bảo mật cao khi giải mã, đáp ứng nhu cầu bảo vệ dữ liệu của người dùng.

8.2.5 Giao diện ứng dụng



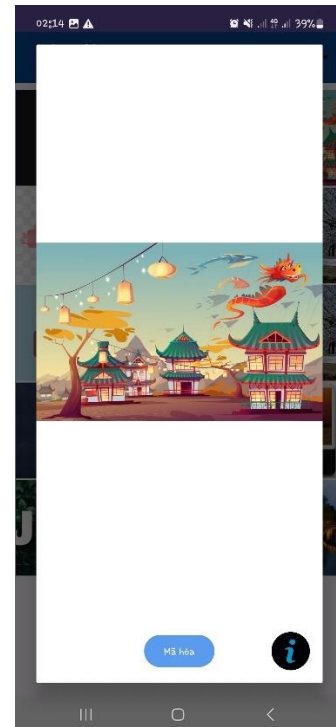
Hình 8.14 Màn hình ngoài của ứng dụng



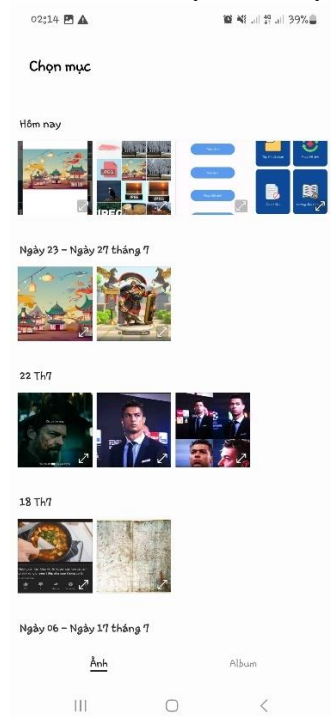
Hình 8.15 Màn hình chính của “Tập tin của bạn”



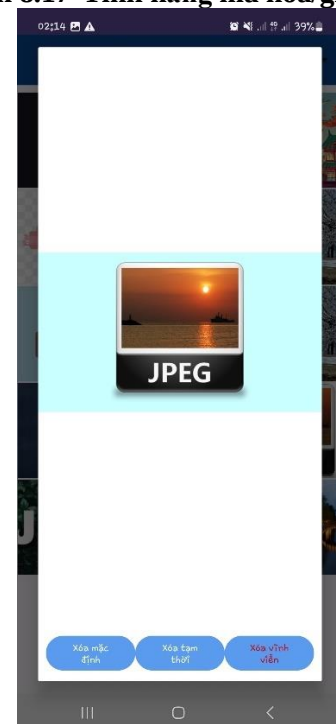
Hình 8.16 Thư viện hiển thị ảnh



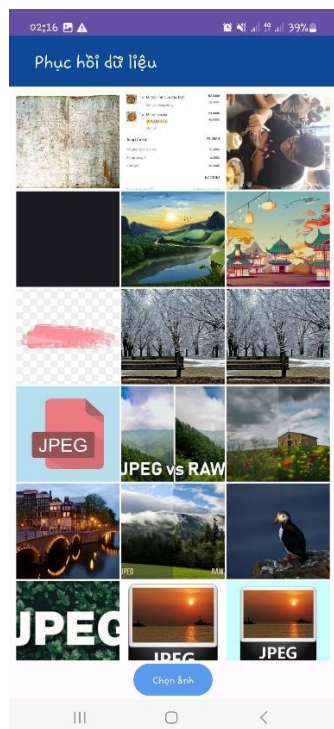
Hình 8.17 Tính năng mã hóa/giải mã



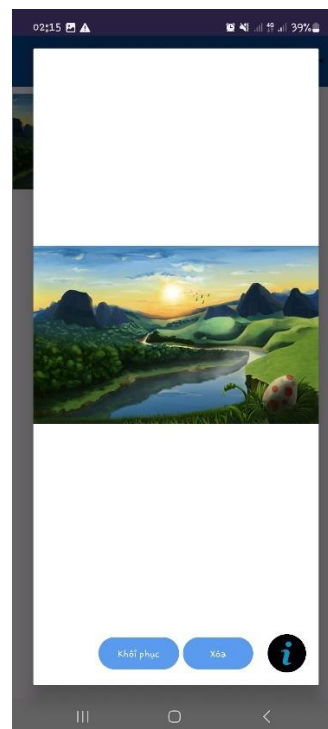
Hình 8.18 Tính năng thêm ảnh



Hình 8.19 Tính năng xóa ảnh



Hình 8.20 Phục hồi ảnh trên điện thoại android



Hình 8.21 Phục hồi ảnh trên hệ thống PPHV

8.2.6 Các khó khăn hiện hữu

Hiện tại ứng dụng Android chưa tối ưu với tất cả các điện thoại, nên có một số điện thoại không phổ biến thì ứng dụng Android có thể bị một số lỗi hiển thị hoặc không tương thích

Thời gian thực thi của tính năng phục hồi ảnh PNG và JPEG cho nhiều loại hệ thống quản lý tập tin của ổ đĩa khác nhau sẽ dựa vào độ lớn của bước nhảy do chưa thể phân tích được ổ đĩa đó thuộc loại hệ thống tập gì.

Chưa thể hiển thị được thông tin vị trí của những tấm ảnh (nếu ảnh đó có thông tin vị trí)

8.2.7 Hướng phát triển

Tích hợp tính năng hiển thị địa điểm chụp ảnh (nếu ảnh đó có thông tin vị trí) vào ứng dụng Android.

Tối ưu hóa ứng dụng và cải thiện tốc độ thực thi của ứng dụng

Cải thiện tính năng password bằng cách thêm dữ liệu “rác” cho key của AES hoặc tích hợp thêm một phương pháp bảo mật.

Phát triển cho phép hệ thống lưu mọi định dạng tập tin bao gồm docs, pdf, xlsx,...

Phát triển thêm tính năng “Thư mục” cho PPHV để người dùng có thể lưu nhưng bức ảnh chung nội dung ở cùng một nơi

Chương 9. TỔNG KẾT

9.1 Thực nghiệm an toàn dữ liệu

9.1.1 Thiết lập các biện pháp bảo vệ trên Android

Bảng 9.1 Bảng thử nghiệm mã hoá ảnh

Tình huống	Số lượng ảnh mã hóa	Số lượng ảnh được mã hóa thực tế	Thời gian thực thi trung bình
Mã hóa ảnh (<3MB)	100	100	~3 giây
Mã hóa những ảnh vừa (~3MB)	200	200	~8 giây
Mã hóa những ảnh nặng (>3MB)	200	200	~8 giây

Bảng 9.2 Bảng thử nghiệm lưu trữ ảnh

Tình huống	Số lượng được lưu trữ trong Volume dự kiến	Số lượng được lưu trữ thực tế	Ghi chú
Lưu những bức ảnh nặng (~10MB)	100 tấm	100 tấm	
Lưu những bức ảnh nhẹ (<1MB)	100 tấm	100 tấm	
Lưu tối đa ảnh	10240 tấm ảnh (dùng hết tất cả Entry)	10240 tấm ảnh	

9.1.2 Đánh giá hiệu quả của các biện pháp bảo vệ

Hệ thống về cơ bản có thể lưu trữ mọi tấm ảnh mà người dùng mong muốn lưu trữ.

Hệ thống có thể xử lý được các tấm ảnh ở nhiều kích thước khác nhau

Mã hóa AES rất hiệu quả về mặt hiệu năng trong việc mã hóa dù là ảnh nhỏ hay lớn. Bên cạnh đó AES còn mang tính bảo mật cao.

9.2 Thực nghiệm phục hồi dữ liệu

Bảng 9.3 Bảng thử nghiệm phục hồi ảnh

Tính năng	Tình huống	Số lượng	Thời gian thực thi trung bình
Phục hồi ảnh JPEG và PNG mọi loại ổ đĩa (bước nhảy 512byte)	Ảnh (10MB) trong ổ đĩa không xác định (1 allocation unit = 8192 byte)	10 ảnh Tổng dung lượng ảnh 100MB	~ 7 phút (thử nghiệm 20 lần)
Phục hồi ảnh JPEG và PNG mọi loại ổ đĩa (bước nhảy 8192 byte)	Ảnh (10MB) trong ổ đĩa không xác định (1 allocation unit = 8192 byte)	10 ảnh Tổng dung lượng ảnh 100MB	~26 giây (thử nghiệm 15 lần)
Truy xuất file trong EXT4	Docx (2.5MB) trong tập tin image EXT4	1000 file Tổng dung lượng ảnh 2500MB	~9 giây (thử nghiệm 12 lần)
Phục hồi ảnh JPEG và PNG mọi loại ổ đĩa	Ảnh (~1-3MB) trong ổ đĩa không phải EXT4	20 ảnh	~ 6 phút (thử nghiệm 20 lần)

(bước nhảy 512byte)		Tổng dung lượng ảnh 100MB	
Phục hồi dữ liệu trên hệ thống quản lý tập tin EXT4	10 file ảnh (<1 MB) và các file định dạng khác nhau (~2 – 5MB) trong tập tin image EXT4	Tổng dung lượng khoảng 100MB	~ 20 giây (thử nghiệm 4 lần)

9.3 Thử nghiệm các tính năng

9.3.1 Ứng dụng Android

Bảng 9.4 Bảng kiểm thử tính năng

Tính năng	Trường hợp test	Kết quả dự kiến	Kết quả thực tế	Ghi chú
Thêm ảnh	Thêm ảnh lấy từ thư viện cá nhân	Thêm đúng ảnh	Thêm đúng ảnh	
Xóa ảnh mặc định	Xóa ảnh mặc định trong hệ thống	Ảnh trong hệ thống chuyên biệt mất đi và hiển thị ở phần “phục hồi ảnh” của hệ thống	Ảnh trong hệ thống chuyên biệt mất đi và hiển thị ở phần “phục hồi ảnh” của hệ thống	
Xóa ảnh có thể phục hồi	Xóa ảnh có thể phục hồi trong hệ thống	Ảnh bị xóa đi và không hiển thị trong hệ thống, hiển thị dữ liệu ảnh ở khi đọc binary file hệ thống	Ảnh bị xóa đi và không hiển thị trong hệ thống, hiển thị dữ liệu ảnh ở khi đọc binary file hệ thống	
Xóa ảnh vĩnh	Xóa ảnh vĩnh	Ảnh bị xóa đi và	Ảnh bị xóa đi và	

viễn	viễn trong hệ thống	không thể tìm thấy ở bất kì đâu	không thể tìm thấy ở bất kì đâu	
Phục hồi ảnh trong hệ thống	Phục hồi ảnh trong hệ thống bị xóa mặc định	Khôi phục lại thông tin và hiển thị trong thư viện	Khôi phục lại thông tin và hiển thị trong thư viện	
Hiển thị ảnh	Hiển thị 1 ảnh	Hiển thị 1 ảnh trong hệ thống	Hiển thị 1 ảnh trong hệ thống	
Hiển thị ảnh	Hiển thị 20 ảnh	Hiển thị 15 ảnh 1 trang và 5 ảnh ở trang sau	Hiển thị 15 ảnh 1 trang và 5 ảnh ở trang sau	
Xem ảnh	Xem ảnh với nhiều kích thước khác nhau	Xem được ảnh ở tất cả kích thước khác nhau	Xem được ảnh ở tất cả kích thước khác nhau	
Phục hồi ảnh trong thiết bị Android	Phục hồi ảnh vừa xóa	Khôi phục được các tấm ảnh vừa xóa đó	Khôi phục được các tấm ảnh đã bị xóa trên điện thoại	

9.4 Tổng kết các kết quả nghiên cứu

Với mục tiêu đặt ra, đề tài đã thực hiện được :

Phần nghiên cứu cơ sở lý thuyết :

Tìm hiểu về các phương pháp an toàn dữ liệu.

Tìm hiểu về các phương pháp phục hồi dữ liệu.

Tìm hiểu về cấu trúc của loại ảnh thông dụng JPEG và PNG cũng như là thông tin siêu dữ liệu EXIF bên trong nó.

Tìm hiểu về cấu trúc của hệ thống lưu trữ EXT4 và có thể đọc trực tiếp một số thông tin giúp ích cho việc phục hồi dữ liệu cũng như truy xuất trực tiếp đến các tập tin mà không qua hệ điều hành.

Tìm hiểu về phương pháp ẩn/nhúng dữ liệu vào trong ảnh LSB.

Tìm hiểu về phương pháp tái cấu trúc tập tin CDR.

Qua việc tìm hiểu cơ sở lý thuyết trên đề tài mang lại kiến thức về cách tổ chức lưu trữ dữ liệu hệ thống và một số trường hợp mất mát dữ liệu thường gặp trong thực tế, từ đó đưa ra một số giải pháp để có thể phục hồi dữ liệu tương ứng với các trường hợp mất mát dữ liệu.

Phần thực nghiệm :

Đề tài đã xây dựng được 2 chương trình với các tính năng sau :

- Ứng dụng chạy trên Windows

Cứu được các tập tin JPEG và PNG trên nhiều hệ thống tập tin mà không cần biết định dạng.

Cứu được các loại tập tin khác nhau đã bị xóa hoàn toàn trên hệ thống tập tin EXT4.

Đọc và hiển thị thông tin vị trí của ảnh JPEG và PNG (nếu có).

Đọc và hiển thị các thông tin quan trọng của hệ thống quản lý tập tin EXT4.

Trích xuất các tập tin trong hệ thống quản lý tập tin EXT4.

Xóa thông tin metadata ra khỏi ảnh JPEG.

Thiết kế thuật toán CDR dành riêng cho ảnh để bỏ thông tin bất thường, thông tin ẩn bằng thuật toán LSB.

Sử dụng LSB để ẩn dữ liệu văn bản vào bên trong ảnh và trích xuất dữ liệu ẩn đó từ ảnh.

- Ứng dụng chạy trên Android

Thiết kế một hệ thống quản lý tập tin riêng biệt tích hợp các tính năng an toàn và phục hồi dữ liệu ảnh trên điện thoại Android. Các tính năng này bao gồm sao lưu, ẩn giấu, mã hóa, đảm bảo việc lưu trữ và truy xuất được thực hiện một cách an toàn và hiệu quả.

Phục hồi dữ liệu hình ảnh đã bị xóa hoặc không nằm trong thư viện.

9.5 Đánh giá và so sánh

Bảng 9.5 Bảng so sánh với các phần mềm khác

Tính năng	Ứng dụng của đề tài	Recuva	TestDisk	PhotoRec
Phục hồi file bị xóa	có	có	có	có
Trích xuất file trong hệ thống quản lý tập tin EXT4 không qua hệ điều hành	có	không	có	có
Tìm và phục hồi dữ liệu theo một số file thông dụng dựa vào cấu trúc của file	có	có	có	có
Cung cấp công cụ để lưu trữ, bảo mật và ẩn dữ liệu	có	không	không	không
Đọc và hiển thị thông tin metadata của ảnh	có	có	không	có
Tái cấu trúc ảnh để loại bỏ thông tin bất thường, mã độc được ẩn trong ảnh	có	không	không	không

Mặc dù nhóm không có thành viên nào có nền tảng về lập trình Android và phải bắt đầu từ việc học từ đầu, nhưng chúng em vẫn cố gắng hoàn thành các chức năng và tối ưu chương trình hết mức có thể. Chương trình này không chỉ đảm bảo an toàn cho người dùng mà còn hỗ trợ phục hồi dữ liệu ảnh trên điện thoại Android, mang lại sự riêng tư và tiện lợi trong việc quản lý thông tin ảnh cá nhân.

Ngoài việc tập trung vào phục hồi dữ liệu, ứng dụng còn mở rộng thêm nhiều tính năng để cung cấp một giải pháp toàn diện cho việc quản lý, bảo mật và bảo vệ dữ liệu. Các tính năng như trích xuất file từ hệ thống tập tin EXT4, lưu trữ và bảo mật dữ liệu, và hiển thị thông tin metadata của ảnh làm cho ứng dụng trở nên đa năng và hữu ích hơn cho người dùng. Với những khả năng này, ứng dụng của chúng tôi không chỉ đáp ứng nhu cầu phục hồi dữ liệu mà còn cung cấp các công cụ mạnh mẽ để quản lý và bảo vệ dữ liệu, mang lại giá trị cao cho người dùng trong việc sử dụng và quản lý thông tin cá nhân và doanh nghiệp.

PHỤ LỤC

PHỤ LỤC 1. Giới thiệu Định dạng File Ảnh .HEIC.....	132
---	-----

PHỤ LỤC 1. Giới thiệu Định dạng File Ảnh .HEIC

1.1 Định nghĩa và Lịch sử phát triển

Định dạng file ảnh .HEIC (High Efficiency Image Coding) là một định dạng file ảnh tiên tiến được phát triển bởi Moving Picture Experts Group (MPEG). Được giới thiệu lần đầu vào năm 2015 như một phần của tiêu chuẩn High Efficiency Video Coding (HEVC), .HEIC là một phương pháp nén hình ảnh hiệu quả cao, được thiết kế để thay thế định dạng ảnh truyền thống như JPEG.

1.2 Ưu điểm của Định dạng .HEIC

- **Dung lượng nhỏ hơn:** Ảnh .HEIC có thể nén hình ảnh với chất lượng tương đương JPEG nhưng dung lượng file giảm tới 50%. Điều này giúp tiết kiệm không gian lưu trữ trên thiết bị di động.
- **Chất lượng ảnh cao hơn:** .HEIC hỗ trợ hình ảnh với dải màu rộng hơn và độ sâu màu lớn hơn so với JPEG, mang lại chất lượng hình ảnh tốt hơn, đặc biệt trong các điều kiện ánh sáng phức tạp.
- **Hỗ trợ ảnh động:** Định dạng .HEIC có thể chứa nhiều ảnh trong một file, giúp lưu trữ các ảnh động (live photos) mà không cần sử dụng định dạng video.
- **Thông tin metadata phong phú:** .HEIC hỗ trợ lưu trữ nhiều thông tin metadata, bao gồm các thông số chụp ảnh, địa điểm, và nhiều thông tin khác, giúp quản lý và sắp xếp ảnh dễ dàng hơn.

1.3 Khả năng sử dụng trong tương lai

Với những ưu điểm vượt trội về dung lượng lưu trữ và chất lượng hình ảnh, định dạng .HEIC đang dần được các hãng sản xuất thiết bị di động và các nhà phát triển phần mềm ưu tiên sử dụng. Apple đã chính thức sử dụng định dạng .HEIC cho ảnh trên các thiết bị iOS từ phiên bản iOS 11. Các thiết bị Android cũng dần hỗ trợ định dạng này, và nhiều ứng dụng chỉnh sửa ảnh, xem ảnh đã bắt đầu tích hợp khả năng xử lý ảnh .HEIC.

Trong tương lai, khi nhu cầu lưu trữ ảnh chất lượng cao với dung lượng nhỏ trở nên cấp bách hơn, định dạng .HEIC nhiều khả năng sẽ trở thành tiêu chuẩn mới trên các thiết bị di động, thay thế cho định dạng JPEG đã tồn tại từ lâu.

1.4 Kết luận

Định dạng file ảnh .HEIC mang đến một bước tiến lớn trong công nghệ nén và lưu trữ hình ảnh, với nhiều ưu điểm vượt trội so với định dạng JPEG. Với sự hỗ trợ ngày càng rộng rãi từ các thiết bị và ứng dụng, .HEIC nhiều khả năng sẽ trở thành định dạng phổ biến trên điện thoại di động trong vài năm tới, giúp người dùng lưu trữ được nhiều hình ảnh hơn với chất lượng tốt hơn.

TÀI LIỆU THAM KHẢO

- [1] PLO, "Điện thoại Android chiếm đến 85% thị phần tại Việt Nam," PLO, 29-Jun-2024. [Trực tuyến]. Có sẵn: <https://plo.vn/ky-nguyen-so/dien-thoai-Android-chiem-den-85-thi-phan-tai-viet-nam-post709570.html#:~:text=Z%20Flip%204.-,Th%E1%BB%8B%20ph%E1%BA%A7n%20Android%20tr%C3%AA%20to%C3%A0n%20th%E1%BA%BF%20gi%E1%BB%9Bi%20l%C3%A0%20kho%E1%BA%A3ng%2071,%C4%91ang%20d%E1%BA%A7n%20chuy%E1%BB%83n%20sang%20iOS.>
- [2] "Màu đơn sắc," Wikipedia, 13-Jul-2024. [Trực tuyến]. Có sẵn: https://vi.wikipedia.org/wiki/M%C3%A0u_%C4%91%C6%A1n_s%E1%BA%AFc.
- [3] "EXIF file format," MIT Media Lab. [Trực tuyến]. Có sẵn: <https://www.media.mit.edu/pia/Research/deepview/EXIF.html#:~:text=Basic+ally%2C%20EXIF%20file%20format%20is,viewer%2FPhoto%20retouch%20software%20etc>.
- [4] "EXIFTool Tag Names," EXIFTool, n.d. [Trực tuyến]. Có sẵn: <https://EXIFtool.org/TagNames/EXIF.html>.
- [5] Camera & Imaging Products Association, "Exchangeable image file format for digital still cameras: EXIF Version 3.0," CIPA DC-008-Translation-2023-E.pdf, May 2023.
- [6] "PNG," Wikipedia, 16-Jul-2024. [Trực tuyến]. Có sẵn: <https://en.wikipedia.org/wiki/PNG>.
- [7] "PNG (Portable Network Graphics) Specification, Version 1.2," libpng.org, n.d. [Trực tuyến]. Có sẵn: <http://libpng.org/pub/png/spec/1.2/PNG-Structure.html>.
- [8] "W3C Recommendation: PNG (Portable Network Graphics) Specification,"

- W3C, 10-Nov-2003. [Trực tuyến]. Có sẵn: <https://www.w3.org/TR/png/>.
- [9] Thanh Niên, "Google thừa nhận xóa nhầm tài khoản quỹ hưu trí 135 tỉ USD," Thanh Niên. [Trực tuyến]. Có sẵn: <https://thanhnien.vn/google-thua-nhan-xoa-nham-tai-khoan-quy-huu-tri-135-ti-usd-185240531212910199.htm>.
- [10] FileGi, "Full Disk Encryption (FDE)," FileGi, n.d. [Trực tuyến]. Có sẵn: <https://filegi.com/tech-term/full-disk-encryption-fde-4078/>.
- [11] Wikipedia, "EXT4," [Trực tuyến]. Có sẵn: <https://en.wikipedia.org/wiki/Ext4>.
- [12] K. D. Fairbanks, "An analysis of EXT4 for digital forensics," Digital Investigation, Johns Hopkins University Applied Physics Laboratory, Laurel, MD 20723, USA.
- [13] The kernel development community, "EXT4 Data Structures and Algorithms," [Trực tuyến]. Có sẵn: <https://www.kernel.org/doc/html/latest/filesystems/EXT4/index.html>.
- [14] Srivathsa Dara, "Understanding EXT4 Disk Layout, Part 1," [Trực tuyến]. Có sẵn: <https://blogs.oracle.com/linux/post/understanding-EXT4-disk-layout-part-1>.
- [15] Srivathsa Dara, "Understanding EXT4 Disk Layout, Part 2," [Trực tuyến]. Có sẵn: <https://blogs.oracle.com/linux/post/understanding-EXT4-disk-layout-part-2>.
- [16] Cục An toàn thông tin, "Kỹ thuật giấu tin trong ảnh," Cục An toàn thông tin, 7 Tháng 6, 2022. [Trực tuyến]. Có sẵn: [https://antoanthongtin.gov.vn/giai-phap-khac/ky-thuat-giau-tin-trong-anh-109550#:~:text=Gi%E1%BA%A5u%20tin%20\(steganography\)%20l%C3%A0%20m%E1%BB%99t,qu%C3%A1%20tr%C3%ACnh%20%C4%91i%E1%BB%81u%20tra%20s%E1%BB%91](https://antoanthongtin.gov.vn/giai-phap-khac/ky-thuat-giau-tin-trong-anh-109550#:~:text=Gi%E1%BA%A5u%20tin%20(steganography)%20l%C3%A0%20m%E1%BB%99t,qu%C3%A1%20tr%C3%ACnh%20%C4%91i%E1%BB%81u%20tra%20s%E1%BB%91).
- [17] "filetype 1.2.0," PyPI. [Trực tuyến]. Có sẵn: <https://pypi.org/project/filetype/>